

Politechnika Wrocławska
Wydział Informatyki i Telekomunikacji

Kierunek: Informatyka Stosowana (IST)

Specjalność: Zastosowania Specjalistycznych Technologii Informatycznych (ZSTI)

PRACA DYPLOMOWA
MAGISTERSKA

Wykrywanie Fake News przy użyciu metod
zespołowego uczenia maszynowego

Vadym Liss

Opiekun pracy
Dr inż. Bernadetta Maleszka

Słowa kluczowe: uczenie maszynowe, Fake News, sztuczna inteligencja

WROCLAW 2025

STRESZCZENIE

Od wiosny 2022 roku uczenie maszynowe zyskało ogromną popularność jako narzędzie do rozwiązywania szerokiego zakresu problemów. Zarówno duże korporacje technologiczne, jak i małe zespoły oraz indywidualni użytkownicy dążą do integracji sztucznej inteligencji w swoich produktach i aplikacjach. Niezależnie od poziomu wiedzy technicznej, coraz więcej osób dostrzega rosnącą rolę SI w niemal każdej dziedzinie. Chociaż rozwój ten niesie wiele korzyści, wiąże się również z poważnymi zagrożeniami. Rozpowszechnienie sztucznej inteligencji ułatwiło tworzenie i dystrybucję treści dezinformacyjnych, co zagraża obiektywizmowi i może prowadzić do manipulacji informacjami. W jednym z najnowszych badań [4] autorzy zwracają uwagę na narastający problem fałszywych wiadomości w mediach społecznościowych. Pomimo postępów w dziedzinie sztucznej inteligencji, wykrywanie i przeciwdziałanie dezinformacji pozostaje znaczącym wyzwaniem. Niniejszy przegląd analizuje obecne metody oraz przyszłe kierunki rozwoju w walce z tym kluczowym problemem. W ramach tej pracy przeprowadzono przegląd istniejących metod i narzędzi służących do ograniczania treści dezinformacyjnych, ze szczególnym uwzględnieniem ich praktycznego zastosowania w XXI wieku. Zaproponowane rozwiązania opierają się na odpowiednim przetwarzaniu i modyfikacji danych wejściowych, co umożliwi lepszą klasyfikację treści. Praca ta obejmuje przegląd wybranych metod zespołowego uczenia maszynowego do wykrywania i klasyfikacji fałszywych wiadomości, ich implementację, przeprowadzenie testów oraz propozycję nowych rozwiązań. Ostatecznym wynikiem jest analiza porównawcza, uwzględniająca specyfikę różnych źródeł danych i zastosowanych metod.

ABSTRACT

Since the spring of 2022, machine learning has gained tremendous popularity as a tool for addressing a wide range of problems. Both large technology corporations and small teams or individuals are increasingly striving to integrate artificial intelligence into their products and applications. Regardless of their technical expertise, more and more people recognize the rapidly growing role of AI in nearly every domain. While this development brings numerous benefits, it also poses significant challenges. The widespread adoption of AI has facilitated the production and dissemination of disinformation, threatening objecti-

vity and potentially leading to information manipulation. In a recent study by Aimeur et al. (2023), the authors highlight the escalating issue of fake news on social media platforms. Despite progress in artificial intelligence, detecting and countering disinformation remains a formidable challenge. This review examines current methods and explores future directions for tackling this critical problem. This work provides a comprehensive overview of existing methods and tools designed to mitigate disinformation, emphasizing their practical applications in the 21st century. The proposed solutions rely on effective processing and modification of input data to improve content classification. In this paper, we review selected ensemble machine learning methods for detecting and classifying fake news, implement these techniques, conduct tests, and propose new solutions. The final result is a comparative analysis that accounts for the specificities of various data sources and the methods applied.

SPIS TREŚCI

1. Wstęp	4
2. Przegląd literatury	6
2.1. Korzyści zastosowania metod zespołowego uczenia maszynowego	6
2.2. Wprowadzenie do metod zespołowych	7
2.2.1. Wyjaśnienie idei za tymi technikami	7
2.2.2. Dlaczego są skuteczne w wykrywaniu fake newsów?	8
2.3. Kluczowe modele w analizie i detekcji dezinformacji	9
2.3.1. Regresja logistyczna	9
2.3.2. Random Forest	10
2.3.3. Boosting: AdaBoost i Gradient Boosting	10
2.3.4. XGBoost i LightGBM	11
2.3.5. Stacking	11
2.3.6. Sieci neuronowe	12
2.3.7. Voting Classifier	12
2.3.8. Modele z post-processingiem	12
2.3.9. CatBoost	13
2.4. Reprezentacje semantyczne tekstu	13
2.4.1. BERT (Bidirectional Encoder Representations from Transformers)	13
2.4.2. RoBERTa (Robustly Optimized BERT Approach)	14
2.4.3. Sentence-BERT (SBERT)	14
2.4.4. Porównanie wspomnianych reprezentacji	14
2.5. Techniki uczenia w różnych kontekstach danych	15
2.5.1. One-Shot Learning (OSL)	15
2.5.2. Few-Shot Learning (FSL)	15
2.5.3. Porównanie technik uczenia w różnych kontekstach danych	15
2.6. Implementacja przygotowania danych	16
2.7. Implementacja wybranych naukowych metod	17
2.7.1. Modele oparte na drzewach	18
2.7.2. Sieci neuronowe	19
2.7.3. Zespoły klasyfikatorów	21
2.7.4. Modele zoptymalizowane	33
2.8. Implementacja analizy wyników	34
3. Własne metody	37

3.1.	Pierwsza metoda (Metoda17)	37
3.1.1.	Pomysł na metodę	37
3.1.2.	Wejściowe parametry	37
3.1.3.	Wyjściowe parametry	38
3.1.4.	Algorytm	38
3.1.5.	Kod	39
3.1.6.	Schemat blokowy	40
3.1.7.	Właściwości analityczne	40
3.2.	Druga metoda (Metoda18)	41
3.2.1.	Pomysł na metodę	41
3.2.2.	Wejściowe parametry	41
3.2.3.	Wyjściowe parametry	42
3.2.4.	Algorytm	42
3.2.5.	Kod	43
3.2.6.	Schemat blokowy	44
3.2.7.	Właściwości analityczne	44
3.3.	Trzecia metoda (Metoda19)	45
3.3.1.	Pomysł na metodę	45
3.3.2.	Wejściowe dane	45
3.3.3.	Wyjściowe dane	46
3.3.4.	Algorytm	46
3.3.5.	Kod	47
3.3.6.	Schemat blokowy	48
3.3.7.	Właściwości analityczne	48
3.4.	Czwarta metoda (Metoda20)	49
3.4.1.	Pomysł na metodę	49
3.4.2.	Wejściowe dane	49
3.4.3.	Wyjściowe dane	50
3.4.4.	Algorytm	50
3.4.5.	Kod	51
3.4.6.	Schemat blokowy	52
3.4.7.	Właściwości analityczne	52
4.	Szczegóły implementacyjne	54
4.1.	Technologie	54
4.2.	Wymagania systemowe	55
4.2.1.	Sprzęt	55
4.2.2.	Oprogramowanie	55
4.3.	Architektura systemu	56
	Warstwa Wczytywania Danych i Preprocessingu	56
4.3.1.	Warstwa Trenowania Modeli	56
4.3.2.	Warstwa Ewaluacji	56

4.3.3.	Warstwa Wizualizacji	57
4.3.4.	Uruchomienie Treningu	57
4.3.5.	Generowanie Wykresów	57
5.	Badania eksperymentalne	58
5.1.	Wprowadzenie do badań	58
5.1.1.	Cel przeprowadzonych eksperymentów	58
5.1.2.	Uzasadnienie zastosowania wybranych metryk	58
5.2.	Metodologia	59
5.2.1.	Opis zestawów danych użytych w eksperymentach	59
5.2.2.	Szczegóły dotyczące analizy PR Curve	60
5.3.	Wyniki eksperymentalne dla bazowych metod	61
5.3.1.	Porównanie wyników bazowych rozwiązań według reprezentacji semantycznej tekstu, a mianowicie BERT	61
5.3.2.	Porównanie wyników bazowych rozwiązań według reprezentacji semantycznej tekstu, a mianowicie RoBERTa	69
5.3.3.	Porównanie wyników bazowych rozwiązań według reprezentacji semantycznej tekstu, a mianowicie Sentence-BERT (SBERT)	75
5.4.	Porównanie wyników najlepszych bazowych rozwiązań	81
5.4.1.	Proponowane testy statystyczne	81
5.4.2.	Wybrane najlepsze bazowe metody	82
5.4.3.	Reprezentacja wszystkich danych statystycznych	82
5.4.4.	Analiza wyników autorskich rozwiązań w zależności od architektury	88
	Podsumowanie	91
	Bibliografia	92
	Spis rysunków	94
	Dodatki	96

1. WSTĘP

Tematem niniejszej pracy magisterskiej jest *Wykrywanie Fake News przy użyciu metod zespołowego uczenia maszynowego*. Problem dezinformacji, w tym tak zwanego fake news, jest jednym z najpoważniejszych wyzwań współczesnego społeczeństwa informacyjnego.

Motywacją do podjęcia tego tematu wynika z realnych zagrożeń, jakie niesie za sobą dezinformacja oraz potrzeby opracowania skutecznych narzędzi do jej zwalczania. W ostatnich latach zaobserwowano liczne przypadki wpływu **fake news** na wyniki wyborów, podziały społeczne czy reakcje na kryzysy zdrowotne, takie jak pandemia COVID-19. Informacje nieprawdziwe, ale celowo rozpowszechniane, potrafią wywoływać panikę, dezinformować w kluczowych sprawach publicznych, a nawet prowadzić do eskalacji konfliktów międzynarodowych. W tym kontekście rola zaawansowanych technologii informatycznych w przeciwdziałaniu tym zjawiskom staje się kluczowa.

Dodatkową motywacją do podjęcia tematu jest rosnące zainteresowanie zastosowaniem sztucznej inteligencji w analizie danych tekstowych oraz ciągła potrzeba udoskonalania istniejących rozwiązań. Współczesne systemy detekcji **fake news** często bazują na standardowych algorytmach, które, mimo wysokiej popularności, nie zawsze radzą sobie w sytuacjach wymagających analizy danych o dużej zmienności, nierównowadze klas czy subtelnych różnicach semantycznych. Opracowanie nowych metod, które potrafią efektywnie radzić sobie z tymi wyzwaniami, może znacząco przyczynić się do zwiększenia skuteczności detekcji **fake news**.

W odpowiedzi na te wyzwania, dziedzina sztucznej inteligencji, a w szczególności uczenie maszynowe, oferuje zaawansowane narzędzia umożliwiające skuteczne wykrywanie i klasyfikację dezinformacji. Wśród tych narzędzi szczególną rolę odgrywają metody zespołowego **uczenia maszynowego** (*ensemble learning*), które dzięki integracji wyników wielu modeli bazowych, pozwalają na osiąganie wyższej dokładności predykcji i większej stabilności wyników. Techniki te, takie jak **bagging**, **boosting** czy **stacking**, zdobyły uznanie dzięki swojej skuteczności w rozwiązywaniu problemów nierównowagi klas oraz w zadaniach wymagających analizy dużych i zróżnicowanych zbiorów danych.

Celem niniejszej pracy jest opracowanie i implementacja autorskich metod zespołowego uczenia maszynowego dedykowanych do detekcji **fake news** oraz analiza ich skuteczności w porównaniu z tradycyjnymi podejściami. Praca koncentruje się na tworzeniu innowacyjnych rozwiązań, które wykraczają poza standardowe techniki. Szczególny nacisk położono na projektowanie algorytmów i użycie modeli które łączą w sobie no-

woczesne podejścia do przetwarzania języka naturalnego z zaawansowanymi technikami optymalizacji wyników klasyfikacji.

Główne założenia pracy obejmują:

1. Opracowanie nowych metod zespołowych wykorzystujących.
2. Analizę efektywności autorskich rozwiązań w kontekście kluczowych miar, takich jak **Log Loss**, **Accuracy** i **Execution Time**.
3. Praktyczne zastosowanie proponowanych metod na różnych zbiorach danych zawierających fałszywe i prawdziwe wiadomości, w tym na benchmarkowych zestawach, takich jak ISOT Fake News Dataset i innych.

Celem pracy jest również zbadanie, które podejścia w ramach metod zespołowych okazują się najskuteczniejsze w detekcji dezinformacji oraz jak innowacyjne rozwiązania mogą przyczynić się do dalszego zwiększenia efektywności tych systemów. Dzięki temu niniejsze badania mają na celu nie tylko teoretyczne rozwinięcie wiedzy w obszarze wykrywania **fake news**, ale również dostarczenie praktycznych narzędzi, które mogą być wykorzystywane zarówno w badaniach naukowych, jak i w zastosowaniach przemysłowych.

Wykrywanie fake news to złożone zadanie, które wymaga nie tylko zaawansowanych algorytmów, ale także odpowiedniego przetwarzania danych wejściowych. Przedstawione w tej pracy podejścia bazują się na połączeniu między tradycyjnymi technikami uczenia maszynowego a nowoczesnymi metodami przetwarzania języka naturalnego. W efekcie celem pracy jest stworzenie kompleksowego obrazu możliwości i ograniczeń współczesnych metod zespołowych w wykrywaniu **fake news**, a także wskazanie dalszych kierunków rozwoju w tym obszarze.

Niniejsze badania mają na celu nie tylko teoretyczne rozwinięcie wiedzy w obszarze wykrywania fake news, ale również dostarczenie praktycznych narzędzi, które mogą być wykorzystywane zarówno w badaniach naukowych, jak i w zastosowaniach komercyjnych.

Praca ta stanowi próbę odpowiedzi na pytanie, które podejścia w ramach metod zespołowych okazują się najskuteczniejsze w detekcji dezinformacji, oraz jakie innowacyjne rozwiązania mogą jeszcze bardziej zwiększyć efektywność takich systemów. Dzięki temu niniejsze badania przyczynią się do rozwinięcia narzędzi, które mogą być wykorzystywane zarówno w badaniach naukowych, jak i w praktycznych zastosowaniach w walce z dezinformacją.

2. PRZEGLĄD LITERATURY

2.1. KORZYŚCI ZASTOSOWANIA METOD ZESPOŁOWEGO UCZENIA MASZYNOWEGO

Jak wspomniano w podanym artykule [8], **ensemble learning** to metoda uczenia maszynowego, która polega na łączeniu wyników wielu modeli w celu poprawy dokładności predykcji. Główne klasy metod ensemble learning obejmują **bagging**, **boosting**, **stacking** oraz inne. Każda z tych technik znajduje zastosowanie w rozwiązaniu problemów nierównowagi klas (class imbalance), gdzie liczba próbek jednej klasy jest znacznie większa od liczby próbek drugiej klasy. Takie sytuacje mogą prowadzić do stronniczości modeli maszynowych na korzyść klasy większościowej, co prowadzi do błędnej klasyfikacji próbek klasy mniejszościowej. Bardziej szczegółowy opis wspomnianych powyżej metod zostanie omówiony w następnym podrozdziale.

W artykule wykazano, że metody ensemble są szczególnie skuteczne w przypadku danych nierównoważnych, takich jak:

- Wykrywanie oszustw (fraud detection),
- Analiza sentymentu,
- Diagnostyka medyczna,
- Prognozowanie awarii urządzeń.

W porównaniu do pojedynczych modeli, metody zespołowe osiągają lepsze wyniki zarówno w metrykach klasyfikacyjnych, jak i ogólnej dokładności. Ponadto, odpowiednio zaprojektowane metody ensemble mogą redukować problem nadmiernego dopasowania (overfitting). Chcielibyśmy wyróżnić następujące **zalety** stosowania metod zespołowego uczenia maszynowego:

- **Zwiększona odporność na błędy:** Ensemble learning zmniejsza ryzyko błędów spowodowanych nadmierną wariancją lub stronniczością pojedynczych modeli, co zapewnia bardziej stabilne i wiarygodne wyniki.
- **Integracja różnorodnych modeli:** Możliwość łączenia różnych typów modeli pozwala na wykorzystanie ich indywidualnych mocnych stron, co zwiększa skuteczność w zadaniach klasyfikacyjnych i regresyjnych.
- **Skuteczność w rozwiązywaniu problemów nierównowagi klas:** Ensemble learning wykazuje lepsze wyniki w zadaniach z nierównomiernym rozkładem klas, zwłaszcza

gdzie jest stosowany w połączeniu z metodami wzbogacania danych, takimi jak SMOTE czy generatywne sieci przeciwstawne (GANs).

2.2. WPROWADZENIE DO METOD ZESPOŁOWYCH

2.2.1. Wyjaśnienie idei za tymi technikami

Rozszerzając wiedzę na temat technik metod zespołowego uczenia maszynowego, szeroko opisanych w artykule [5], można zauważyć ich uniwersalność i znaczenie w różnych zastosowaniach. Jak wskazano w artykule, sukces tych metod wynika z ich zdolności do redukowania błędów systematycznych (bias) i wariancji, efektywnego wykorzystania zasobów obliczeniowych oraz umiejętności reprezentowania bardziej złożonych zależności w danych. W praktyce metody ensemble znajdują zastosowanie w wielu dziedzinach, takich jak diagnostyka medyczna, analiza obrazów, wykrywanie anomalii czy przewidywanie awarii urządzeń. Autorzy podkreślają również ich skuteczność w rozwiązywaniu problemów nierównowagi klas, co jest szczególnie istotne w zadaniach, gdzie jedna klasa danych jest znacząco mniej reprezentowana.

Dzięki integracji z modelami głębokiego uczenia, metody zespołowe łączą zdolność do hierarchicznego uczenia reprezentacji z większą precyzją predykcji. Jednocześnie rozwój tych modeli wymaga rozwiązywania wyzwań, takich jak wysoki koszt obliczeniowy oraz konieczność zapewnienia różnorodności pomiędzy modelami bazowymi, co pozwala na ich skuteczne wykorzystanie w praktyce.

Bagging (Bootstrap Aggregating)

Bagging, znany także jako bootstrap aggregating, to metoda redukująca wariancję modeli bazowych poprzez trenowanie ich na różnych, losowych próbkach danych wybranych z zamianą (bootstrap). Jak wspomniano w artykule, bagging polega na tworzeniu wielu niezależnych modeli bazowych, których wyniki są następnie łączone poprzez głosowanie większościowe (w przypadku klasyfikacji) lub uśrednianie (w przypadku regresji).

Przykładem zastosowania baggingu jest **Random Forest**, w którym dodatkowo losowo wybierane są podzbiory cech na każdym etapie budowania drzewa decyzyjnego. W ten sposób zwiększa się różnorodność modeli bazowych, zmniejszając jednocześnie ryzyko nadmiernego dopasowania i poprawiając stabilność wyników.

Boosting

Boosting to metoda polegająca na iteracyjnym trenowaniu modeli, gdzie każdy kolejny model skupia się na przykładach trudnych do sklasyfikowania przez poprzedników. W artykule podkreślono, że proces ten umożliwia tworzenie silnych klasyfikatorów zdolnych do radzenia sobie z bardziej złożonymi problemami.

Przykładem jest **AdaBoost**, który nadaje większe wagi błędnie sklasyfikowanym przykładom w kolejnych iteracjach, minimalizując funkcję błędu eksponencjalnego. Natomiast **Gradient Boosting** rozszerza to podejście, optymalizując dowolną różniczkowalną funkcję błędu. Boosting efektywnie redukuje błąd systematyczny (bias) i wariancję, co prowadzi do lepszych wyników.

Stacking (Stacked Generalization)

Stacking, czyli uogólnione zestawianie (stacked generalization), to metoda zespołowa, w której różne modele bazowe są łączone za pomocą meta-modelu. Meta-model uczy się, jak najlepiej łączyć prognozy modeli bazowych, korzystając z ich wyników jako danych wejściowych.

Jak podano w artykule, stacking pozwala na łączenie różnych typów modeli (np. drzewa decyzyjne, sieci neuronowe, SVM), co czyni go bardziej elastycznym i zdolnym do generalizacji. W literaturze metoda ta jest również stosowana w złożonych problemach, takich jak klasyfikacja obrazów czy przewidywanie trendów.

Negative Correlation Learning (NCL)

W technice **Negative Correlation Learning** (NCL) dąży się do zwiększenia różnorodności wśród modeli bazowych poprzez wprowadzenie funkcji kosztu, która karze zbyt podobne prognozy między modelami. W artykule podkreślono, że metoda ta jest szczególnie skuteczna w problemach wymagających wysokiej zdolności do generalizacji.

Przykładem zastosowania NCL są zadania regresyjne i klasyfikacyjne, w których różnorodność modeli poprawia ogólną wydajność zespołu.

Homogeniczne i heterogeniczne zespoły

- **Homogeniczne zespoły**: Wykorzystują jeden typ klasyfikatorów (np. drzewa decyzyjne w Random Forest). Kluczowym wyzwaniem jest wprowadzenie różnorodności, np. poprzez losowe próbkowanie danych treningowych lub cech.
- **Heterogeniczne zespoły**: Łączą różne typy modeli (np. sieci neuronowe, SVM, regresję logistyczną). Są bardziej elastyczne i zdolne do uchwycenia różnych cech danych, co czyni je użytecznymi w złożonych problemach, takich jak analiza tekstu czy przewidywanie awarii.

2.2.2. Dlaczego są skuteczne w wykrywaniu fake newsów?

Bagging (Bootstrap Aggregating)

Bagging zwiększa stabilność modeli w sytuacjach, gdy dane są niestabilne lub różnorodne – co jest powszechne w zadaniach wykrywania fake news. Przykładem jest Random Forest, który dzięki losowemu wyborowi podzbiorów cech i danych jest odporny

na przetrenowanie, co pozwala mu skutecznie radzić sobie z różnymi rodzajami tekstów. Redukcja wariancji umożliwia bardziej spójne klasyfikowanie nawet w przypadku danych zawierających subtelne różnice.

Boosting

Boosting koncentruje się na trudno klasyfikowalnych przykładach, które są częste w zbiorach danych dotyczących fake news. Fałszywe informacje często charakteryzują się subtelnymi manipulacjami w języku, co wymaga modeli zdolnych do uchwycenia takich szczegółów. Boosting, np. w formie Gradient Boostingu lub XGBoost, jest w stanie modelować złożone zależności między cechami, co prowadzi do wyższej dokładności w identyfikowaniu fałszywych wiadomości.

Stacking (Stacked Generalization)

Stacking umożliwia łączenie różnych metod analizy tekstu, takich jak modele oparte na n-gramach, TF-IDF, czy głębokie sieci neuronowe. Dzięki temu można wykorzystać mocne strony każdego z tych podejść, jednocześnie minimalizując ich słabości. W zadaniach wykrywania fake news stacking pozwala połączyć modele bazujące na różnych źródłach danych (np. tekst, metadane, analiza kontekstu), co daje bardziej kompleksowy obraz problemu.

Negative Correlation Learning (NCL)

NCL zwiększa różnorodność modeli bazowych, co sprawdza się w wykrywaniu fake news, gdzie dane są niejednorodne i zawierają różnorodne wzorce. Dzięki promowaniu różnorodności w predykcjach, modele uczą się szerokiego zakresu cech, co prowadzi do lepszego uogólnienia wyników na danych testowych.

Homogeniczne i heterogeniczne zespoły

Homogeniczne zespoły, dobrze radzą sobie z dużą liczbą cech tekstowych i ich interakcjami. Z kolei heterogeniczne zespoły, łączące różne typy modeli, pozwalają lepiej uchwycić różnorodne cechy danych fake news, takie jak manipulacyjny język, brak źródeł czy określone wzorce emocjonalne w tekstach.

2.3. KLUCZOWE MODELE W ANALIZIE I DETEKCJI DEZINFORMACJI

2.3.1. Regresja logistyczna

Regresja logistyczna jest modelem statystycznym stosowanym głównie w klasyfikacji binarnej. Opiera się na wykorzystaniu funkcji sigmoidalnej, która przekształca wyniki

liniowej kombinacji cech wejściowych w wartość prawdopodobieństwa, umożliwiając klasyfikację na dwie klasy. Model ten jest szczególnie ceniony za prostotę i interpretowalność – współczynniki regresji wskazują na wpływ poszczególnych cech na wynik predykcji.

W praktyce regresja logistyczna jest wykorzystywana w analizach, takich jak:

- **Prognozowanie ryzyka chorób** w medycynie,
- **Segmentacja klientów** w marketingu,
- **Ocena ryzyka kredytowego** w finansach.

Największym ograniczeniem regresji logistycznej jest jej niezdolność do modelowania złożonych, nieliniowych zależności oraz tendencja do faworyzowania klasy dominującej w przypadku nierównoważnych danych.

2.3.2. Random Forest

Random Forest to rozwinięcie drzew decyzyjnych, które wprowadza element losowości, aby zwiększyć różnorodność modeli bazowych i zmniejszyć ich podatność na przeuczenie. Model ten wykorzystuje:

- **Bagging** (Bootstrap Aggregating): losowe próbkowanie z zamianą, aby stworzyć wiele zbiorów danych treningowych,
- **Losowy wybór cech** na każdym etapie budowy drzewa, co zmniejsza korelację między drzewami.

Według przeprowadzonej analizy zastosowań, Random Forest jest przydatny w zadaniach takich jak:

- **Diagnoza medyczna**, gdzie umożliwia ocenę ważności cech diagnostycznych,
- **Wykrywanie intruzji** w systemach bezpieczeństwa,
- **Analiza danych obrazowych**, dzięki możliwości przetwarzania dużych zbiorów w sposób równoległy.

Pomimo swoich zalet, Random Forest jest trudniejszy w interpretacji w porównaniu do pojedynczych drzew decyzyjnych.

2.3.3. Boosting: AdaBoost i Gradient Boosting

Boosting to rodzina algorytmów, które iteracyjnie wzmacniają słabe klasyfikatory. Każda iteracja skupia się na poprawie błędów poprzednich, co prowadzi do bardziej precyzyjnego modelu.

- **AdaBoost**: Używa prostych modeli i nadaje większe wagi przykładom błędnie sklasyfikowanym. Znajduje zastosowanie w:
 - **Wykrywaniu intruzji** w systemach komputerowych,

- **Analizie tekstu,**
- **Prognozowaniu upadłości firm.**
- **Gradient Boosting:** Wykorzystuje optymalizację gradientową, aby iteracyjnie korygować błędy. Modeluje złożone zależności nieliniowe, co sprawia, że jest bardzo dokładny, ale wymaga starannego strojenia hiperparametrów. Zastosowania obejmują:
 - **Diagnozę medyczną,**
 - **Prognozowanie sprzedaży,**
 - **Analizę ryzyka kredytowego.**

Boosting jest bardziej podatny na szum w danych, ale pozwala uzyskać wysoką dokładność nawet w trudnych zadaniach.

2.3.4. XGBoost i LightGBM

Te zaawansowane modele boostingu zostały zoptymalizowane pod kątem wydajności i skalowalności:

- **XGBoost:** Wprowadza regularyzację (L1 i L2) i przetwarzanie równoległe, co pozwala na szybkie trenowanie dużych zbiorów danych. Jest szeroko stosowany w:
 - **Prognozowaniu cen akcji,**
 - **Wykrywaniu rzadkich chorób,**
 - **Analizie danych przestrzennych.**
- **LightGBM:** Korzysta z technik takich jak budowa drzewa na podstawie histogramów, co znacząco przyspiesza trenowanie. Zastosowania obejmują:
 - **Wykrywanie oszustw,**
 - **Analizę tekstu, obrazów i dźwięków.**

Oba modele oferują wysoką dokładność, ale ich strojenie może być bardziej skomplikowane w porównaniu do prostszych rozwiązań.

2.3.5. Stacking

Stacking to metoda łączenia różnych modeli bazowych za pomocą meta-modelu. Meta-model uczy się, jak najlepiej łączyć prognozy, co pozwala na:

- **Uwzględnienie różnorodnych aspektów danych,**
- **Zwiększenie skuteczności poprzez integrację różnych podejść.**

Stacking jest szczególnie efektywny w zadaniach, gdzie dane mają skomplikowaną strukturę, a różne modele bazowe mogą uchwycić różne cechy. Zastosowania obejmują:

- **Analiza sentymentu w mediach społecznościowych,**
- **Rozpoznawanie mowy.**

2.3.6. Sieci neuronowe

Sieci neuronowe to zaawansowane modele klasyfikacyjne oparte na warstwach neuronów, które przetwarzają dane wejściowe w sposób inspirowany ludzkim mózgiem. Pozwalają na:

- **Modelowanie złożonych, nieliniowych zależności między cechami,**
- **Przetwarzanie różnorodnych typów danych, takich jak tekst czy sekwencje.**

Sieci neuronowe są szczególnie efektywne w zadaniach, gdzie dane mają strukturę sekwencyjną lub przestrzenną. W kontekście klasyfikacji obiekt (np. wektor cech lub tekst) przechodzi przez moduł klasyfikacji składający się z warstw neuronowych, a wynikiem jest etykieta binarna określająca grupę przynależności. Zastosowania obejmują:

- **Wykrywanie dezinformacji w tekstach mediów społecznościowych,**
- **Rozpoznawanie obrazów i analiza danych medycznych.**

2.3.7. Voting Classifier

Voting Classifier to technika zespołowa, która integruje predykcje wielu klasyfikatorów za pomocą mechanizmu głosowania. Umożliwia:

- **Zwiększenie stabilności predykcji poprzez konsensus,**
- **Wykorzystanie różnorodnych podejść klasyfikacyjnych.**

Metoda ta jest skuteczna w zadaniach wymagających niezawodności i odporności na błędy pojedynczych modeli. Obiekt do klasyfikacji (np. wektor cech) jest analizowany przez moduł złożony z równolegle działających klasyfikatorów, a wynikiem jest etykieta binarna wyłoniona w procesie głosowania. Zastosowania obejmują:

- **Analiza sentymentu w danych tekstowych,**
- **Wykrywanie oszustw w systemach finansowych.**

2.3.8. Modele z post-processingiem

Modele z post-processingiem to podejście, które wzbogaca klasyfikację o dodatkowe kroki korygujące oparte na heurystykach lub analizie niepewności. Pozwalają na:

- **Poprawę precyzji predykcji dzięki dodatkowym informacjom kontekstowym,**
- **Uwzględnienie niepewności modelu w procesie decyzyjnym.**

Są one szczególnie przydatne w specyficznych zadaniach, gdzie standardowa klasyfikacja wymaga doprecyzowania. Obiekt do analizy, tekst lub wektor cech, przechodzi przez moduł klasyfikacji, a następnie jego etykieta binarna jest korygowana na podstawie meta-danych, dając ostateczną grupę przynależności. Zastosowania obejmują:

- **Wykrywanie dezinformacji z uwzględnieniem wiarygodności źródła,**
- **Analiza ryzyka w danych finansowych.**

2.3.9. CatBoost

CatBoost to algorytm boostingu zoptymalizowany do pracy z danymi kategorycznymi, który iteracyjnie ulepsza klasyfikację. Umożliwia:

- **Efektywne przetwarzanie zmiennych kategorycznych bez dodatkowego kodowania,**
- **Wysoką dokładność dzięki gradientowemu podejściu do optymalizacji.**

CatBoost sprawdza się w zadaniach z heterogenicznymi danymi. Obiekt do klasyfikacji, wektor cech z danymi kategorycznymi, jest analizowany przez moduł oparty na drzewach decyzyjnych, a wynikiem jest etykieta binarna określająca grupę. Zastosowania obejmują:

- **Ocena ryzyka kredytowego w finansach,**
- **Prognozowanie sprzedaży w e-commerce.**

2.4. REPREZENTACJE SEMANTYCZNE TEKSTU

W ostatnich latach modele głębokiego uczenia, w szczególności te oparte na architekturze transformatorowej, zrewolucjonizowały przetwarzanie języka naturalnego (NLP). W kontekście generowania osadzeń tekstowych, technologie te umożliwiły znaczący postęp w jakości reprezentacji semantycznych zdań. Poniżej przedstawiono szczegółowy opis trzech kluczowych metod omawianych w artykule [15], z uwzględnieniem ich zalet, ograniczeń oraz zastosowań.

2.4.1. BERT (Bidirectional Encoder Representations from Transformers)

Model BERT stanowi przełom w NLP dzięki dwukierunkowemu przetwarzaniu tekstu, co umożliwia pełne uchwycenie kontekstu zdania. Dzięki warstwom transformatorowym BERT ustanowił nowe standardy w takich zadaniach jak klasyfikacja zdań czy analiza podobieństwa tekstowego (STS). Jego zastosowanie wymaga jednak jednoczesnego podania obu zdań do sieci, co prowadzi do ogromnych kosztów obliczeniowych. Przykładowo, analiza 10 000 zdań za pomocą BERT wymaga przeprowadzenia około 50 milionów obliczeń (~65 godzin na nowoczesnej GPU).

Pomimo sukcesów, BERT nie generuje osadzeń zdań w sposób natywny. Stosowane podejścia, takie jak uśrednianie wyjść z warstwy końcowej (average pooling) czy wykorzystanie tokenu (CLS), często prowadzą do wyników gorszych od metod tradycyjnych, takich jak GloVe. Zastosowanie BERT w zadaniach takich jak klasteryzacja czy wyszukiwanie semantyczne jest ograniczone ze względu na niską jakość generowanych osadzeń oraz bardzo wysokie wymagania obliczeniowe.

2.4.2. RoBERTa (Robustly Optimized BERT Approach)

RoBERTa to ulepszona wersja BERT, zoptymalizowana pod kątem wydajności w zadaniach z nadzorem. Dzięki dłuższemu treningowi, większej ilości danych i lepszej optymalizacji hiperparametrów, RoBERTa osiąga lepsze wyniki w zadaniach klasyfikacyjnych. Usunięcie tokenów (SEP) i (CLS) w procesie pretreningu pozwoliło na zwiększenie efektywności modelu.

Jednak w kontekście generowania osadzeń zdań RoBERTa wykazuje wyniki porównywalne z BERT. W implementacji SBERT (jako SRoBERTa) nie przyniosła istotnej przewagi nad standardowym SBERT. RoBERTa pozostaje bardziej wydajna w zadaniach klasyfikacyjnych, lecz jej zastosowanie w klasteryzacji czy wyszukiwaniu semantycznym jest ograniczone podobnie jak w przypadku BERT.

2.4.3. Sentence-BERT (SBERT)

SBERT stanowi odpowiedź na ograniczenia BERT w generowaniu osadzeń zdań. Wykorzystuje sieci syjamskie (Siamese Networks), które umożliwiają jednoczesne przetwarzanie dwóch zdań za pomocą identycznych wag modelu BERT. Wynikowe osadzenia są porównywane przy użyciu miar, takich jak kosinusowe podobieństwo, co pozwala na szybkie określenie relacji semantycznych między zdaniami.

Jednym z kluczowych usprawnień SBERT jest wprowadzenie warstwy "pooling", która uśrednia wartości tokenów (MEAN), wybiera wartości maksymalne (MAX) lub wykorzystuje token (CLS). Model ten został przeszkolony na zbiorach NLI (Natural Language Inference) oraz STS. Dzięki temu SBERT znacząco redukuje czas obliczeń – analiza 10 000 zdań zajmuje ~5 sekund, w porównaniu do ~65 godzin z BERT.

SBERT przewyższa jakością osadzenia generowane przez BERT, Universal Sentence Encoder czy InferSent. Na przykład w zadaniach STS osiągnął średnią poprawę o 11.7 punktów w porównaniu do InferSent i 5.5 punktów w stosunku do Universal Sentence Encoder.

2.4.4. Porównanie wspomnianych reprezentacji

SBERT to istotny krok naprzód w generowaniu osadzeń zdań, który łączy precyzję i elastyczność modelu BERT z wysoką efektywnością obliczeniową. Dzięki zdolności do szybkiego tworzenia wysokiej jakości osadzeń, SBERT znajduje zastosowanie w zadaniach takich jak klasteryzacja, wyszukiwanie semantyczne oraz analiza podobieństwa tekstowego. Przeanalizujemy jakość osadzeń i zbadajemy, czy faktycznie SBERT zapewni lepsze wyniki, wskazując konkretne zalety i ograniczenia SBERT w kontekście wybranych zastosowań.

Metoda	Opis	Zalety	Ograniczenia
BERT	Generuje osadzenia tokenów dla klasyfikacji zdań i STS.	Dwukierunkowe przetwarzanie, wszechstronność, nowe standardy NLP.	Nie generuje samodzielnych osadzeń, wysokie koszty obliczeń.
RoBERTa	Ulepszony BERT do zadań nadzorowanych NLP.	Lepsza wydajność w klasyfikacji, usunięcie ograniczeń BERT.	Brak poprawy jakości osadzeń względem BERT, wymagania obliczeniowe.
SBERT	Syjamskie sieci BERT do semantycznych osadzeń zdań.	Wysokiej jakości osadzenia, szybka analiza, wszechstronność.	Wymaga dodatkowego treningu, zależność od jakości danych.

Tabela 2.1: Porównanie metod osadzeń tekstowych.

2.5. TECHNIKI UCZENIA W RÓŻNYCH KONTEKSTACH DANYCH

One-Shot Learning (OSL) i Few-Shot Learning (FSL) różnią się podejściem do problemu ograniczonych danych treningowych, a wybór odpowiedniej metody zależy od konkretnego przypadku użycia.

2.5.1. One-Shot Learning (OSL)

OSL wyróżnia się błyskawiczną adaptacją do nowych klas, co czyni ją idealnym wyborem w sytuacjach, gdzie czas reakcji ma kluczowe znaczenie, np. przy identyfikacji nowo pojawiających się form fake news. Jest to rozwiązanie szybkie i skuteczne, szczególnie w sytuacjach, gdzie dostępny jest tylko jeden przykład.

2.5.2. Few-Shot Learning (FSL)

FSL, dzięki wykorzystaniu większej liczby przykładów, oferuje większą elastyczność i dokładność. To podejście sprawdzi się lepiej w scenariuszach, gdzie dostęp do kilku przykładów nowych klas jest możliwy, a kluczowa jest stabilność i precyzja modelu.

2.5.3. Porównanie technik uczenia w różnych kontekstach danych

W artykule [12] szczegółowo opisano metodę CAPD, która łączy zalety obu podejść. CAPD umożliwia dynamiczną adaptację modeli w obu przypadkach, dzięki czemu stanowi uniwersalne rozwiązanie dla problemów związanych z ograniczoną dostępnością danych.

Na podstawie analizy przeprowadzonej w artykule można stwierdzić, że **FSL jest bardziej uniwersalne** i lepiej dostosowane do długoterminowych scenariuszy, gdzie jakość klasyfikacji i stabilność modelu mają kluczowe znaczenie. Niemniej jednak, **OSL pozostaje niezastąpione w sytuacjach krytycznych**, gdy czas reakcji ma większe znaczenie niż precyzja.

Dzięki innowacyjnemu podejściu opartemu na CAPD, zarówno OSL, jak i FSL mogą zostać zoptymalizowane pod kątem dynamicznego środowiska, takiego jak wykrywanie fake news, co zostało szczegółowo omówione w artykule. CAPD wykorzystuje zarówno minimalne dane OSL, jak i większe zbiory treningowe FSL, aby zwiększyć zdolności adaptacyjne modelu.

2.6. IMPLEMENTACJA PRZYGOTOWANIA DANYCH

Proces przygotowania danych wejściowych stanowi wspólny etap dla wszystkich implementowanych rozwiązań metod uczenia zespołowego, niezależnie od tego, czy opisujemy implementacje z naukowych artykułów, czy autorskie metody. Proces przygotowania danych został zaimplementowany w skrypcie głównym i obejmuje kilka kluczowych kroków, które zapewniają spójną bazę danych dla dalszego przetwarzania klasyfikacyjnego.

Pierwszym krokiem jest wczytanie danych przy użyciu funkcji `load_and_preprocess_data`. Nasza implementacja wykorzystuje pliki `Fake.csv`, zawierający fałszywe wiadomości, oraz `True.csv`, który obejmuje prawdziwe wiadomości. Funkcja `load_and_preprocess_data` wczytuje oba zbiory, a następnie nadaje im binarne etykiety: 0 dla fałszywych wiadomości i 1 dla prawdziwych, co odpowiada zadaniu klasyfikacji binarnej w wykrywaniu fake newsów. Zwracamy uwagę, że dataset musi zawierać obowiązkowe kolumny: `title`, `text`, `subject`. Kolejnym etapem jest połączenie tych zbiorów w jedną tabelę danych. Na koniec dane tekstowe, oznaczone jako `X`, oraz etykiety, oznaczone jako `y`, są przygotowywane do dalszej analizy.

Następnym krokiem jest tworzenie reprezentacji kontekstowej, realizowane przez funkcję `get_embeddings`. Do dyspozycji mamy trzy możliwe opcje, przedstawione poniżej:

- **BERT**: funkcja `get_bert_embeddings` tokenizuje teksty przy użyciu `BertTokenizer`, a następnie generuje osadzenia w partiach (`batch_size=32`) za pomocą modelu `TFBertForSequenceClassification`.
- **RoBERTa**: funkcja `get_roberta_embeddings` tokenizuje teksty przy użyciu `RobertaTokenizer`, a następnie generuje osadzenia w partiach (`batch_size=32`) za pomocą modelu `TFRobertaForSequenceClassification`.
- **Sentence Transformers**: funkcja `get_transformer_embeddings` wykorzystuje model `all-MiniLM-L6-v2` do generowania osadzeń.

Alternatywnie, jeśli wybrano wektoryzację TF-IDF, funkcja `vectorize_data` przekształca dane tekstowe na wektor cech TF-IDF, z maksymalną liczbą cech ustawioną na `max_features=5000`, uwzględniając bigramy i usuwając stopwords.

Ostatnim krokiem jest podział danych na zbiory treningowe i testowe, realizowany przez jedną z poniższych funkcji:

- **split_data**: wykonuje klasyczny podział w proporcji 80:20 (z `test_size=0.2`) z zachowaniem losowości (`random_state=42`).
- **split_data_one_shot**: wybiera dokładnie jeden przykład na klasę do zbioru testowego, a resztę danych przypisuje do zbioru treningowego, co jest przydatne w scenariuszach z ograniczoną liczbą danych.
- **split_data_few_shot**: wybiera kilka przykładów na klasę (`few_shot_examples=5`), co pozwala na symulację scenariuszy z małą liczbą przykładów treningowych.

Wszystkie te kroki przygotowują dane do dalszego etapu przetwarzania klasyfikacyjnego, opisanego w kolejnych sekcjach, gdzie są wykorzystywane przez każdą metodę jako:

- **X_train** – tablica ndarray przechowująca cechy zbioru treningowego, takie jak wektory TF-IDF lub osadzenia kontekstowe z BERT, RoBERTa czy Sentence Transformers, wykorzystywane do trenowania modelu klasyfikacyjnego.
- **y_train** – lista lub tablica ndarray zawierająca etykiety binarne zbioru treningowego, gdzie 0 oznacza fałszywe wiadomości, a 1 oznacza prawdziwe.
- **X_test** – tablica ndarray przechowująca cechy zbioru testowego, w formacie analogicznym do **X_train**, używane do ewaluacji skuteczności modelu.
- **y_test** – lista lub tablica ndarray zawierająca etykiety binarne zbioru testowego, gdzie 0 oznacza fałszywe wiadomości, a 1 oznacza prawdziwe, służące jako punkt odniesienia dla predykcji modelu.

2.7. IMPLEMENTACJA WYBRANYCH NAUKOWYCH METOD

Celem niniejszego przeglądu jest analiza wybranych implementacji stosowanych w wykrywaniu fake newsów. Podzielono je na pięć głównych kategorii: **modele oparte na drzewach** (Tree-Based Models), **modele liniowe i statystyczne** (Linear and Statistical Models), **sieci neuronowe** (Neural Networks), **zespoły klasyfikatorów** (Ensemble Classifiers) oraz **modele zoptymalizowane** (Optimized Models). W każdej z tych kategorii omówiono charakterystyczne cechy oraz innowacyjne modyfikacje w implementacjach, które przyczyniają się do poprawy skuteczności klasyfikacji. Pragniemy podkreślić, że wszystkie 16 omawianych podejść i implementacji bazują na 16 różnych metodach naukowych, stanowiących fundament analizy. Wybór tych konkretnych metod został podyktowany ich udokumentowaną skutecznością w literaturze naukowej, ze szczególnym uwzględnieniem publikacji z ostatnich lat, które koncentrują się na analizie danych tekstowych i detekcji dezinformacji. Metody te zostały wyselekcjonowane na podstawie ich zdolności do adresowania kluczowych wyzwań w tym obszarze, takich jak przetwarzanie danych, modelowanie złożonych zależności semantycznych oraz radzenie sobie z nieźrównoważonymi zbiorami danych, charakterystycznymi dla problematyki fake newsów. Dodatkowo, istotnym czynnikiem wyboru była łatwość implementacji, dostępność propozycji kodu

źródłowego w popularnych bibliotekach, gotowych implementacji, oraz programistyczna wygoda, ułatwiająca adaptację i testowanie tych metod w praktyce.

Dodatkowo, proponujemy traktować każde podejście, opisane w jednym z 16 artykułów naukowych, jako odrębną metodę, co ułatwi ich prezentację i wyjaśnienie. Jednocześnie podkreślamy, że autorskie metody, będą omówione w kolejnych rozdziałach, zostaną przedstawione jako metody numer 17, 18, 19 i 20, uzupełniając cel badania. Autorskie metody zostały opracowane na podstawie analizy 16 wspomnianych wyżej podejść naukowych, co pozwoliło na stworzenie skutecznych rozwiązań.

2.7.1. Modele oparte na drzewach

metoda12

metoda12 – Random Forest i CatBoost: ensemble learning z baggingiem i boostingiem. Metoda ta trenuje równolegle klasyfikatory Random Forest i CatBoost, oceniając ich predykcje na danych testowych za pomocą metryk takich jak dokładność, precyzja, czułość, F1 i ROC-AUC.

W oryginalnym podejściu opisanym w artykule [1] proces klasyfikacji opiera się na ensemble learningu z baggingiem Random Forest i boostingiem CatBoost dla binarnej detekcji fałszywych wiadomości. Dane z zestawów ISOT Fake News i COVID-19 Fake News są oczyszczane: usuwanie stop-słów, lematyzacja za pomocą NLTK WordNetLemmatizer, tworząc wektory cech. Zbiór jest dzielony na 80% treningowy i 20% testowy, a modele Random Forest oraz CatBoost są trenowane na danych treningowych, z CatBoost monitorującym dane testowe. Wyniki są oceniane na zestawie testowym, osiągając dokładność 99% dla Random Forest i 98% dla CatBoost.

Nasza implementacja metoda12 zachowuje wspólne cechy tej metody opisanej w [1], opierając się na równoległym treningu Random Forest i CatBoost oraz ocenie metryk. Dane wejściowe, czyli `X_train` reprezentujące cechy treningowe oraz `X_test` odpowiadające cechom testowym, są przyjmowane jako tablice bez dodatkowego preprocessingu, co różni się od oczyszczania i kodowania TF-IDF w artykule, ale zachowuje ideę pracy na wektorach cech. Proces zaczyna się od inicjalizacji `RandomForestClassifier` i `CatBoostClassifier`, co jest zgodne z konfiguracją z artykułu. Random Forest jest trenowany na `X_train` i `y_train`, generując predykcje `rf_y_pred` na `X_test`, podobnie jak w artykule. CatBoost jest trenowany na `X_train` i `y_train` z monitorowaniem `X_test` i `y_test`, generując predykcje `catboost_y_pred`, co odzwierciedla ustawienie `eval_set` z artykułu. Wyniki są oceniane za pomocą funkcji `evaluate_model`, obliczającej dokładność, precyzję, czułość, F1 i ROC-AUC na podstawie `y_test`, co odpowiada metrykom z artykułu. Metoda zwraca słownik zawierający wytrenowane modele (`rf_classifier`, `catboost_classifier`) oraz ich metryki (`rf_metrics`, `catboost_metrics`), co umożliwia dalszą analizę, podobnie jak w artykule, gdzie porównywano wydajność modeli.

2.7.2. Sieci neuronowe

metoda1

`metoda1` – Ensemble sieci neuronowych: klasyfikacja binarna z różnorodnymi hiperparametrami. Metoda ta trenuje wiele modeli sieci neuronowych o różnych konfiguracjach hiperparametrów, łącząc ich przewidywania w ensemble, a wyniki ocenia na danych testowych za pomocą uśrednionych predykcji.

Nasza implementacja `metoda1` całkowicie opiera się na artykule [3], który opisuje trening ensemble składający się z 5 sieci neuronowych, z losowo wybieranymi hiperparametrami. Każdy model tworzony jest za pomocą funkcji `create_model`, budującej sieć sekwencyjną z trzema warstwami gęstymi (`units1`, `units2`, `units3`), z regularizacją L2, normalizacją (`BatchNormalization`), aktywacją `LeakyReLU`, warstwami `Dropout` i wyjściową warstwą z aktywacją `sigmoid` dla klasyfikacji binarnej. Model jest kompilowany z optymalizatorem `Adam` i funkcją straty `binary_crossentropy`. Trening każdego modelu odbywa się na danych `X_train` i `y_train` przez 20 iteracji, z walidacją na `X_test` i `y_test`, wykorzystując redukcję `ReduceLROnPlateau` o czynnik 0.5 przy stagnacji straty walidacyjnej przez 3 iteracji. Predykcje każdego modelu na `X_test` są zbierane i uśredniane, tworząc finalne przewidywania `predictions`. Metoda zwraca listę wytrenowanych modeli `models`, `X_test` oraz `y_test`, umożliwiając dalszą analizę.

metoda4

`metoda4` – Bi-LSTM, CNN i MLP z klasyfikacją sigmoidalną. Metoda ta wykorzystuje podejście ensemble, w którym Bi-LSTM, CNN i MLP przetwarzają dane wejściowe, a wyniki są łączone i klasyfikowane za pomocą funkcji sigmoidalnej.

Nasza implementacja `metoda4` zachowuje oryginalne cechy tego podejścia opisanego w [16], opierając się na ensemble learningu z Bi-LSTM, CNN i MLP, ale koncentruje się na klasyfikacji binarnej. Dane wejściowe, czyli `X_train` reprezentujące cechy treningowe oraz `X_test` odpowiadające cechom testowym, są skalowane za pomocą `StandardScaler` i przekształcane na format trójwymiarowy (`X_train_reshaped`, `X_test_reshaped`), co różni się od osadzania wektorowego w artykule, ale zachowuje ideę przygotowania danych do sieci neuronowych. Proces zaczyna się od inicjalizacji warstwy wejściowej (`Input`) o dynamicznej długości sekwencji, przez którą dane przechodzą do trzech ścieżek: `Bidirectional(LSTM)` dla sekwencyjnego przetwarzania, `Conv1D` z `MaxPooling1D` dla wykrywania wzorców oraz `Dense` dla prostego przetwarzania cech.

Wyjścia są spłaszczane, łączone i przekazywane do warstwy `Dense` z aktywacją `ReLU`, a następnie do warstwy wyjściowej z aktywacją `softmax` `Dense`, co odpowiada podejściu w artykule. Model jest kompilowany z optymalizatorem `Adadelta` i trenowany przez 3 rodzaje próbek, używając `X_test_reshaped` i `y_test` do walidacji, podobnie jak w artykule, gdzie testowy zestaw LIAR był używany do oceny.

Metoda zwraca wytrenowany model (`Model`), `X_test_reshaped` oraz `y_test`, co umożliwia dalszą analizę, zgodnie z ideą oceny wyników w artykule.

metoda15

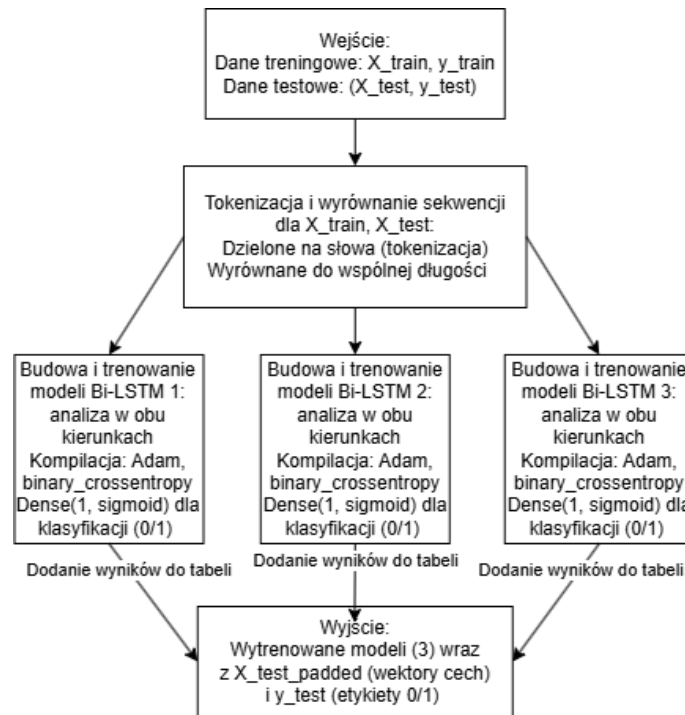
`metoda15` – Ensemble modeli Bi-LSTM: modeli zespołowe oparte na dwukierunkowej warstwie LSTM z tokenizacją i wyrównaniem długości sekwencji tekstowych.

Dane wejściowe są najpierw dzielone na pojedyncze słowa (tokenizacja), a ich długości są wyrównywane do wspólnej wartości, aby pasowały do wymagań sieci. Następnie tworzona jest warstwa wejściowa, przez którą dane przechodzą do trzech modeli Bi-LSTM, które analizują tekst w obu kierunkach aby lepiej zrozumieć kontekst. Każdy model jest kompilowany z optymalizatorem Adam, używając funkcji straty `binary_crossentropy` do klasyfikacji binarnej. Wyniki z tych modeli są łączone i przetwarzane, a na końcu klasyfikowane za pomocą funkcji sigmoidalnej, która określa, czy wiadomość jest prawdziwa (1) czy fałszywa (0).

Modele są trenowane przez maksymalnie 20 cykli z partiami po 64 próbki, z danymi testowymi używanymi do sprawdzenia postępów. W trakcie trenowania stosowany jest mechanizm wczesnego zatrzymania (`EarlyStopping`), który monitoruje stratę na danych walidacyjnych i przywraca najlepsze wagi, jeśli wyniki przestają się poprawiać przez 2 cykle, co zapobiega przeuczeniu. Na koniec tworzona jest lista trzech wytrenowanych modeli Bi-LSTM. Wyniki są zwracane jako lista wytrenowanych modeli oraz przygotowane dane testowe.

Lista wytrenowanych modeli to trzy gotowe sieci neuronowe zdolne do przewidywania, czy wiadomość jest prawdziwa (1) czy fałszywa (0). Przygotowane dane testowe obejmują tablicę cech (`X_test`), która zawiera przetworzone teksty po tokenizacji i wyrównaniu, oraz tablicę etykiet (`y_test`) z wartościami 0 lub 1, służącymi do weryfikacji predykcji.

Dzięki analizie artykułu [17], powstała implementacja `metoda15`. Różnica pomiędzy naszą implementacją a oryginałem polega na tym, że artykuł opisuje ensemble Bi-LSTM z VGG-19 i mechanizmem uwagi do klasyfikacji multimodalnej (tekst i obrazy) z niestandardową funkcją straty, podczas gdy nasza implementacja skupia się na klasyfikacji binarnej tekstu za pomocą Bi-LSTM bez dodatkowej warstwy wizualnej. Chcąc zaprezentować nasz pomysł, szczegóły implementacji są pokazane na podanym diagramie 2.1.



Rys. 2.1: Schemat metody 15.

2.7.3. Zespoły klasyfikatorów

metoda2

metoda2 – Stacking z GradientBoosting i MLP z Regresją Logistyczną. W artykule [13] się opiuje podejście stackingu, w którym modele bazowe generują predykcje prawdopodobieństw, a meta-klasyfikator na ich podstawie dokonuje ostatecznej klasyfikacji.

Nasza implementacja metoda2 zachowuje kluczowe wspólne cechy tego podejścia, opierając się na podobnej strukturze stackingu i wykorzystaniu ensemble learningu. W naszej wersji dane wejściowe, czyli `X_train` reprezentujące cechy treningowe oraz `X_test` odpowiadające cechom testowym. Proces zaczyna się od inicjalizacji modeli bazowych: `GradientBoostingClassifier` oznaczanego jako `boosted_tree` oraz `MLPClassifier` oznaczanego jako `neural_net`, co odpowiada użyciu Boosted Decision Tree i Neural Network w artykule.

Następnie generujemy predykcje dla danych treningowych, czyli `boosted_tree_preds_test` i `neural_net_preds_test`, obliczając prawdopodobieństwa klas odpowiednio od `boosted_tree` na podzbiorze zachowań społecznych (`social_behavior_features_test`) i `neural_net` na podzbiorze demograficznym (`demographic_features_test`), które są łączone w `combined_preds_test`, co jest zgodne z podejściem artykułu, w którym predykcje modeli bazowych stają się meta-cechami. Te meta-cechy są używane do treningu meta-klasyfikatora `LogisticRegression`, ozna-

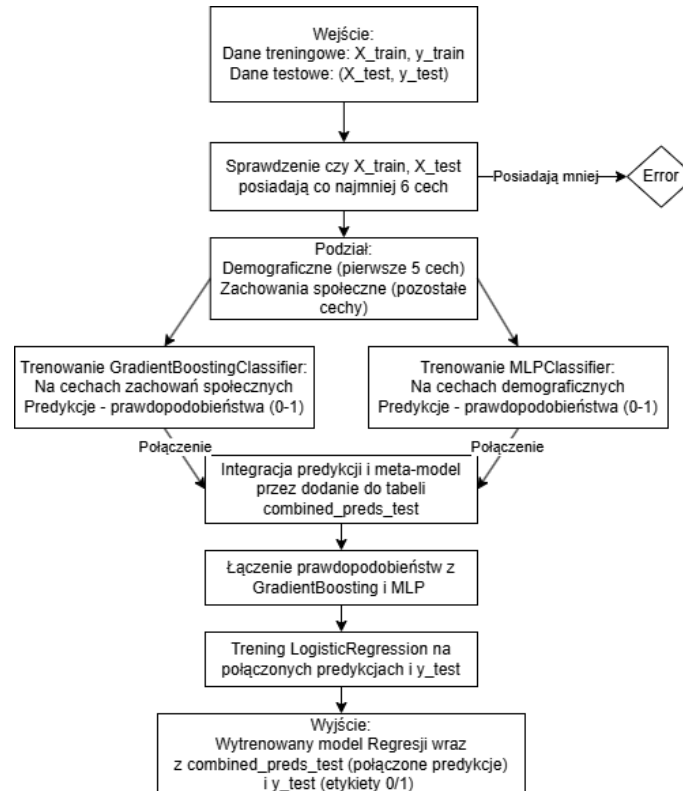
czonego jako `logistic_reg`, na `combined_preds_test` i `y_test`, co się bazuje na etapie treningu regresji logistycznej w artykule.

Modele bazowe `boosted_tree` i `neural_net` są trenowane na odpowiednich podzbiorach `X_train` (`social_behavior_features_train` i `demographic_features_train`) z `y_train`, podobnie jak w artykule, gdzie następuje trening na podzielonych danych.

Ostatecznie `logistic_reg` dokonuje predykcji na `combined_preds_test`, a wyniki mogą być oceniane na podstawie `y_test` (etykiety testowe), co jest wspólnym elementem z artykułem, gdzie stosowano metryki takie jak dokładność. Metoda zwraca `logistic_reg`, `combined_preds_test` oraz `y_test`, co umożliwia dalszą analizę.

Nasza implementacja różni się od podejścia w artykule [13], ponieważ zamiast `TwoClass Boosted Decision Tree` i `TwoClass Neural Network` stosuje `GradientBoostingClassifier` i `MLPClassifier`, co upraszcza strukturę modeli. Artykuł wykorzystuje preprocessing danych: czyszczenie, zastępowanie brakujących wartości modą.

Podczas gdy nasz kod działa na ogólnych danych bez preprocessingu, wymagając jedynie podziału na podzbiory demograficzne i społeczne. Wspólnym elementem jest stacking z Regresją Logistyczną jako meta-modelem, ale nasza wersja jest bardziej uniwersalna, tracąc specyficzność cech wiarygodności kluczowych w artykule. Szczegóły implementacji są pokazane na podanym diagramie 2.2.



Rys. 2.2: Schemat metody 2.

metoda3

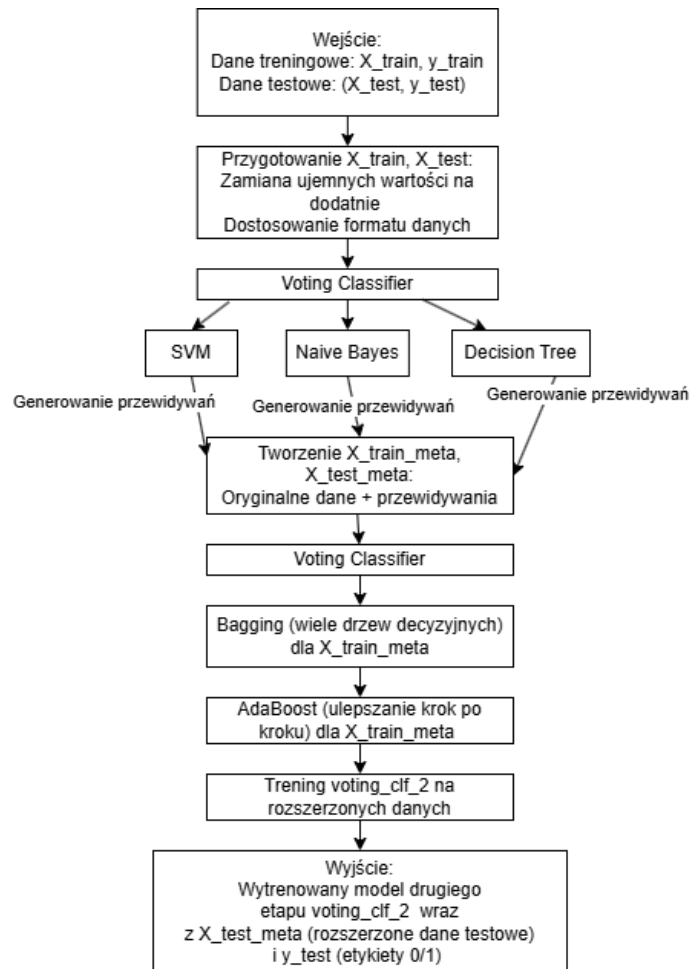
metoda3 – Dwupoziomowy Voting Classifier: SVM, Naive Bayes, Decision Tree i Bagging/AdaBoost.

Nasza implementacja posiada trzy klasyfikatory bazowe: SVM SVC, Naive Bayes MultinomialNB oraz Decision Tree DecisionTreeClassifier. Klasyfikatory te są łączone przez VotingClassifier z miękkim głosowaniem, które generuje prawdopodobieństwa predykcji `predict_proba` dla `X_train` i `X_test`, tworząc meta-cechy: `predictions_1_train` i `predictions_1_test`. Te meta-cechy są łączone z oryginalnymi danymi, tworząc rozszerzone zbiory `X_train_meta` i `X_test_meta`, gdzie każda próbka zawiera oryginalne cechy oraz prawdopodobieństwa predykcji z pierwszego poziomu.

Następnie trenowany jest kolejny zespół VotingClassifier `voting_clf_2`, składający się z dwóch klasyfikatorów: BaggingClassifier oraz AdaBoostClassifier. Model ten jest dopasowywany do `X_train_meta` i `y_train`, łącząc predykcje z pierwszego poziomu z oryginalnymi cechami, aby uzyskać ostateczną decyzję klasyfikacyjną. Proces ten odzwierciedla ideę zespołowego uczenia z artykułu [18], gdzie stosowano voting classifier do agregacji wyników różnych modeli, choć w artykule nacisk położono na cechy językowe i word embedding.

Wyniki zwracane przez metoda3 obejmują: gotowy model drugiego poziomu (`voting_clf_2`), czyli wytrenowany VotingClassifier z ustawieniami i wagami, meta-cechy testowe (`X_test_meta`), które łączą oryginalne cechy `X_test` z predykcjami pierwszego poziomu, oraz etykiety testowe (`y_test`) z wartościami 0 lub 1, umożliwiające ocenę skuteczności modelu.

Główna różnica między naszą implementacją a artykułem dotyczy podejścia do danych i struktury klasyfikacji – artykuł opiera się na ekstrakcji cech językowych i word embedding (TF-IDF, Count Vectorizer) z trzema zestawami LFS, podczas gdy nasza implementacja działa na ogólnych danych, wykorzystując predykcje pierwszego poziomu jako meta-cechy, bez preprocessingu tekstowego. Wspólnym elementem jest użycie voting classifiera i ensemble learningu z modelami takimi jak SVM, Naive Bayes czy Decision Tree, co zwiększa dokładność. Nasza implementacja jest bardziej uniwersalna, ale traci specyficzność cech tekstowych, kluczowych w artykule. Szczegóły implementacji, w tym przepływ danych przez klasyfikatory przedstawia diagram 2.3.



Rys. 2.3: Schemat metody 3.

metoda5

metoda5 – Dwuwarstwowy Stacking Classifier z Random Forest, XGBoost i regresją logistyczną: Nasza implementacja stosuje podejście stackingowe do klasyfikacji, zgodne z ideami ensemble learningu opisanymi w artykule [7], gdzie różne modele maszynowe, takie jak Decision Tree, Random Forest i Extra Tree Classifier, są łączone w celu poprawy dokładności wykrywania fałszywych wiadomości.

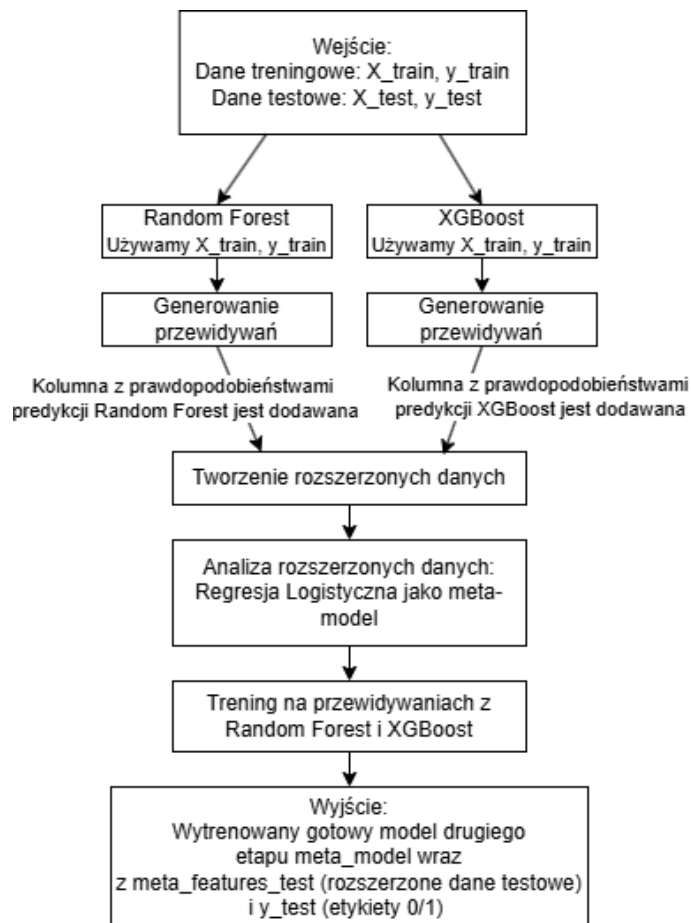
Nasza implementacja w pierwszym etapie ma trenowanie dwóch modeli bazowych: Random Forest RandomForestClassifier oraz XGBoost XGBClassifier. Modele te są dopasowywane do `X_train` i `y_train`, generując prawdopodobieństwa predykcji (`predict_proba`) dla klasy pozytywnej, co tworzy nowe cechy dla meta-modelu: `rf_preds_train` i `xgb_preds_train`. Te predykcje są łączone w tablicę danych `meta_features_train` z kolumnami `rf_preds` i `xgb_preds`, zawierającą dwie cechy na próbkę.

W drugim etapie meta-model, czyli regresja logistyczna (LogisticRegression), jest trenowany na `meta_features_train` i `y_train`, ucząc się łączenia predykcji modeli bazowych w celu uzyskania ostatecznej decyzji. Dla danych testowych generowa-

ne są predykcje modeli bazowych (`rf_preds_test`, `xgb_preds_test`), które tworzą `meta_features_test`. W artykule [7] podobnie jak w naszej implementacji różne klasyfikatory są łączone poprzez agregację ich wyników, choć w artykule stosowano bagging zamiast stackingu.

Wyniki zwracane przez `metoda5` obejmują: gotowy meta-model (`meta_model`), czyli wytrenowaną regresję logistyczną z ustawieniami i wagami, cechy testowe (`meta_features_test`), które łączą predykcje Random Forest i XGBoost dla `X_test` oraz etykiety testowe (`y_test`) z wartościami 0 lub 1, umożliwiające ocenę modelu. Struktura ta pozwala na analizę skuteczności modelu, podobnie jak w artykule, gdzie oceniano dokładność na zbiorach ISOT i Liar.

`metoda5` różni się od podejścia opisanego w artykule [7]. W artykule autorzy skupili się na ekstrakcji 26 cech tekstowych z danych tekstowych, które następnie klasyfikowano za pomocą ensemble opartego na baggingu z Decision Tree, Random Forest i Extra Tree Classifier. Nasza implementacja nie wymaga ekstrakcji specyficznych cech tekstowych, działając na ogólnych danych wejściowych (`X_train`, `X_test`), i wykorzystuje stacking z Random Forest i XGBoost jako modelami bazowymi oraz regresją logistyczną jako meta-modelem, zamiast baggingu. Wspólną cechą jest zastosowanie ensemble learningu do poprawy dokładności poprzez łączenie różnych klasyfikatorów. Nasza metoda jest bardziej uniwersalna, ponieważ nie zakłada określonego typu danych, ale traci specyficzność cech tekstowych, które w artykule znacząco poprawiły wyniki po ekstrakcji cech. Artykuł podkreśla znaczenie preprocessingu: tokenizacja, usuwanie stop-words, stemming z użyciem NLTK, czego nasza implementacja nie obejmuje, koncentrując się na warstwie modelowania. Szczegóły implementacji przedstawia diagram 2.4.



Rys. 2.4: Schemat metody 5.

metoda6

metoda6 – Weighted Convolutional Neural Network Ensemble (WCNNE) z AdaBoost i opcjonalnymi osadzeniami BERT: Nasza implementacja stosuje podejście zespołowe do klasyfikacji sentymentów, zgodne z ideami ensemble learningu opisanymi w artykule [6], gdzie różne modele, w tym konwolucyjne sieci neuronowe (CNN), są łączone w celu poprawy dokładności analizy sentymentów, szczególnie na danych tekstowych.

Nasza implementacja w metoda6 rozpoczyna się od otrzymania danych treningowych (X_{train} , y_{train}) i testowych (X_{test} , y_{test}). Dane tekstowe w X_{train} i X_{test} są przekształcane na osadzenia za pomocą pretrenowanego modelu BERT (`bert-base-uncased`), co pozwala na lepsze zrozumienie semantyki tekstu, podobnie jak w artykule, gdzie BERT jest wykorzystywany do ekstrakcji cech tekstowych. Osadzenia są generowane z użyciem `BertTokenizer` i `TFBertModel`, tworząc trójwymiarowe tablice o wymiarach: liczba próbek, maksymalna długość sekwencji, wymiar osadzenia. Następnie dane są skalowane za pomocą `StandardScaler`, tworząc X_{train_scaled} i X_{test_scaled} , aby zapewnić normalizację wymaganą przez modele CNN, co odpowiada standardowemu preprocessingowi w artykule. Określa się kształt wejścia `input_shape`, `embedding_dim` na podstawie wymiarów X_{train_scaled} oraz liczbę klas

(`num_classes`) na podstawie unikalnych wartości w `y_train`, umożliwiając elastyczne dostosowanie modelu do klasyfikacji binarnej lub wieloklasowej.

W kolejnym etapie definiuje się klasyfikator bazowy w postaci konwolucyjnej sieci neuronowej (CNN) zaimplementowanej w metodzie `create_cnn_model`, zgodnej z konfiguracją z artykułu. Model CNN składa się z dwóch warstw `Conv1D`, przeplatanych warstwami `MaxPooling1D`, warstwy `Dropout` dla regularyzacji oraz warstwy `Flatten` i `Dense` z aktywacją sigmoidalną dla klasyfikacji binarnej, skompilowanej z optymalizatorem `Adam`. Klasyfikator CNN jest opakowany w `KerasClassifier`. Następnie tworzony jest klasyfikator zespołowy `WCNNE` z użyciem `AdaBoostClassifier`, który łączy trzy instancje CNN z szybkością uczenia `learning_rate=0.01`, co odpowiada strukturze `WCNNE` opartej na `AdaBoost` opisanej w [6]. Model `model` jest trenowany na `X_train_scaled` i `y_train`, co odzwierciedla sekwencyjne uczenie słabych klasyfikatorów w `AdaBoost` dla poprawy wydajności.

Wyniki są zwracane w następujący sposób: gotowy klasyfikator zespołowy (`model`), zawierający wytrenowane CNN i ustawienia `AdaBoost`, dane testowe (`X_test`), które są osadzeniami oraz etykiety testowe (`y_test`) z wartościami 0, 1, umożliwiające ocenę modelu.

metoda8

`metoda8` – Voting Classifier z Random Forest i XGBoost: probabilistyczne głosowanie predykcji.

W artykule [5] omówiono różne strategie ensemble learningu, takie jak bagging, boosting oraz głębokie modele neuronowe, stosowane w celu poprawy generalizacji modeli uczenia maszynowego, w tym Random Forest oparty na baggingu, modele oparte na boostingu, takie jak XGBoost, oraz głębokie sieci neuronowe, takie jak DBN, które zwiększają wydajność dzięki hierarchicznemu uczeniu cech, osiągając lepsze wyniki w klasyfikacji dzięki różnorodności predykcji.

Nasza implementacja w `metoda8` całkowicie jest podobna do wspomianej z [5], realizując podejście zespołowe do klasyfikacji binarnej lub wieloklasowej poprzez połączenie Random Forest, XGBoost oraz głębokiej sieci neuronowej (DBN) w klasyfikatorze Voting Classifier z miękkim głosowaniem, co w pełni odpowiada strategiom ensemble learningu opisanym w artykule, szczególnie w aspekcie włączenia głębokich modeli. Proces rozpoczyna się od przygotowania danych treningowych (`X_train`, `y_train`) i testowych (`X_test`, `y_test`). Jeśli `X_train` lub `X_test` są macierzami rzadkimi, konwertuje się je na macierze gęste, co jest zgodne z potrzebą odpowiedniego formatowania danych w artykule, gdzie różne zestawy danych wymagały preprocessingu.

Następnie dane są skalowane za pomocą `StandardScaler`, tworząc `X_train_scaled` i `X_test_scaled`, aby zapewnić normalizację wymaganą przez głębokie modele neuronowe, co odzwierciedla standardowe praktyki preprocessingu dla DBN. Określa się

liczbę cech (`input_dim`) na podstawie `X_train_scaled.shape[1]` oraz liczbę klas (`num_classes`) na podstawie unikalnych wartości w `y_train`, co umożliwia dynamiczne dostosowanie modelu DBN do problemu klasyfikacji. Definiuje się trzy klasyfikatory bazowe: `RandomForestClassifier` z 100 estymatorami oznaczany jako `rf`, oparty na strategii baggingu; `XGBClassifier` z 100 estymatorami, oznaczany jako `xgb`, oparty na boostingu; oraz `KerasClassifier` opakowujący model DBN zdefiniowany w `create_dbn_model`, oznaczany jako `dbn`, z warstwami z aktywacją ReLU, warstwami Dropout dla regularyzacji oraz warstwą wyjściową z aktywacją sigmoidalną dla klasyfikacji binarnej lub softmax dla wieloklasowej. Wybór tych klasyfikatorów dokładnie pokrywa się z omówieniem baggingu, boostingu i głębokich modeli w artykule jako kluczowych technik ensemble learningu.

Klasyfikatory są łączone w `VotingClassifier`, oznaczany jako `ensemble_model`, z miękkim głosowaniem, które uśrednia prawdopodobieństwa predykcji z `rf`, `xgb` i `dbn`, podobnie jak w artykule strategie ensemble łączyły predykcje dla poprawy wydajności. Model `ensemble_model` jest trenowany na `X_train_scaled` i `y_train`, co odpowiada etapowi uczenia w artykule, gdzie modele ensemble, w tym głębokie, były trenowane na różnych zbiorach danych po odpowiednim preprocessingu. Po wytrenowaniu metoda zwraca `ensemble_model`, `X_test` oraz `y_test`, gdzie `y_test` zawiera etykiety binarne (0 lub 1) lub wieloklasowe, umożliwiając dalszą ocenę modelu, co jest zgodne z praktyką oceny modeli w [5].

metoda9

`metoda9` – Voting Classifier z Logistic Regression, Random Forest i SVC oraz porównanie modeli zespołowych.

W artykule [2] opisano podejście oparte na ensemble learning do wykrywania fałszywych wiadomości, stosując różne techniki zespołowe, w tym Voting Classifier z `LogisticRegression`, `RandomForestClassifier` oraz innymi modelami, takimi jak KNN lub LSVM, a także Bagging i Boosting, w tym AdaBoost i XGBoost, oceniane za pomocą metryk takich jak dokładność, precyzja, recall i F1-score na czterech zbiorach danych, osiągając średnią dokładność 92.25% dla metod zespołowych.

Nasza implementacja w `metoda9` zachowuje te kluczowe cechy, opierając się na podobnym podejściu ensemble learningu do klasyfikacji binarnej fałszywych wiadomości, z wykorzystaniem kombinacji modeli i ich predykcji dla poprawy dokładności. W artykule dane są przygotowywane z użyciem narzędzia LIWC do ekstrakcji cech językowych, co w naszym procesie odpowiada treningowi na danych `X_train` (cechy treningowe) i `y_train` (etykiety treningowe), które są już w odpowiedniej formie wektorowej, umożliwiając bezpośrednie zastosowanie modeli. Proces rozpoczyna się od zdefiniowania modeli bazowych: `LogisticRegression` oznaczanego jako `log_reg`, `RandomForestClassifier` oznaczanego jako `rf` oraz `KNeighborsClassifier` oznaczanego jako `knn`, które pokrywają się z wyborem modeli jak w artykule, tworząc podstawę dla uczenia zespołowego.

Dodatkowo stosujemy `BaggingClassifier` z `rf` jako estymatorem, oznaczany jako bagging, oraz `AdaBoostClassifier`, oznaczany jako boosting, co jest zgodne z technikami Bagging i Boosting (AdaBoost) opisanymi w [2], wzmacniając ideę ensemble learningu. Modele te są łączone w `VotingClassifier`, oznaczanym jako voting, co pozwala na integrację predykcji prawdopodobieństw z każdego modelu, podobnie jak w artykule `Voting Classifier` łączył wyniki modeli dla uzyskania wysokiej dokładności. Trening wszystkich modeli, w tym `rf`, bagging, boosting i voting, odbywa się na `X_train` i `y_train`, co odpowiada etapowi uczenia w artykule, gdzie różne algorytmy były trenowane na danych z cechami językowymi.

Po wytrenowaniu każdy model generuje predykcje `y_pred` na `X_test` (cechy testowe), a wyniki są oceniane za pomocą `accuracy_score`, `roc_auc_score` (jeśli model wspiera predykcje prawdopodobieństw) oraz `classification_report` na podstawie `y_test` (etykiety testowe z wartościami 0 lub 1), co jest wspólnym elementem z artykułem, gdzie stosowano metryki takie jak dokładność, precyzja, recall i F1-score do oceny wydajności. Metoda zwraca model (najlepszy model), `X_test` oraz `y_test`, umożliwiając dalszą analizę, podobnie jak w artykule wyniki były dostępne do porównania na różnych zbiorach danych.

metoda10

`metoda10` – `Voting Classifier` z `MLP`, `Logistic Regression` i `XGBoost` z miękkim głosowaniem.

W artykule [10] jest zaproponowany `Voting Classifier` z trzema najlepszymi klasyfikatorami wybranymi po walidacji krzyżowej: `MLPClassifier`, `LogisticRegression` i `XGBClassifier`. Modele te są łączone z miękkim głosowaniem (`voting='soft'`) na podstawie prawdopodobieństw, osiągając dokładność 94.47% na danych tekstowych po preprocessingu.

Nasza implementacja w `metoda10` zachowuje kluczowe wspólne cechy tego podejścia, wykorzystując identyczne modele i strukturę ensemble do klasyfikacji binarnej na podstawie danych tekstowych. W artykule modele są trenowane na danych po preprocessingu, co znajduje swoje odzwierciedlenie w naszym procesie, gdzie dane wejściowe `X_train` (cechy treningowe) i `y_train` (etykiety treningowe) są bezpośrednio używane do treningu, choć w naszym kodzie pomijamy preprocessing, korzystając z wspólnego procesu przygotowania danych dla wszystkich metod, co czyni podejście bardziej uniwersalnym, ale zachowuje zgodność z ideą klasyfikacji na przygotowanych danych.

Proces rozpoczyna się od inicjalizacji tych samych klasyfikatorów co w artykule: `MLPClassifier` oznaczanego jako `mlp`, `LogisticRegression` oznaczanego jako `log_reg` oraz `XGBClassifier` oznaczanego jako `xgb`, które są zgodne z wyborem najlepszych modeli w [10], tworząc podstawę dla uczenia zespołowego. Te modele są następnie łączone w `VotingClassifier`, oznaczanym jako `voting_clf`, z miękkim głosowaniem

(`voting='soft'`), co pozwala na integrację predykcji prawdopodobieństw z każdego modelu, dokładnie tak jak w artykule Voting Classifier opierał się na kombinacji wyników dla osiągnięcia wysokiej dokładności.

Trening `voting_clf` odbywa się na `X_train` i `y_train`, co odpowiada etapowi uczenia w artykule, gdzie modele były trenowane na danych tekstowych po preprocessingu, a w naszym przypadku dane są już w odpowiedniej formie dzięki wspólnemu pipeline. Po wytrenowaniu `voting_clf` jest gotowy do predykcji na `X_test` (macierz cech testowych), a metoda zwraca `voting_clf`, który zawiera ustawienia i wyniki uczenia, wraz z `X_test` oraz `y_test` (wektor etykiet testowych z wartościami 0 lub 1), co umożliwia ocenę jakości modelu, podobnie jak w artykule wyniki klasyfikacji służyły do weryfikacji skuteczności, osiągając wspomnianą dokładność.

Mamy dokładnie to samo podejście co w [10], opierając się na Voting Classifier z identycznymi modelami `MLPClassifier`, `LogisticRegression` i `XGBClassifier` oraz `soft` głosowaniem, z tą różnicą, że pomijamy preprocessing w kodzie na rzecz uniwersalnego przygotowania danych, co nie zmienia istoty ensemble methods opisanych w artykule.

metoda11

`metoda11` – Voting Classifier z Random Forest, Logistic Regression i AdaBoost po selekcji cech.

W artykule [14] opisano zastosowanie ensemble methods, w tym Voting Classifier z `RandomForestClassifier`, `LogisticRegression` i `AdaBoostClassifier`, do wykrywania fałszywych wiadomości na podstawie cech tekstowych, osiągając wysoką dokładność dzięki kombinacji predykcji.

Nasza implementacja `metoda11` całkiem reprezentuje to podejście, opierając się na identycznym zestawie modeli i idei łączenia ich predykcji w celu poprawy klasyfikacji binarnej fałszywych wiadomości. W artykule wykorzystano cechy tekstowe do treningu modeli, co znajduje swoje odzwierciedlenie w naszym procesie, gdzie dane wejściowe `X_train` (cechy treningowe) i `X_test` (cechy testowe) są poddawane normalizacji za pomocą `MinMaxScaler`, tworząc `X_train_scaled` i `X_test_scaled`, aby zapewnić nieujemne wartości wymagane do dalszej selekcji cech, co jest wspólnym krokiem przygotowania danych do klasyfikacji.

Następnie stosujemy selekcję cech za pomocą `SelectKBest` z testem Chi-kwadrat, przekształcając

`X_train_scaled` w `X_train_selected` oraz `X_test_scaled` w `X_test_selected`, co pozwala skoncentrować się na najważniejszych cechach, podobnie jak w artykule dane tekstowe były optymalizowane przed klasyfikacją. Proces kontynuowany jest przez zdefiniowanie tych samych modeli bazowych co w artykule: `RandomForestClassifier` oznaczanego jako `rf`, `LogisticRegression` oznaczanego jako `lr` oraz `AdaBoostClassifier`

oznaczanego jako `adb`, które są zgodne z wyborem modeli w [14], tworząc podstawę dla uczenia zespołowego.

Te modele są następnie łączone w `VotingClassifier`, oznaczanym jako `voting_clf`, z miękkim głosowaniem (`voting='soft'`), co pozwala na integrację predykcji prawdopodobieństw z każdego modelu, dokładnie tak jak w artykule `ensemble methods` opierały się na kombinacji wyników dla zwiększenia dokładności. Trening `voting_clf` odbywa się na `X_train_selected` i `y_train` (etykiety treningowe), co odpowiada etapowi uczenia w artykule, gdzie modele były trenowane na przygotowanych danych tekstowych.

Po wytrenowaniu `voting_clf` jest gotowy do predykcji na `X_test_selected`, a metoda zwraca `voting_clf`, który zawiera ustawienia i wyniki uczenia, wraz z przetworzonymi danymi testowymi `X_test_selected` w formie macierzy oraz `y_test` (etykiety testowe jako wektor z wartościami 0 i 1), co umożliwia ocenę jakości modelu, podobnie jak w artykule wyniki klasyfikacji służyły do weryfikacji skuteczności. Mamy dokładnie to samo podejście co w [14], opierając się na `Voting Classifier` z tymi samymi modelami bazowymi i miękkim głosowaniem, z dodaną normalizacją i selekcją cech jako częścią naszego pipeline, co wzmacnia zgodność z ideą `ensemble methods` opisaną w artykule.

metoda13

`metoda13` – `Stacking` z `SMOTETomek`, wykorzystujący `Random Forest`, `Gradient Boosting`, `XGBoost` i `Logistic Regression` jako meta-model.

`Metoda13` trenuje model zespołowy `stacking` z wykorzystaniem `Random Forest`, `Gradient Boosting` i `XGBoost` jako klasyfikatorów bazowych, z `Logistic Regression` jako meta-ucznem, po uprzednim zbalansowaniu danych za pomocą `SMOTETomek`. Wyniki są oceniane na danych testowych za pomocą raportu klasyfikacji.

Nasza implementacja `metoda13` opiera się na `ensemble learningu` typu `stacking`, wspartym metodą `SMOTETomek` do obsługi problemu niezrównoważonych klas. Dane wejściowe, czyli `X_train` (cechy treningowe) i `y_train` (etykiety treningowe), są najpierw zbalansowane za pomocą `SMOTETomek`, co łączy nadpróbkowanie `SMOTE` z usuwaniem próbek metodą `Tomek Links`, tworząc zbalansowany zbiór `X_train_resampled` i `y_train_resampled`.

Następnie definiowane są trzy klasyfikatory bazowe: `RandomForestClassifier`, `GradientBoostingClassifier` oraz `XGBClassifier`. Te modele są łączone w `StackingClassifier` z `LogisticRegression` jako meta-uczeniem, stosując 5-krotną walidację krzyżową. Model jest trenowany na zbalansowanych danych, a predykcje `y_pred` generowane na `X_test` są oceniane za pomocą `classification_report`, który dostarcza metryki takie jak precyzja, czułość i `F1` dla `y_test`.

Metoda zwraca wytrenowany model `stacking_model`, `X_test` oraz `y_test`, umożliwiając dalszą analizę. Nasza implementacja zachowuje kluczowe cechy podejścia opisanego w artykule [8], koncentrując się na `stackingu` i obsłudze niezrównoważonych klas.

metoda14

metoda14 – Stacking z Random Forest, AdaBoost, SVC i Logistic Regression jako meta-model.

Metoda14 opisana w artykule [9] trenuje model zespołowy stacking, wykorzystujący Random Forest, AdaBoost i SVC jako klasyfikatory bazowe, z Logistic Regression jako meta-ucznem.

Nasza implementacja metoda14 bazuje na koncepcji ensemble learningu opisanej w artykule [9], w szczególności na wykorzystaniu modeli takich jak AdaBoost i SVM, ale stosuje bardziej zaawansowane podejście stackingowe. Dane wejściowe `X_train` (cechy treningowe) i `y_train` (etykiety treningowe) oraz `X_test` (cechy testowe) i `y_test` (etykiety testowe) zakładają, że tekst został wcześniej przetworzony na osadzenia wektorowe. Funkcja `preprocess_text` oczyszcza tekst, usuwając URL-e, znaki specjalne, pojedyncze litery, liczby, konwertując na małe litery i stosując stemming za pomocą `SnowballStemmer`.

Najpierw definiowane są klasyfikatory bazowe: `RandomForestClassifier`, `AdaBoostClassifier` oraz `SVC`. Następnie `StackingClassifier` łączy je z `LogisticRegression` jako metauczeniem, stosując 5-krotną walidację krzyżową. Model jest trenowany na `X_train` i `y_train`, a po zakończeniu zwraca wytrenowany `stack_model`, `X_test` i `y_test`, umożliwiając dalszą ewaluację. Nasza implementacja nie generuje cech w kodzie i pomija bezpośrednią ocenę wyników, ponieważ `X_test` i `y_test` zostaną użyte do predykcji i ewaluacji poza funkcją.

Metoda16

Metoda16 – Stacking z meta-cechami: Random Forest, SVC z Logistic Regression. Metoda ta wykorzystuje podejście stackingu, w którym modele bazowe generują meta-cechy, a meta-klasyfikator na ich podstawie dokonuje ostatecznej predykcji.

W oryginalnym podejściu opisanym w artykule [11] proces klasyfikacji przebiega w kilku etapach. Najpierw dane tekstowe są przetwarzane za pomocą biblioteki NLTK, co obejmuje usuwanie stop-słów, stemming i lematyzację, a następnie konwertowane na cechy numeryczne za pomocą metod takich jak TF-IDF i `CountVectorizer`. Następnie stosuje się ensemble learning w formie stackingu, gdzie modele bazowe, takie jak Random Forest i SVM, generują predykcje prawdopodobieństw klas dla danych treningowych za pomocą walidacji krzyżowej. Te predykcje są używane jako meta-cechy do treningu meta-klasyfikatora, którym jest regresja logistyczna. Modele bazowe są następnie trenowane na pełnym zbiorze treningowym, a meta-cechy dla danych testowych są generowane na podstawie ich predykcji. Ostatecznie meta-klasyfikator dokonuje predykcji na meta-cechach testowych, a wyniki są oceniane za pomocą metryk takich jak dokładność i raport klasyfikacji.

Nasza implementacja metoda16 zachowuje kluczowe wspólne cechy tego podejścia, opierając się na podobnej strukturze stackingu i wykorzystaniu ensemble learningu. W naszej wersji dane wejściowe, czyli

`X_train` reprezentujące cechy treningowe oraz `X_test` odpowiadające cechom testowym, są już przygotowane jako osadzenia wektorowe w ramach wspólnego pipeline, co różni się od preprocessingu NLTK i metod TF-IDF/CountVectorizer stosowanych w artykule, ale zachowuje ideę przekształcenia danych do postaci numerycznej przed klasyfikacją. Proces zaczyna się od inicjalizacji modeli bazowych: `RandomForestClassifier` oznaczanego jako `rf_model` oraz `SVC` z liniowym jądrem oznaczanego jako `svc_model`, co odpowiada użyciu Random Forest i SVM w artykule.

Następnie generujemy meta-cechy dla danych treningowych, czyli `meta_features_train`, poprzez 5-krotną walidację krzyżową z rozwarstwieniem (`StratifiedKfold`), gdzie `rf_preds_train` i `svc_preds_train` to predykcje prawdopodobieństw odpowiednio od `rf_model` i `svc_model` na `X_train` i `y_train` (etykiety treningowe), łączone w macierz `meta_features_train`, co jest zgodne z podejściem artykułu, w którym predykcje z walidacji krzyżowej stają się meta-cechami. Te meta-cechy są używane do treningu meta-klasyfikatora `LogisticRegression`, oznaczonego jako `meta_model`, na `meta_features_train` i `y_train`, co odzwierciedla etap treningu regresji logistycznej w artykule.

Po tym kroku modele bazowe `rf_model` i `svc_model` są trenowane na pełnym zestawie `X_train` i `y_train`, podobnie jak w artykule, gdzie następuje trening na całym zbiorze treningowym. Dla danych testowych generujemy meta-cechy `meta_features_test`, obliczając `rf_preds_test` i `svc_preds_test` jako predykcje prawdopodobieństw od `rf_model` i `svc_model` na `X_test`, a następnie łącząc je w macierz `meta_features_test`, co odpowiada generowaniu meta-cech testowych w artykule. Ostatecznie `meta_model` dokonuje predykcji `y_pred` na `meta_features_test`, a wyniki są oceniane za pomocą `classification_report` i `accuracy_score` na podstawie `y_test` (etykiety testowe), co jest wspólnym elementem z artykułem, gdzie również stosowano metryki takie jak dokładność i raport klasyfikacji. Metoda zwraca `meta_model`, `meta_features_test` oraz `y_test`, co umożliwia dalszą analizę.

2.7.4. Modele zoptymalizowane

Metoda7

Metoda7 – Random Forest z Monte Carlo Dropout i heurystyczną obróbką post-procesową. Metoda7 opiera się na Random Forest jako bazowym klasyfikatorze.

Nasza implementacja w `metoda7` w pełni bazuje na tym podejściu, zachowując jego kluczowe elementy, ale dostosowuje je do dostępnych zasobów. Zamiast ensemble transformerów i NewsBERT, wykorzystujemy pojedynczy model Random Forest `RandomForestClassifier`, co wynika z faktu, że dane wejściowe `X_train` (cechy treningowe) i `X_test` (cechy testowe) są już przekształcone w osadzenia wektorowe, eliminując potrzebę tokenizacji czy generowania cech przez modele językowe.

Proces rozpoczyna się od przygotowania danych, gdzie `X_train` i `y_train` (etykiety treningowe) są konwertowane na `train_embeddings` i `train_labels`, a `X_test` oraz `y_test` (etykiety testowe) na `test_embeddings`. Dane są standaryzowane przed dalszym przetwarzaniem. Model Random Forest jest trenowany na `train_embeddings` i `train_labels`, co odpowiada etapowi uczenia w artykule, choć bez użycia transformerów. Predykcje na `test_embeddings` są generowane za pomocą symulacji Monte Carlo Dropout (`monte_carlo_dropout_inference`), która w 50 iteracjach próbkując 80% drzew, tworzy `mean_preds` (średnie prawdopodobieństwa), `uncertainty` (niepewność) oraz etykiety `pred_labels`. Ten krok odwzorowuje oszacowanie niepewności z artykułu, dostosowane do Random Forest.

Wprowadzamy meta-atrybuty w `df` jako kolumnę "source", domyślnie ustawioną na "unknown", które są przetwarzane w `compute_meta_attribute_probabilities` do `attribute_probs`. Te dane są wykorzystywane w heurystycznej obróbce post-procesowej (`heuristic_post_processing`) z progiem 0.9, korygując `pred_labels` na `final_predictions`, podobnie jak moduł SFTN w artykule. Jednak wpływ meta-atrybutów jest ograniczony z powodu ich uproszczenia w porównaniu do różnorodnych źródeł w oryginalnym podejściu.

Wyniki modelu są oceniane na `y_test` za pomocą `classification_report`. Funkcja zwraca `model`, `X_test` i `y_test`, umożliwiając dalszą analizę, co jest zgodne z podejściem artykułu.

2.8. IMPLEMENTACJA ANALIZY WYNIKÓW

W podanym rozdziale opisujemy końcowy etap implementacji, wspólny dla wszystkich implementowanych rozwiązań metod uczenia zespołowego, obejmujący ewaluację modeli oraz analizę i wizualizację uzyskanych wyników. Ten proces został zaimplementowany w dwóch głównych etapach i koncentruje się na porównaniu skuteczności klasyfikacji dla metod naukowych zarówno jak autorskich.

Dane wejściowe do analizy wyników są generowane przez każdą z 20 metod (`metoda1` do `metoda20`), które zwracają wytrenowany model oraz dane testowe w następującej postaci:

- **model** – wytrenowany klasyfikator, reprezentujący implementację danej metody, gotowy do generowania predykcji.
- **X_test** – tablica `ndarray` zawierająca cechy zbioru testowego, które mogą być wektorami TF-IDF lub osadzeniami kontekstowymi z BERT, RoBERTa czy Sentence Transformers, wykorzystywanymi do oceny modelu.
- **y_test** – lista lub tablica `ndarray` zawierająca etykiety binarne zbioru testowego: 0 dla fałszywych wiadomości, 1 dla prawdziwych; służące jako punkt odniesienia dla predykcji.

Te dane są przekazywane do dalszej ewaluacji i analizy, a ich struktura jest spójna dla wszystkich metod, co umożliwia standaryzowane porównanie wyników.

Pierwszym krokiem jest ewaluacja modeli przy użyciu funkcji `evaluate_model`, wywoływanej dla każdej metody po jej uruchomieniu. Poniżej znajduje się szczegółowy opis parametrów przekazywanych do funkcji `evaluate_model`:

- **model** – zwrócony parametr wyniku metody, szczegółowo opisaliśmy poprzednio.
- **X_test** – zwrócony parametr wyniku metody, szczegółowo opisaliśmy poprzednio.
- **y_test** – zwrócony parametr wyniku metody, szczegółowo opisaliśmy poprzednio.
- **run_type** – identyfikator typu uruchomienia, używany do oznaczania wyników w plikach wyjściowych.
- **output_path** – ścieżka do katalogu, w którym zapisywane są wyniki, umożliwiającą organizację danych wyjściowych.
- **start_time** – znacznik czasu rozpoczęcia uruchomienia, służący do obliczania całkowitego czasu wykonania.

Wspominając o szczegółach tego kroku, funkcja rozpoczyna od przeprowadzenia walidacji krzyżowej (StratifiedKFold, 5 podziałów, `random_state=42`), obliczając średnią dokładność (`cv_accuracy_mean`) oraz odchylenie standardowe (`cv_accuracy_std`). Następnie model generuje predykcje prawdopodobieństw (`predict_proba` dla modeli scikit-learn lub `predict` dla modeli Keras), które są konwertowane na etykiety binarne przy progu 0.5. Obliczane są metryki oceny, w tym:

- **Accuracy**
- **Precision**
- **Recall**
- **F1-Score**
- **ROC-AUC**
- **MCC**
- **Log Loss**
- **Cohen's Kappa**

Dodatkowo mierzony jest czas wykonania (`Execution Time`), a wyniki walidacji krzyżowej są dołączane do metryk. Wszystkie wartości są zapisywane do pliku CSV w formacie `runX_results.csv`, a krzywa Precision-Recall (`precision_recall_curve`) jest generowana i zapisywana jako `runX_pr_curve.csv`. Na koniec metryki są wyświetlane w konsoli, co pozwala na szybką weryfikację skuteczności modelu. W tym procesie model jest używany do generowania predykcji, `X_test` i `y_test` służą do obliczania metryk oceny i walidacji krzyżowej, `run_type` oraz `output_path` określają format i lokalizację zapisu wyników, a `start_time` pozwala zmierzyć czas wykonania. Kluczowe parametry wykorzystane w procesie ewaluacji modeli obejmują:

- **n_splits=5** – liczba podziałów w walidacji krzyżowej, określająca, ile razy dane są dzielone na zestawy treningowe i testowe w celu obliczenia średniej dokładności.
- **random_state=42** – ziarno losowości, zapewniające odtwarzalność podziałów danych w walidacji krzyżowej i podziale treningowym/testowym.
- **threshold=0.5** – próg prawdopodobieństwa, powyżej którego predykcje są klasyfikowane jako klasa pozytywna (1), stosowany w konwersji prawdopodobieństw na etykiety binarne.
- **zero_division=0** – wartość domyślna dla obsługi dzielenia przez zero w metrykach precyzji i czułości, zapobiegająca błędom przy braku predykcji jednej z klas.
- **flatten=True** – parametr określający, czy predykcje z modeli Keras powinny być spłaszczane do jednowymiarowej tablicy, ułatwiając dalsze przetwarzanie.

Drugim krokiem jest analiza i wizualizacja wyników, realizowana przez skrypt generujący wykresy porównawcze. Skrypt ten przetwarza wyniki z różnych folderów `results_1_classic`, `results_2_few_shot` oraz `results_3_no_shot`, obejmujących uruchomienia metod od 1 do 20 oraz dodatkowe pliki porównawcze. Wyniki ze wszystkich plików są łączone w jedną tabelę danych z dodaną kolumną Metoda wskazującą numer metody, których posiadamy 20. Dla metryki Execution Time (s) usuwane są wartości odstające. Następnie generowane są wykresy słupkowe dla wybranych metryk (Accuracy, Execution Time (s), Log Loss), z różnym kolorem dla danych metod autorskich oznaczonych na czerwono i naukowych na niebiesko. Dodatkowo, dla wszystkich uruchomień generowany jest wykres krzywych Precision-Recall na podstawie danych z plików `runX_pr_curve.csv`, z legendą wskazującą metodę i kolorem odróżniającym źródła danych. Poniżej są opisane parametry wykorzystane w procesie analizy i wizualizacji wyników:

- **IQR=1.5** – mnożnik zakresu międzykwartylowego, określający granice usuwania wartości odstających dla metryki Execution Time (s).
- **figsize=(10, 6)** – rozmiar wykresów słupkowych w calach: szerokość 10, wysokość 6; zapewniający czytelność wizualizacji.
- **figsize=(12, 8)** – rozmiar wykresu krzywych Precision-Recall w calach: szerokość 12, wysokość 8; dostosowany do większej liczby linii.
- **color=['red', 'skyblue']** – kolory słupków na wykresach, gdzie czerwony oznacza dane autorskich metod, a niebieski dane naukowych implementacji.

3. WŁASNE METODY

Autorskie metody zostały opracowane na bazie wyników badań, które szczegółowo przeanalizowały i określiły najlepsze podejścia pod względem **Accuracy**, **Log Loss** oraz **Execution Time**. Bazując na zaobserwowanych wzorcach oraz kluczowych wnioskach płynących z tych analiz, stworzyliśmy unikalne rozwiązania, które łączą skuteczność i wydajność, dostosowując je do specyficznych wymagań i wyzwań problematyki wykrywania **fake news**.

3.1. PIERWSZA METODA (METODA17)

3.1.1. Pomysł na metodę

Metoda Metoda17 została zainspirowana wcześniejszym podejściem z Metoda10, które wykorzystuje VotingClassifier łączący różne modele, takie jak MLPClassifier, Logistic Regression i XGBoost. W Metoda10 szczególną uwagę poświęcono synergii modeli liniowych i nieliniowych, co pozwoliło uzyskać wysoką stabilność wyników dzięki miękkiemu głosowaniu. Jednocześnie Metoda6 dostarczył inspiracji w postaci użycia elastycznych sieci neuronowych jako narzędzi do analizy tekstu, szczególnie w pracy z osadzeniami BERT.

W Metoda17 skoncentrowano się na dynamicznym dostosowywaniu wag próbek, co pozwala modelowi skupić się na trudniejszych przykładach. Wykorzystano tu architekturę Sequential, która dzięki regularizacji L2 i Dropout radzi sobie z problemem przeuczenia. Warstwy BatchNormalization i LeakyReLU zapewniają stabilność i efektywność trenowania. Podobnie jak w Metoda10, waga danych i ich trudność odgrywają kluczową rolę, jednak Metoda17 idzie krok dalej, umożliwiając iteracyjne dostosowanie wag na podstawie błędów popełnianych w każdej iteracji. Ta iteracyjność była inspirowana sukcesem VotingClassifier w Metoda10, który również uczył się z różnych perspektyw dzięki modelom bazowym.

3.1.2. Wejściowe parametry

Metoda train_run17 przyjmuje następujące parametry wejściowe:

1. **X_train**: Macierz danych treningowych, gdzie każdy wiersz reprezentuje jedną próbkę, a kolumny odpowiadają cechom. Rozmiar: $n \times mn \times mn \times m$, gdzie nnn to liczba próbek, a mmm to liczba cech.

2. **y_train**: Wektor etykiet dla danych treningowych. Rozmiar: nnn, gdzie nnn to liczba próbek w danych treningowych.
3. **X_test**: Macierz danych testowych, analogiczna do X_train, ale używana do oceny modelu.
4. **y_test**: Wektor etykiet dla danych testowych.
5. **n_models**: Liczba iteracji, czyli modeli, które będą trenowane w celu zwiększenia skuteczności klasyfikacji dzięki dynamicznemu ważeniu próbek.

3.1.3. Wyjściowe parametry

Metoda zwraca następujące elementy:

1. **model**: Ostateczny model wytrenowany po n_models iteracjach.
2. **X_test**: Macierz danych testowych, identyczna z wejściowym X_test.
3. **y_test**: Wektor etykiet testowych, identyczny z wejściowym y_test.

3.1.4. Algorytm

1. Dla wszystkich próbek w danych treningowych wagi są inicjalizowane jako 1. Każda próbka na początku ma równą wagę, co zapewnia równoważne traktowanie wszystkich przykładów w początkowym etapie treningu. Wagi te są przechowywane w wektorze **sample_weights** o długości równej liczbie próbek w zbiorze treningowym.
2. Iteracje modeli:
 - W każdej iteracji tworzony jest model sieci neuronowej typu **Sequential**.
 - Architektura modelu składa się z następujących warstw:
 - **Pierwsza warstwa ukryta**:
 - Warstwa **Dense** z 128 neuronami, regularizacją **L2** ($\lambda = 0.01$) i bez funkcji aktywacji.
 - Warstwa **BatchNormalization**.
 - Funkcja aktywacji **LeakyReLU** ($\alpha = 0.1$).
 - Warstwa **Dropout** ($p = 0.2$).
 - **Druga warstwa ukryta**:
 - Warstwa **Dense** z 64 neuronami, regularizacją **L2** ($\lambda=0.01$) i bez funkcji aktywacji.
 - Warstwa **BatchNormalization**.
 - Funkcja aktywacji **LeakyReLU** ($\alpha=0.1$).
 - Warstwa **Dropout** ($p=0.2$).
 - **Warstwa wyjściowa**:
 - Warstwa **Dense** z 1 neuronem i funkcją aktywacji **sigmoid**, która polega na dostosowaniu do problemów binarnej klasyfikacji.
 - Model jest kompilowany z następującymi ustawieniami:

- Optymalizator: **Adam (learning rate=0.001)**.
 - Funkcja kosztu: **binary_crossentropy**.
 - Metryka: **Accuracy**.
3. Model jest trenowany przez 10 epok z użyciem rozmiaru batcha równego 64. Wagi próbek **sample_weight** są wykorzystywane do uwzględnienia trudności klasyfikacji poszczególnych przykładów.
 4. Aktualizacja wag próbek
 - Po każdej iteracji modelu obliczane są predykcje na danych treningowych.
 - Różnica absolutna między rzeczywistymi etykietami **y_train**, a przewidywaniami modelu **y_pred** jest dodawana do wag próbek, zwiększając wagi trudnych przykładów.
 5. Po zakończeniu **n_models** iteracji, ostateczny model oraz dane testowe **X_test, y_test** są zwracane.

3.1.5. Kod

```

1 sample_weights = np.ones(len(y_train))
2 for _ in range(n_models):
3     model = Sequential([
4         Dense(
5             128,
6             input_dim=X_train.shape[1],
7             activation=None,
8             kernel_regularizer=l2(0.01)
9         ),
10        BatchNormalization(),
11        LeakyReLU(alpha=0.1),
12        Dropout(0.2),
13        Dense(64, activation=None, kernel_regularizer=l2(0.01)),
14        BatchNormalization(),
15        LeakyReLU(alpha=0.1),
16        Dropout(0.2),
17        Dense(1, activation='sigmoid')
18    ])
19    model.compile(
20        optimizer=Adam(learning_rate=0.001),
21        loss='binary_crossentropy',
22        metrics=['accuracy']
23    )
24    model.fit(
25        X_train,
26        y_train,
27        epochs=10,
28        batch_size=64,

```

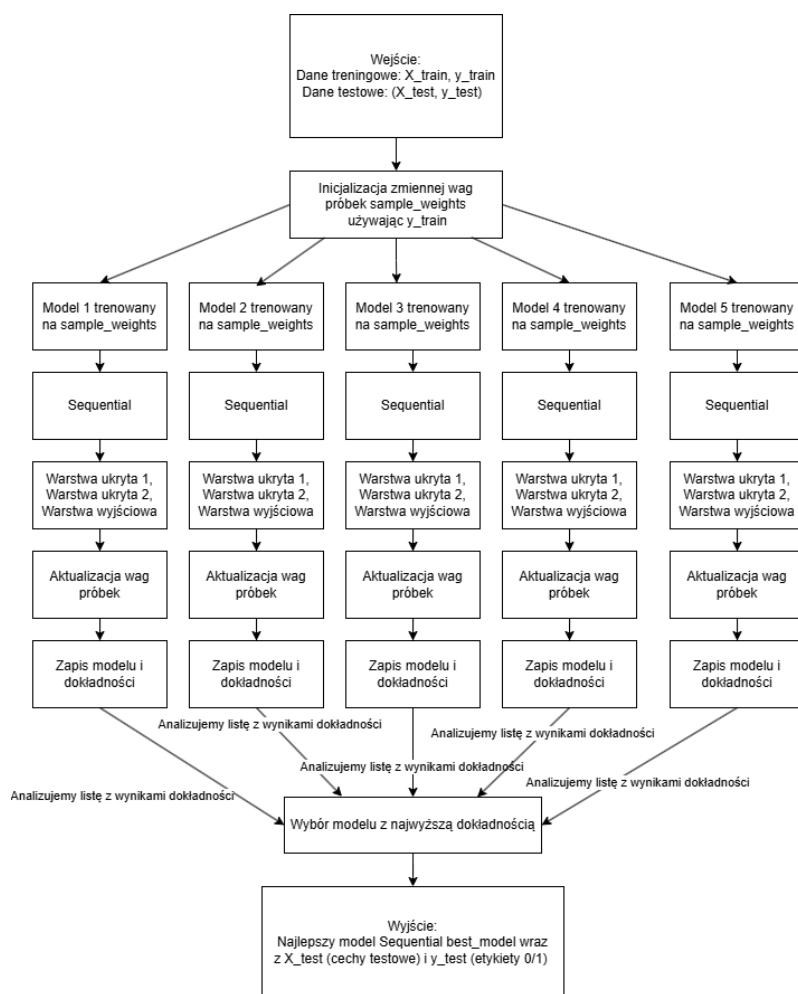
```

29         sample_weight=sample_weights ,
30         verbose=0
31     )
32     y_pred = model.predict(X_train)
33     sample_weights += np.abs(y_train - y_pred.squeeze())
34     return model, X_test, y_test

```

Listing 3.1: Kod do 17 metody

3.1.6. Schemat blokowy



Rys. 3.1: Schemat metody 17.

3.1.7. Właściwości analityczne

1. Efektywność:

- Metoda umożliwia poprawę jakości klasyfikacji poprzez iteracyjne skupianie się na trudniejszych przykładach.
- **Regularizacja L2** oraz mechanizmy **BatchNormalization** i **Dropout** zmniejszają ryzyko przeuczenia.

2. **Elastyczność:** architektura modelu i sposób ważenia próbek pozwalają na zastosowanie metody w różnych problemach klasyfikacyjnych, szczególnie w przypadkach nierównomiernych danych.
3. **Stabilność:** iteracyjne uczenie z dynamicznym ważeniem próbek poprawia spójność wyników i zmniejsza wpływ szumu w danych.
4. **Złożoność obliczeniowa:** złożoność trenowania wzrasta liniowo wraz z liczbą iteracji (n_models), co pozwala na łatwe dostosowanie metody do dostępnych zasobów obliczeniowych.
5. **Wydaźność:** optymalizator Adam zapewnia szybkie i stabilne uczenie, szczególnie w połączeniu z BatchNormalization.

3.2. DRUGA METODA (METODA18)

3.2.1. Pomysł na metodę

W Metoda18 integracja Gradient Boosting i CatBoost została wzbogacona o meta-model w postaci regresji logistycznej. Metoda ta jest wynikiem kombinacji idei pochodzących z Metoda10 oraz Metoda13. W Metoda10 kluczowym aspektem było miękkie głosowanie, które pozwalało uwzględniać probabilistyczne wyniki modeli bazowych w końcowych predykcjach. Z kolei Metoda13 skupił się na użyciu stacking, gdzie Gradient Boosting, XGBoost i Random Forest były łączone przy użyciu regresji logistycznej jako meta-modelu.

Metoda18 czerpie z Metoda13 mechanizm stacking, jednak zamiast wielu modeli bazowych opiera się na dwóch potężnych algorytmach boostingu – Gradient Boosting i CatBoost. Wybór tych modeli wynika z ich zdolności do radzenia sobie z różnymi typami danych i ich nierównowagą. Regresja logistyczna jako meta-model umożliwia skuteczną integrację wyników, co zostało wcześniej zademonstrowane w Metoda13. W porównaniu do Metoda10, Metoda18 idzie krok dalej, stawiając na większą precyzję w meta-modelu poprzez wykorzystanie pełnych rozkładów prawdopodobieństw z boostingu.

3.2.2. Wejściowe parametry

Metoda `train_run18` przyjmuje następujące parametry wejściowe:

1. **X_train :** Macierz danych treningowych, gdzie każdy wiersz to jedna próbka, a kolumny to cechy. Rozmiar: $n \times m$, gdzie m to liczba próbek, a n to liczba cech.
2. **y_train :** Wektor etykiet dla danych treningowych. Rozmiar: n , gdzie n to liczba próbek w danych treningowych.
3. **X_test :** Macierz danych testowych, analogiczna do X_train , używana do oceny modelu.
4. **y_test :** Wektor etykiet testowych, odpowiadający danym testowym.

3.2.3. Wyjściowe parametry

Metoda zwraca następujące elementy:

1. **meta_model**: Meta-model wytrenowany na danych pochodzących z predykcji modeli **Gradient Boosting** i **CatBoost**.
2. **meta_features_test**: Macierz meta-cech, zbudowana na podstawie predykcji modeli bazowych dla danych testowych.
3. **y_test**: Wektor etykiet testowych, identyczny z wejściowym **y_train**.

3.2.4. Algorytm

1. Trenowanie modeli bazowych:
 - **Gradient Boosting**:
 - Model **GradientBoostingClassifier** jest inicjalizowany z **100 estymatorami** i **stanem losowości (random_state) 42**.
 - Model jest trenowany na danych treningowych **X_train** i **y_train**.
 - **CatBoost**:
 - Model **CatBoostClassifier** jest inicjalizowany z **100 iteracjami** i **stanem losowości (random_state) 42**, a tryb **verbose** jest wyłączony.
 - Model jest trenowany na tych samych danych treningowych.
2. Generowanie predykcji dla danych treningowych:
 - Obliczane są prawdopodobieństwa klasy pozytywnej $P(y = 1 | x)$ dla obu modeli bazowych:
 - **gb_preds_train**: Predykcje Gradient Boosting.
 - **cat_preds_train**: Predykcje CatBoost.
 - Predykcje są używane do stworzenia macierzy meta-cech:
 - Kolumny to **gb_preds** i **cat_preds**.
3. Trenowanie meta-modelu:
 - Wytrenowany na danych **meta-cech**, **meta_features_train**, oraz rzeczywistych etykietach **y_train**.
 - Meta-modelem jest **regresja logistyczna**, **LogisticRegression**, która łączy predykcje obu modeli bazowych.
4. **Generowanie predykcji dla danych testowych**:
 - Obliczane są prawdopodobieństwa klasy pozytywnej dla Gradient Boosting i CatBoost:
 - **gb_preds_test**: Predykcje Gradient Boosting.
 - **cat_preds_test**: Predykcje CatBoost.
 - Tworzona jest macierz meta-cech dla danych testowych o analogicznej strukturze jak dla danych treningowych.

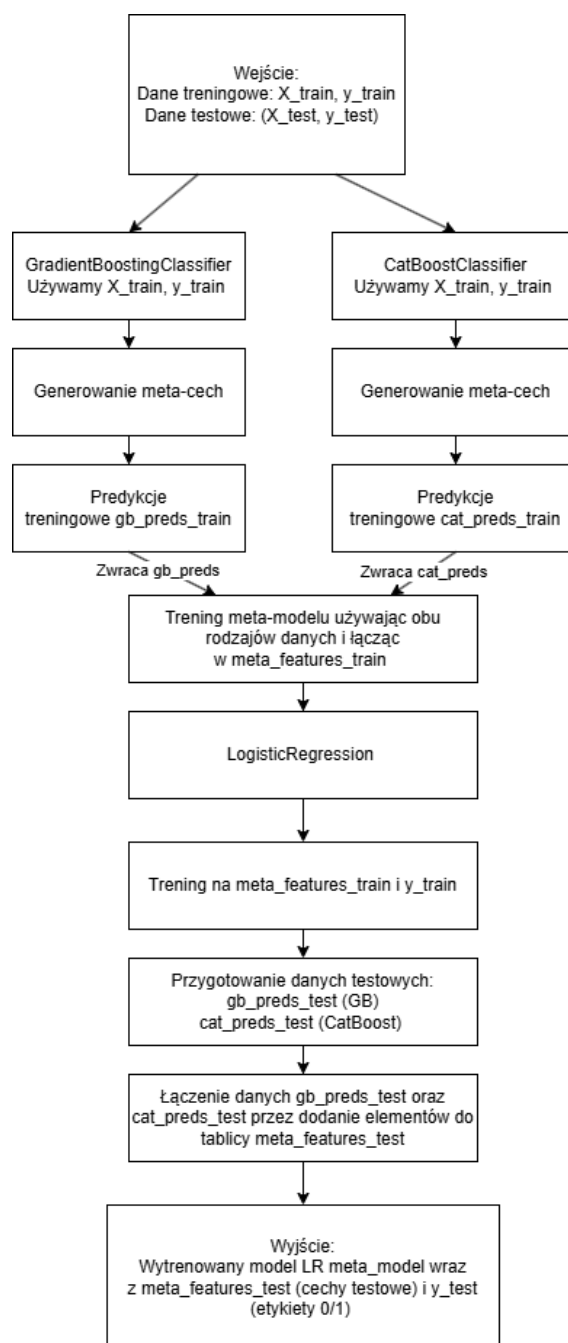
5. Zwracanie wyników: meta-model, macierz meta-cech dla testu oraz etykiety testowe są zwracane.

3.2.5. Kod

```
1 gb_clf = GradientBoostingClassifier(  
2     n_estimators=100,  
3     random_state=42  
4 ) # Gradient Boosting  
5 cat_clf = CatBoostClassifier(  
6     iterations=100,  
7     verbose=0,  
8     random_state=42  
9 ) # CatBoost  
10 gb_clf.fit(X_train, y_train)  
11 cat_clf.fit(X_train, y_train)  
12 gb_preds_train = gb_clf.predict_proba(X_train)[: , 1]  
13 cat_preds_train = cat_clf.predict_proba(X_train)[: , 1]  
14 meta_features_train = pd.DataFrame({  
15     'gb_preds': gb_preds_train ,  
16     'cat_preds': cat_preds_train  
17 })  
18 meta_model = LogisticRegression()  
19 meta_model.fit(meta_features_train, y_train)  
20 gb_preds_test = gb_clf.predict_proba(X_test)[: , 1]  
21 cat_preds_test = cat_clf.predict_proba(X_test)[: , 1]  
22 meta_features_test = pd.DataFrame({  
23     'gb_preds': gb_preds_test ,  
24     'cat_preds': cat_preds_test  
25 })  
26 return meta_model, meta_features_test, y_test
```

Listing 3.2: Kod do 18 metody

3.2.6. Schemat blokowy



Rys. 3.2: Schemat metody 18.

3.2.7. Właściwości analityczne

1. Efektywność:

- Kombinacja **Gradient Boosting** i **CatBoost** pozwala na efektywne uchwycenie zarówno liniowych, jak i nieliniowych zależności w danych.
- Meta-model optymalizuje końcowe wyniki poprzez fuzję predykcji z modeli bazowych.

2. **Elastyczność:** metoda może być zastosowana w różnych problemach klasyfikacyjnych, szczególnie w przypadkach, gdzie różne modele bazowe wykazują różne mocne strony.
3. **Stabilność:** meta-model działa jako mechanizm konsolidujący, zmniejszając wariancję predykcji i poprawiając stabilność wyników.
4. **Złożoność obliczeniowa:**
 - **Gradient Boosting** i **CatBoost** mają złożoność liniową względem liczby próbek i estymatorów, co pozwala na efektywne przetwarzanie dużych zbiorów danych.
 - Dodanie meta-modelu zwiększa nieznacznie złożoność, ale poprawia jakość wyników.
5. **Wydaźność:** CatBoost jest zoptymalizowany pod kątem szybkiego trenowania, a Gradient Boosting zapewnia interpretowalność wyników. Kombinacja tych dwóch modeli z meta-modelem znacząco poprawia skuteczność klasyfikacji.

3.3. TRZECIA METODA (METODA19)

3.3.1. Pomysł na metodę

Metoda19 jest efektem rozwoju pomysłów z Metoda5 i Metoda9, które opierały się na algorytmach drzewiastych i metodach integracji wyników. W Metoda5 zastosowano Random Forest i XGBoost jako modele bazowe, które następnie były integrowane za pomocą regresji logistycznej. W Metoda9 kluczową rolę odgrywał Random Forest jako jeden z głównych komponentów VotingClassifier, co podkreśliło jego efektywność w różnorodnych konfiguracjach.

W Metoda19 połączono Random Forest i Extra Trees, dwa modele bazowe opierające się na konstrukcji drzew decyzyjnych. Extra Trees, w przeciwieństwie do Random Forest, wprowadza dodatkowy element losowości w wyborze punktów podziału, co zwiększa różnorodność wyników. Inspiracja pochodząca z Metoda5 była widoczna w zastosowaniu regresji logistycznej jako meta-modelu, który skutecznie integruje predykcje obu modeli. Metoda19 dodatkowo rozwija te pomysły, kładąc nacisk na stabilność wyników dzięki różnorodności predykcji generowanych przez Random Forest i Extra Trees.

3.3.2. Wejściowe dane

Metoda train_run19 przyjmuje następujące parametry wejściowe:

1. **X_train:** Macierz danych treningowych, gdzie każdy wiersz to jedna próbka, a kolumny to cechy. Rozmiar: $\mathbf{n} \times \mathbf{m}$, gdzie $\mathbf{n}$ to liczba próbek, a $\mathbf{m}$ to liczba cech.
2. **y_train:** Wektor etykiet dla danych treningowych. Rozmiar: $\mathbf{n}$, gdzie $\mathbf{n}$ to liczba próbek w danych treningowych.
3. **X_test:** Macierz danych testowych, analogiczna do **X_train**, używana do oceny modelu.
4. **y_test:** Wektor etykiet testowych, odpowiadający danym testowym.

3.3.3. Wyjściowe dane

Metoda zwraca następujące elementy:

1. **meta_model**: Meta-model wytrenowany na danych pochodzących z predykcji modeli **Random Forest** i **Extra Trees**.
2. **meta_features_test**: Macierz meta-cech, zbudowana na podstawie predykcji modeli bazowych dla danych testowych.
3. **y_test**: Wektor etykiet testowych, identyczny z wejściowym **y_train**.

3.3.4. Algorytm

1. Trenowanie modeli bazowych:
 - **Random Forest**:
 - Model **RandomForestClassifier** jest inicjalizowany z **100 estymatorami** i **stanem losowości (random_state) 42**.
 - Model jest trenowany na danych treningowych **X_train** i **y_train**.
 - **Extra Trees**:
 - Model **ExtraTreesClassifier** jest inicjalizowany z **100 estymatorami** i **stanem losowości (random_state) 42**.
 - Model jest trenowany na tych samych danych treningowych.
2. Generowanie predykcji dla danych treningowych:
 - Obliczane są prawdopodobieństwa klasy pozytywnej $P(y = 1 | x)$ dla obu modeli bazowych:
 - **rf_preds_train**: Predykcje **Random Forest**.
 - **et_preds_train**: Predykcje **Extra Trees**.
 - Predykcje są używane do stworzenia macierzy meta-cech, gdzie kolumny to **rf_preds** i **et_preds**.
3. Trenowanie meta-modelu:
 - Wytrenowany na **danych meta-cech, meta_features_train**, oraz rzeczywistych etykietach **y_train**.
 - Meta-modelem jest **regresja logistyczna, LogisticRegression**, która łączy predykcje obu modeli bazowych.
4. Generowanie predykcji dla danych testowych:
 - Obliczane są prawdopodobieństwa klasy pozytywnej dla **Random Forest** i **Extra Trees**:
 - **rf_preds_test**: Predykcje **Random Forest**.
 - **et_preds_test**: Predykcje **Extra Trees**.
 - Tworzona jest **macierz meta-cech** dla danych testowych, analogiczna struktura jak dla danych treningowych.

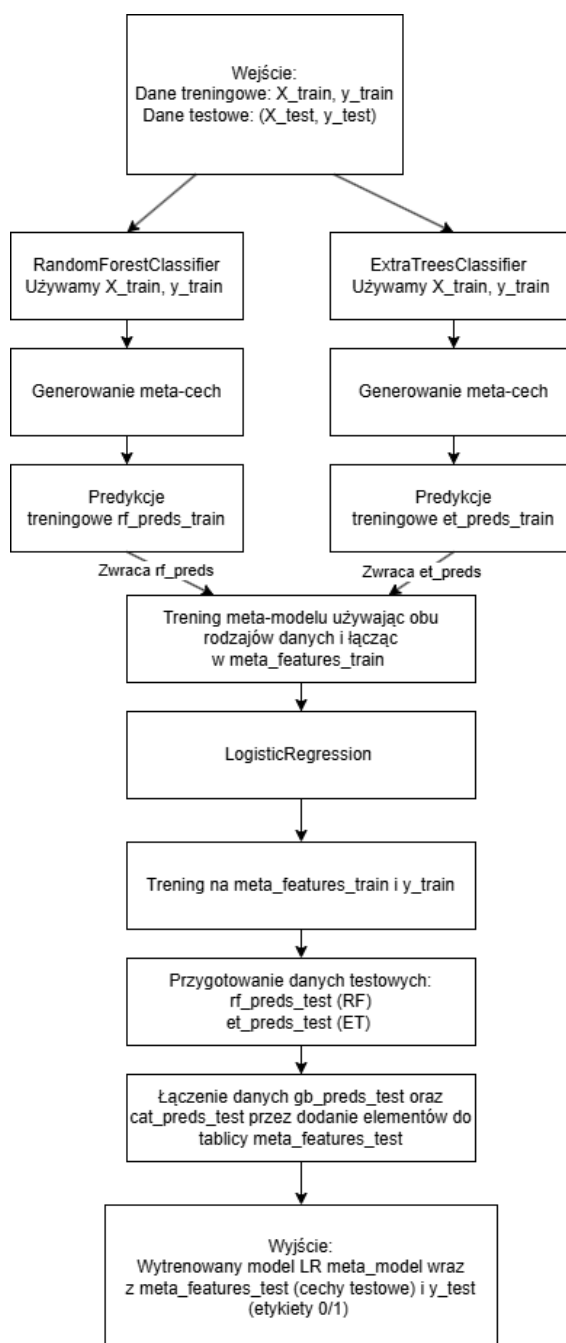
5. Zwracanie wyników: meta-model, macierz meta-cech dla testu oraz etykiety testowe są zwracane.

3.3.5. Kod

```
1 rf_clf = RandomForestClassifier(  
2     n_estimators=100,  
3     random_state=42  
4 ) # Random Forest  
5 et_clf = ExtraTreesClassifier(  
6     n_estimators=100,  
7     random_state=42  
8 ) # Extra Trees  
9 rf_clf.fit(X_train, y_train)  
10 et_clf.fit(X_train, y_train)  
11 rf_preds_train = rf_clf.predict_proba(X_train)[: , 1]  
12 et_preds_train = et_clf.predict_proba(X_train)[: , 1]  
13 meta_features_train = pd.DataFrame({  
14     'rf_preds': rf_preds_train ,  
15     'et_preds': et_preds_train  
16 })  
17 meta_model = LogisticRegression()  
18 meta_model.fit(meta_features_train, y_train)  
19 rf_preds_test = rf_clf.predict_proba(X_test)[: , 1]  
20 et_preds_test = et_clf.predict_proba(X_test)[: , 1]  
21 meta_features_test = pd.DataFrame({  
22     'rf_preds': rf_preds_test ,  
23     'et_preds': et_preds_test  
24 })  
25 return meta_model, meta_features_test, y_test
```

Listing 3.3: Kod do 19 metody

3.3.6. Schemat blokowy



Rys. 3.3: Schemat metody 19.

3.3.7. Właściwości analityczne

1. Efektywność:

- **Random Forest** i **Extra Trees** bazują na zbiorach drzew decyzyjnych, które są w stanie uchwycić zarówno zależności liniowe, jak i nieliniowe.
- Meta-model, w podanej implementacji **regresja logistyczna**, umożliwia dodatkową poprawę wyników dzięki łączeniu predykcji z modeli bazowych.

2. **Elastyczność:** metoda może być łatwo dostosowana do różnych problemów klasyfikacyjnych i różnorodnych zbiorów danych.
3. **Stabilność:**
 - Modele bazowe, takie jak **Random Forest** i **Extra Trees**, są naturalnie stabilne dzięki losowemu wybieraniu próbek i cech podczas tworzenia drzew.
 - Meta-model dodatkowo redukuje wariancję wyników, zwiększając spójność predykcji.
4. **Złożoność obliczeniowa:**
 - Zarówno **Random Forest**, jak i **Extra Trees** mają złożoność liniową względem liczby próbek i liczby estymatorów, co pozwala na efektywne przetwarzanie dużych zbiorów danych.
 - Regresja logistyczna jako meta-model jest szybka i wydajna, dzięki czemu metoda jest odpowiednia nawet dla dużych zbiorów danych.
5. **Wydajność:** **Random Forest** i **Extra Trees** są zoptymalizowane do przetwarzania dużych zbiorów danych i cech o różnym typie. Ich kombinacja pozwala na uzyskanie bardziej dokładnych wyników.

3.4. CZWARTA METODA (METODA20)

3.4.1. Pomysł na metodę

Metoda ta powstała na bazie Metoda13 oraz Metoda6, które dostarczyły inspiracji w zakresie stosowania zaawansowanych algorytmów boostingu i integracji wyników za pomocą meta-modeli. W Metoda13 Gradient Boosting i XGBoost były kluczowymi elementami w ramach stacking, co wykazało, że boostingi są wyjątkowo skuteczne w integracji wyników. Z kolei Metoda6 skupił się na integracji osadzeń BERT z klasycznymi algorytmami, takimi jak AdaBoost i regresja logistyczna, co uwypukliło znaczenie zaawansowanego przetwarzania cech.

W Metoda20 połączono LightGBM i CatBoost, dwa potężne algorytmy boostingu, które wyróżniają się swoją szybkością i wydajnością. LightGBM został wybrany ze względu na szybkość trenowania i efektywność w pracy z dużymi zbiorami danych, natomiast CatBoost zapewnił wysoką dokładność i odporność na brakujące wartości. Regresja logistyczna, tak jak w Metoda13, służy jako meta-model, integrując wyniki obu modeli bazowych. Metoda20 rozwija te koncepcje, koncentrując się na szybkości i precyzji predykcji, co czyni go jednym z najbardziej zaawansowanych podejść w całym zestawie metod.

3.4.2. Wejściowe dane

Metoda train_run20 przyjmuje następujące parametry wejściowe:

1. **X_train**: Macierz danych treningowych, gdzie każdy wiersz to jedna próbka, a kolumny to cechy. Rozmiar: $\mathbf{n} \times \mathbf{m}$, gdzie $\mathbf{n}$ to liczba próbek, a $\mathbf{m}$ to liczba cech.
2. **y_train**: Wektor etykiet dla danych treningowych. Rozmiar: $\mathbf{n}$, gdzie $\mathbf{n}$ to liczba próbek w danych treningowych.
3. **X_test**: Macierz danych testowych, analogiczna do **X_train**, używana do oceny modelu.
4. **y_test**: Wektor etykiet testowych, odpowiadający danym testowym.

3.4.3. Wyjściowe dane

Metoda zwraca następujące elementy:

1. **meta_model**: Meta-model wytrenowany na danych pochodzących z predykcji modeli **LightGBM** i **CatBoost**.
2. **meta_features_test**: Macierz meta-cech, zbudowana na podstawie predykcji modeli bazowych dla danych testowych.
3. **y_test**: Wektor etykiet testowych, identyczny z wejściowym **y_test**.

3.4.4. Algorytm

1. Trenowanie modeli bazowych:
 - **LightGBM**:
 - Model **LGBMClassifier** jest inicjalizowany z 100 estymatorami i stanem losowości (**random_state**) 42.
 - Model jest trenowany na danych treningowych **X_train** i **y_train**.
 - **CatBoost**:
 - Model **CatBoostClassifier** jest inicjalizowany z 100 iteracjami, stanem losowości (**random_state**) 42 i z wyłączonym trybem szczegółowych logów **verbose**.
 - Model jest trenowany na tych samych danych treningowych.
2. Generowanie predykcji dla danych treningowych:
 - Obliczane są prawdopodobieństwa klasy pozytywnej $P(y = 1 | x)$ dla obu modeli bazowych:
 - **lgb_preds_train**: Predykcje **LightGBM**.
 - **cat_preds_train**: Predykcje **CatBoost**.
 - Predykcje są używane do stworzenia macierzy meta-cech, gdzie kolumny to **lgb_preds** i **cat_preds**.
3. Trenowanie meta-modelu:
 - Wytrenowany na **danych meta-cech**, **meta_features_train**, oraz rzeczywistych etykietach **y_train**.
 - Meta-modelem jest **regresja logistyczna**, **LogisticRegression**, która łączy predykcje obu modeli bazowych.

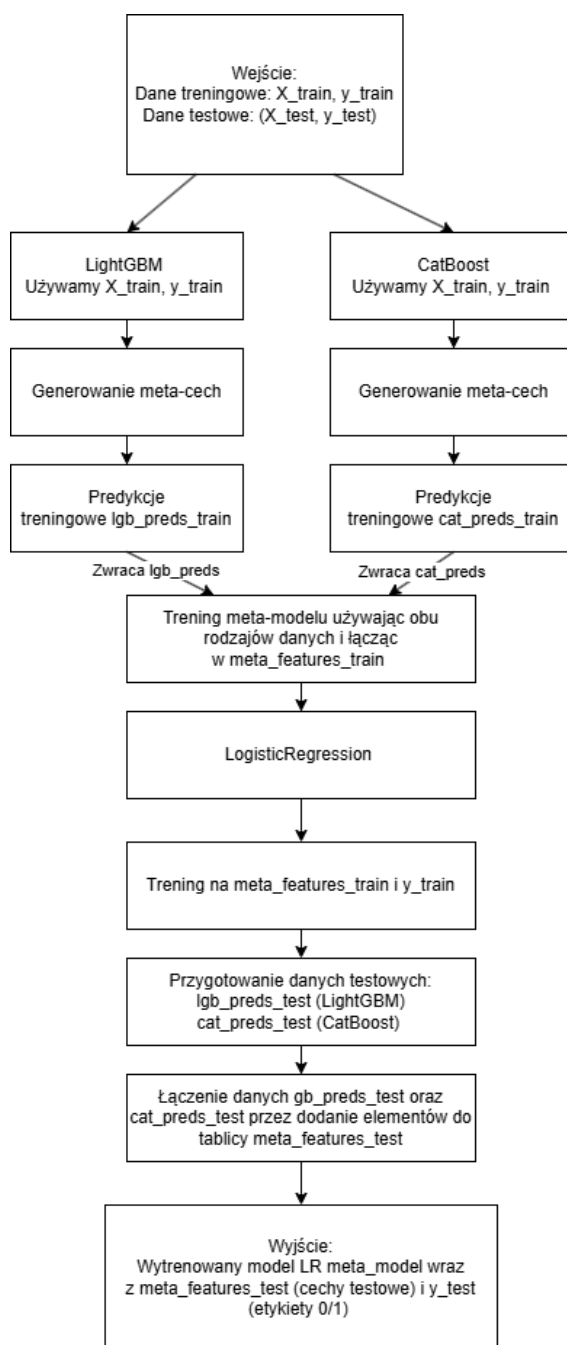
4. Generowanie predykcji dla danych testowych:
 - Obliczane są prawdopodobieństwa klasy pozytywnej dla **LightGBM** i **CatBoost**:
 - **lgb_preds_test: Predykcje LightGBM.**
 - **cat_preds_test: Predykcje CatBoost.**
 - Tworzona jest macierz meta-cech dla danych testowych, analogiczna struktura jak dla danych treningowych.
5. Zwracanie wyników: meta-model, macierz meta-cech dla testu oraz etykiety testowe są zwracane.

3.4.5. Kod

```
1 lgb_clf = LGBMClassifier(  
2     n_estimators=100,  
3     random_state=42  
4 ) # LightGBM  
5 cat_clf = CatBoostClassifier(  
6     iterations=100,  
7     verbose=0,  
8     random_state=42  
9 ) # CatBoost  
10 lgb_clf.fit(X_train, y_train)  
11 cat_clf.fit(X_train, y_train)  
12 lgb_preds_train = lgb_clf.predict_proba(X_train)[: , 1]  
13 cat_preds_train = cat_clf.predict_proba(X_train)[: , 1]  
14 meta_features_train = pd.DataFrame({ # Tworzenie meta-cech  
15     'lgb_preds': lgb_preds_train ,  
16     'cat_preds': cat_preds_train  
17 })  
18 meta_model = LogisticRegression()  
19 meta_model.fit(meta_features_train, y_train)  
20 lgb_preds_test = lgb_clf.predict_proba(X_test)[: , 1]  
21 cat_preds_test = cat_clf.predict_proba(X_test)[: , 1]  
22 meta_features_test = pd.DataFrame({  
23     'lgb_preds': lgb_preds_test ,  
24     'cat_preds': cat_preds_test  
25 })  
26 return meta_model, meta_features_test, y_test
```

Listing 3.4: Kod do 20 metody

3.4.6. Schemat blokowy



Rys. 3.4: Schemat metody 20.

3.4.7. Właściwości analityczne

1. Efektywność:

- **LightGBM** jest zoptymalizowany do szybkiego przetwarzania danych i dobrze radzi sobie z dużymi zbiorami o wysokiej liczbie cech.
- **CatBoost** doskonale działa na danych z kategoriami, minimalizując konieczność ich wcześniejszego przetwarzania.

2. **Elastyczność:** metoda jest łatwo dostosowywalna do różnych problemów klasyfikacyjnych dzięki zastosowaniu zaawansowanych modeli bazowych i uniwersalnego meta-modelu.
3. **Stabilność:**
 - Oba modele bazowe charakteryzują się stabilnością wyników dzięki zaawansowanym mechanizmom redukcji przeuczenia.
 - Meta-model dodatkowo zmniejsza wariancję wyników, zwiększając spójność predykcji.
4. **Złożoność obliczeniowa:**
 - **LightGBM** i **CatBoost** są wydajne obliczeniowo, a ich złożoność zależy liniowo od liczby próbek i estymatorów.
 - Regresja logistyczna jako meta-model jest szybka i wydajna, co umożliwia efektywną konsolidację predykcji.
5. **Wydajność:** **LightGBM** działa szczególnie dobrze na dużych zbiorach danych, podczas gdy **CatBoost** optymalizuje działanie na bardziej złożonych danych z różnorodnymi cechami.

4. SZCZEGÓŁY IMPLEMENTACYJNE

4.1. TECHNOLOGIE

W tej sekcji omówiono technologie wykorzystane w projekcie.

Aplikacja została napisana w języku programowania Python. Technologia ta została wybrana ze względu na szeroki wachlarz dostępnych bibliotek, frameworków oraz narzędzi. Doskonale nadaje się do tworzenia aplikacji wykorzystujących techniki uczenia maszynowego. Zainstalowana wersja to **Python 3.12.6**. Implementacja korzysta z następujących bibliotek:

- `tensorflow` to wszechstronna biblioteka używana do budowy modeli uczenia maszynowego i głębokiego uczenia. Oferuje szerokie wsparcie dla sieci neuronowych, takich jak CNN i RNN. W projekcie została wykorzystana do implementacji zaawansowanych modeli uczenia maszynowego. **Użyta wersja: '2.18.0'**.
- `keras` jest wysokopoziomowym API opartym na TensorFlow, ułatwiającym budowę i trenowanie modeli głębokiego uczenia. W projekcie została wykorzystana do definiowania warstw, architektury sieci i przygotowania modeli. **Użyta wersja: '3.6.0'**.
- `scikit-learn` zawiera szeroką gamę narzędzi do uczenia maszynowego, w tym modele klasyfikacji, regresji, klasteryzacji, a także metody wstępnego przetwarzania danych. W projekcie była używana do transformacji danych i budowy modeli klasyfikacyjnych. **Użyta wersja: '1.5.2'**.
- `xgboost` to wydajna biblioteka do gradientowego zwiększania liczby drzew, szczególnie skuteczna w zadaniach klasyfikacyjnych i regresyjnych. W projekcie została wykorzystana do tworzenia modeli zespołowych. **Użyta wersja: '2.1.2'**.
- `pandas` to biblioteka przeznaczona do manipulacji danymi i analizy. Ułatwia obsługę dużych zbiorów danych w formie tabelarycznej. W projekcie używana była do wczytywania, czyszczenia i przekształcania danych. **Użyta wersja: '2.2.3'**.
- `numpy` to podstawowa biblioteka do obliczeń numerycznych w Pythonie. Dostarcza funkcje do operacji na wielowymiarowych tablicach, generowania liczb losowych oraz wykonywania obliczeń matematycznych. **Użyta wersja: '1.26.4'**.
- `matplotlib` to biblioteka służąca do wizualizacji danych w postaci wykresów i diagramów. W projekcie była używana do przedstawienia wyników w postaci graficznej. **Użyta wersja: '3.9.2'**.
- `seaborn` to rozszerzenie dla `matplotlib`, które umożliwia tworzenie bardziej atrak-

- cyjnych i zaawansowanych wizualizacji statystycznych. W projekcie była używana do generowania szczegółowych wykresów danych. **Użyta wersja: '0.13.2'.**
- `transformers` to biblioteka rozwijana przez Hugging Face, używana do przetwarzania języka naturalnego (NLP). Zawiera zaimplementowane modele, takie jak BERT, GPT, i RoBERTa. W projekcie wykorzystano ją do klasyfikacji tekstu. **Użyta wersja: '4.46.2'.**
 - `torch` (PyTorch) to elastyczna biblioteka do głębokiego uczenia, wspierająca dynamiczne definiowanie sieci neuronowych. W projekcie była używana do tworzenia modeli opartych na sieciach neuronowych i obliczeń tensorowych. **Użyta wersja: '2.5.1'.**

4.2. WYMAGANIA SYSTEMOWE

4.2.1. Sprzęt

W poniższej podsekcji wymieniono minimalne wymagania sprzętowe do uruchomienia aplikacji i testów.

- Procesor: co najmniej 4 rdzenie, 8 wątków, częstotliwość bazowa 3,60 GHz i 6 MB pamięci cache. W testach wykorzystano procesor Intel® Core™ i7 10700f.
- Pamięć: co najmniej 16 GB RAM.
- Karta graficzna: dedykowany układ graficzny z co najmniej 4 GB pamięci. W testach wykorzystano AMD® Radeon RX 6400 (4 GB).
- Dysk: 4,5 GB wolnej przestrzeni na dysku.

4.2.2. Oprogramowanie

Poniżej wymieniono minimalne wymagania dotyczące oprogramowania, niezbędne do uruchomienia aplikacji i testów.

- Zainstalowany Python (wersja określona w Sekcji 4.1).
- Zainstalowana najnowsza wersja `pip`.
- Instalacja bibliotek i frameworków odbywa się za pomocą pliku `requirements.txt`. Aby zainstalować wszystkie wymagane zależności, należy w terminalu uruchomić polecenie `pip install -r requirements.txt`.
- Umieszczenie danych testowych w odpowiednim podfolderze projektu: `projekt\datasets\nazwa`. Dane testowe powinny być zapisane w plikach CSV o następującej strukturze:
 - Kolumny: `title`, `text`, `subject`, `date`.
- W folderze powinny znajdować się dwa pliki danych, jak przedstawiono na zdjęciu:
 - `Fake.csv` - zawierający fałszywe wiadomości.
 - `True.csv` - zawierający prawdziwe wiadomości.
- Wystarczająca ilość miejsca na dane.

4.3. ARCHITEKTURA SYSTEMU

System został zaprojektowany w celu analizy i klasyfikacji wiadomości tekstowych jako fałszywe (fake news) lub prawdziwe. Oparty jest na kombinacji różnych algorytmów uczenia maszynowego oraz technik ensemble. Kluczowe moduły obejmują preprocessowanie danych, trenowanie modeli, ocenę wyników oraz wizualizację wyników. Architektura systemu składa się z czterech głównych warstw:

Warstwa Wczytywania Danych i Preprocessingu

Dane wejściowe są wczytywane z plików CSV: `Fake.csv` (etykieta 0) oraz `True.csv` (etykieta 1).

Dane są łączone i dzielone na zbiór treningowy oraz testowy przy użyciu funkcji `split_data`, `split_data_one_shot` lub `split_data_few_shot`.

Funkcje ekstrakcji cech:

- **TF-IDF**: Przekształca tekst na wektory numeryczne.
- **BERT/Roberta/Sentence Transformers**: Generuje osadzenia kontekstowe.

4.3.1. Warstwa Trenowania Modeli

Każda metoda implementowana jest jako osobna funkcja (na przykład `Metoda17`, `Metoda18`). Kluczowe elementy:

- Modele uczenia maszynowego, takie jak Gradient Boosting, Random Forest, LightGBM, CatBoost.
- Sieci neuronowe implementowane w TensorFlow z dynamicznym ważeniem próbek.
- Łączenie predykcji za pomocą meta-modelu (regresja logistyczna).

Wywołanie treningu odbywa się w module `main.py` za pomocą funkcji `train_method`, gdzie użytkownik może wybrać numer metody (np. od 1 do 20).

4.3.2. Warstwa Ewaluacji

Modele są oceniane za pomocą funkcji `evaluate_model`, która oblicza następujące metryki:

- Dokładność (Accuracy),
- Log Loss.

Wyniki są zapisywane w plikach `MetodaX_results.csv` i `MetodaX_pr_curve.csv` w folderze `results_X_Y`, gdzie X oznacza typ reprezentacji, a Y tryb podziału danych.

4.3.3. Warstwa Wizualizacji

Moduł `all_plots.py` generuje wykresy na podstawie wyników z plików CSV:

- Porównania metryk (Accuracy, Execution Time, Log Loss) w formie wykresów słupkowych.
- Wykresy Precision-Recall dla każdej metody.

Wykresy są zapisywane w folderze `plots`.

4.3.4. Uruchomienie Treningu

Użytkownik uruchamia program `main.py` w konsoli i podaje następujące parametry:

```
PS C:\Users\Vadym\Documents\magisterka> .\master_env\Scripts\activate
(master_env) PS C:\Users\Vadym\Documents\magisterka> python main.py
2025-01-23 19:47:30.972961: I tensorflow/core/util/port.cc:153] oneDNN custom operatio
EDNN_OPTS=0'.
2025-01-23 19:47:33.220182: I tensorflow/core/util/port.cc:153] oneDNN custom operatio
EDNN_OPTS=0'.
WARNING:tensorflow:From C:\Users\Vadym\Documents\magisterka\master_env\Lib\site-packag

Wybierz reprezentację kontekstową (1-3) lub wpisz 'all', aby uruchomić wszystkie:
1 - BERT
2 - RoBERTa
3 - Sentence Transformers
1
Wybierz numer metody początkującej (1-20) lub wpisz 'all', aby uruchomić wszystkie:
6
Wybierz numer metody ostatecznej (1-20)
5
Wybierz tryb podziału danych (1-3) lub wpisz 'all', aby uruchomić wszystkie:
1 - Klasyczny split
2 - One Shot
3 - Few Shot
3
Generowanie reprezentacji kontekstowej 1...
```

Rys. 4.1: Przykładowe odpalenie aplikacji.

4.3.5. Generowanie Wykresów

Po zakończeniu treningu, użytkownik uruchamia `all_plots.py`, aby wygenerować wykresy na podstawie wyników zapisanych w folderach `results_X_Y`. Skrypt automatycznie tworzy wykresy i zapisuje je w folderze `plots`.

5. BADANIA EKSPERYMENTALNE

5.1. WPROWADZENIE DO BADAŃ

5.1.1. Cel przeprowadzonych eksperymentów

Celem przeprowadzonych eksperymentów było zbadanie efektywności, stabilności oraz praktycznej użyteczności czterech autorskich modeli klasyfikacyjnych: **Metoda17**, **Metoda18**, **Metoda19**, oraz **Metoda20**, a także porównanie ich wyników z wynikami uzyskanymi za pomocą istniejących podejść, wspomnianych w rozdziale **Przegląd wybranych artykułów**. Badania miały na celu ocenę skuteczności tych metod w wykrywaniu *fake news* w różnych scenariuszach danych, uwzględniając zmienne proporcje klas oraz różne poziomy złożoności zbiorów danych.

Każdy z analizowanych modeli reprezentuje odmienne podejścia w dziedzinie uczenia maszynowego, takie jak dynamiczne ważenie próbek w sieciach neuronowych (**Metoda17**), kombinacje klasyfikatorów z meta-modelami (**Metoda18**, **Metoda19**, **Metoda20**), czy zaawansowane techniki ensemble. Porównanie wyników autorskich metod z wynikami klasycznych i hybrydowych rozwiązań pozwoliło na ocenę ich konkurencyjności w kontekście istniejących algorytmów.

Eksperymenty miały na celu:

- **Porównanie skuteczności modeli autorskich i klasycznych** pod względem najważniejszych metryk, takich jak **Accuracy**, **Execution Time** i **Log Loss**.
- **Analizę zdolności generalizacyjnych** modeli na danych testowych, z uwzględnieniem ich adaptacyjności do różnych typów danych i scenariuszy.
- **Ocenę wydajności obliczeniowej** modeli (**Execution Time**) w kontekście potencjalnego zastosowania w rzeczywistych systemach wykrywających fake news.

5.1.2. Uzasadnienie zastosowania wybranych metryk

W eksperymentach zastosowano zestaw wszechstronnych metryk, które umożliwiają kompleksową ocenę modeli.

Accuracy jest podstawową miarą skuteczności modelu, wskazującą odsetek poprawnych klasyfikacji. W przypadku zrównoważonych zbiorów danych **Accuracy** jest dobrym wskaźnikiem ogólnej jakości modelu, jednak w scenariuszach z nierównomiernym rozkładem klas może być mniej miarodajne.

Log Loss mierzy niepewność modelu w predykcjach. Niska wartość Log Loss wskazuje, że model generuje bardziej pewne i spójne prognozy.

Execution Time (Czas wykonania) jest kluczową miarą efektywności obliczeniowej. Wskazuje czas potrzebny na wytrenowanie i ocenę modelu, co ma istotne znaczenie w kontekście wdrożeń w systemach produkcyjnych.

5.2. METODOLOGIA

5.2.1. Opis zestawów danych użytych w eksperymentach

ISOT Fake News Dataset to jedno z najbardziej popularnych i szeroko stosowanych zbiorów danych w badaniach nad wykrywaniem fałszywych informacji. Został stworzony przez zespół badawczy z University of Victoria w Kanadzie i służy jako punkt odniesienia dla eksperymentów związanych z klasyfikacją tekstu oraz analizą fałszywych wiadomości.

Struktura zbioru danych

ISOT Fake News Dataset składa się z następujących kategorii danych. **Fake News**, która zawiera nowości o wskazanych poniżej własnościach:

- Zawiera artykuły sklasyfikowane jako fałszywe informacje, których celem jest wprowadzanie w błąd lub manipulacja opinią publiczną.
- Źródła tych artykułów pochodzą głównie z witryn internetowych znanych z publikowania niesprawdzonych, sensacyjnych lub celowo wprowadzających w błąd treści.
- Często charakteryzują się emocjonalnym językiem, prowokacyjnymi nagłówkami i brakiem wiarygodnych źródeł.

Zrozumiałym jest z opisu, że **True News** jest przeciwieństwem oraz wspomnijmy o jego następujących cechach:

- Zawiera artykuły uznane za prawdziwe, publikowane przez renomowane agencje informacyjne takie jak Associated Press, Reuters czy BBC.
- Są to treści oparte na rzetelnym dziennikarstwie, w których stosuje się wiarygodne źródła i odpowiednie standardy edytorskie.

Liczba próbek

Zbiór zawiera około **40 000 artykułów** podzielonych równomiernie między kategorię „Fake” i „True”.

Cechy tekstowe

Próbki zostały przetworzone w formie wektorów cech przy użyciu dwóch metod, a mianowicie **TF-IDF** oraz **SentenceTransformer**. **TF-IDF (Term Frequency-Inverse**

Document Frequency) pozwala na uchwycenie znaczenia słów w kontekście dokumentu i całego korpusu. Natomiast **SentenceTransformer** bardziej zaawansowana metoda bazująca na reprezentacji semantycznej tekstu, wykorzystująca transformery do ekstrakcji cech.

Podział danych

Zbiór danych został odpowiednio podzielony, aby umożliwić skuteczne trenowanie i testowanie modeli:

Część zbioru	Cechy
Zbiór treningowy, wynosi 80% danych	Zawiera próbki używane do trenowania modeli
Zbiór testowy, wynosi 20% danych	Służy do oceny skuteczności wytrenowanego modelu na niewidzianych wcześniej danych

Tabela 5.1: Podział zbioru danych i jego cechy

5.2.2. Szczegóły dotyczące analizy PR Curve

PR Curve (Precision-Recall Curve) to jedno z podstawowych narzędzi służących do oceny skuteczności modeli klasyfikacji binarnej, szczególnie w przypadku problemów z nierównomiernym rozkładem klas. W badaniach nad wykrywaniem fałszywych informacji PR Curve odgrywa kluczową rolę, umożliwiając ocenę zdolności modeli do minimalizowania przypadków False Positives i False Negatives, co jest szczególnie istotne w kontekście klasyfikacji takich danych.

Proces analizy PR Curve składa się z trzech głównych kroków. Pierwszym jest obliczanie metryk Precision i Recall dla różnych wartości progów klasyfikacji, co pozwala na stworzenie pełnej krzywej Precision-Recall. Następnie tworzy się wykresy, gdzie Precision umieszcza się na osi Y, a Recall na osi X, co pozwala na wizualizację zdolności modeli do klasyfikacji próbek. Ostatnim krokiem jest porównanie wartości PR-AUC, czyli obszaru pod krzywą Precision-Recall, który wskazuje na ogólną skuteczność modeli w wykrywaniu istotnych przypadków przy minimalizacji błędów.

Znaczenie analizy PR Curve w eksperymentach:

- **Lepsza interpretacja wyników w nierównomiernych zbiorach danych:** W przypadku problemów takich jak wykrywanie fake news, klasy mogą być silnie niezrównoważone (np. więcej artykułów prawdziwych niż fałszywych). Metryki takie jak Accuracy mogą w takich przypadkach wprowadzać w błąd, dlatego PR Curve jest bardziej miarodajna.
- **Ocena modeli w różnych warunkach:** Analiza PR Curve umożliwia ocenę skuteczności modeli w zależności od zmiany progów klasyfikacji, co ma kluczowe znaczenie w praktycznych zastosowaniach.

- **Możliwość porównania algorytmów:** PR Curve pozwala na porównanie różnych modeli pod kątem ich zdolności do radzenia sobie z trudnymi przypadkami. Dzięki temu można wskazać modele bardziej odpowiednie do wykrywania fake news.

5.3. WYNIKI EKSPERYMENTALNE DLA BAZOWYCH METOD

Porównanie wyników bazowych rozwiązań według miar **Log Loss**, **Accuracy** oraz **Execution Time** pozwala na wszechstronną ocenę skuteczności, precyzji i wydajności analizowanych modeli. Wybór tych kryteriów umożliwia kompleksową analizę, która uwzględnia zarówno jakość predykcji, jak i aspekty praktyczne, takie jak dokładność modeli, precyzja w ocenie błędu oraz czas potrzebny na uzyskanie wyników. Podziały na **few shot**, **one shot** oraz **no shot** są kluczowe, ponieważ pozwalają ocenić skuteczność modeli w różnych scenariuszach dostępności danych. Dzięki nim można zrozumieć, jak dobrze modele radzą sobie w sytuacjach ograniczonych zasobów treningowych, co jest szczególnie istotne w praktycznych zastosowaniach, gdzie pełne dane nie zawsze są dostępne. Te kryteria odgrywają fundamentalną rolę w ocenie uniwersalności i adaptacyjności badanych rozwiązań.

Dla przeprowadzenia eksperymentu wyłącznie autorskich metod, zostały wybrane **Sentence-BERT (SBERT)** jako reprezentację semantyczną tekstu oraz **Few-Shot Learning (FSL)** do techniki uczenia w kontekście danych. W przypadku analizy danych, obie dokładnie **SBERT** oraz **FSL** okazały się kluczowe dla uzyskania wysokiej jakości wyników dla zaimplementowanych metod, **wspominanych w rozdziale „Przegląd literatury”**.

SBERT wyróżnia się na tle innych modeli, takich jak BERT czy RoBERTa, dzięki swojej zdolności do dokładniejszego uchwycenia relacji semantycznych między zdaniami. Pozwoliło to na znaczne zwiększenie efektywności w detekcji fałszywych wiadomości, co czyni go idealnym narzędziem w kontekście tego projektu.

Z kolei Few-Shot Learning pokazał swoją elastyczność i zdolność do działania w sytuacjach, gdzie dostęp do dużych zbiorów danych jest ograniczony. Dzięki umiejętności skutecznego uczenia się na podstawie kilku przykładów, metoda ta umożliwiła osiągnięcie stabilnych i precyzyjnych wyników nawet w trudnych warunkach.

5.3.1. Porównanie wyników bazowych rozwiązań według reprezentacji semantycznej tekstu, a mianowicie BERT

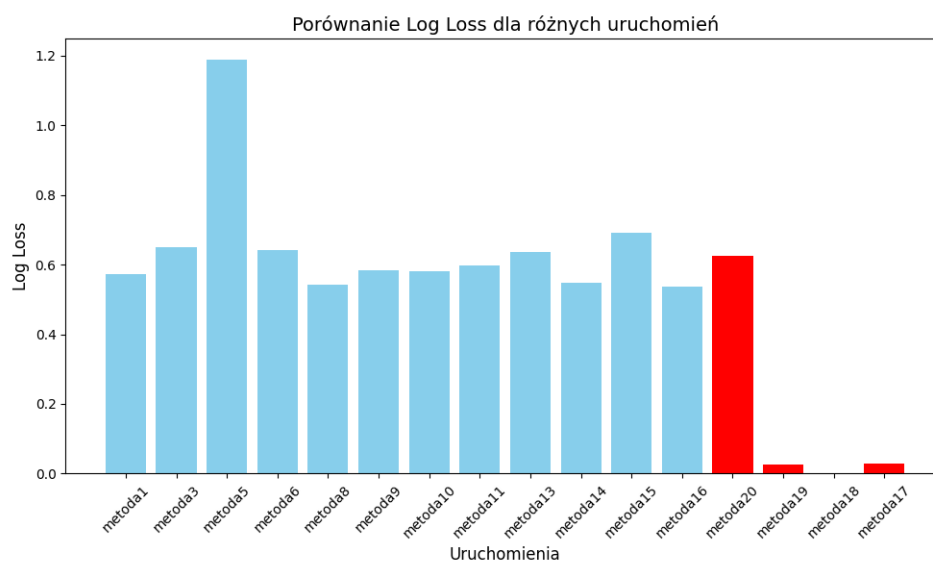
Log Loss

Wyniki Na podstawie przedstawionych wykresów Log Loss dla trzech podejść uczenia maszynowego: klasycznego (no-shot), few-shot oraz one-shot, można zaobserwować następujące zależności w porównaniu z czerwonymi wynikami (Metoda17, Metoda18, Metoda19, Metoda20):

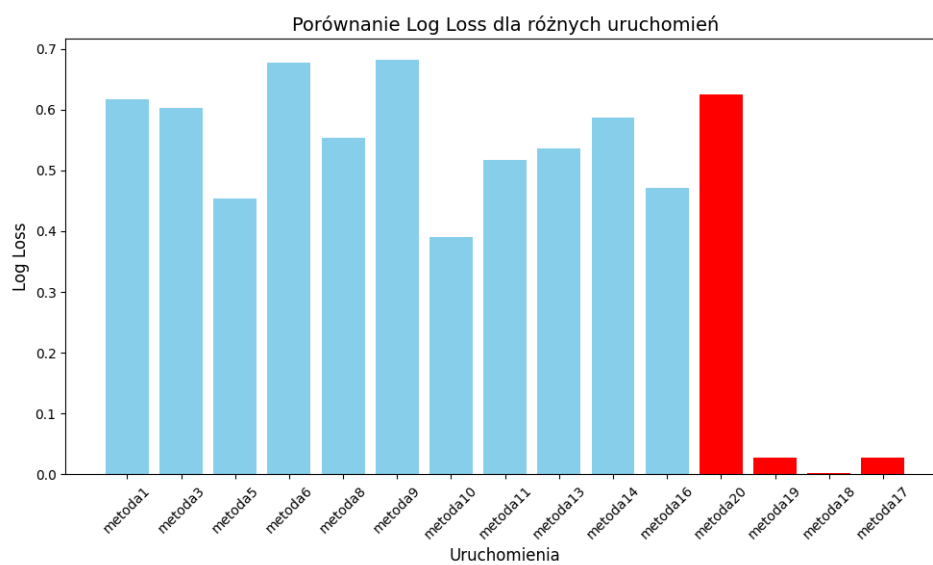
Na pierwszym wykresie (no-shot) widoczna jest umiarkowana stabilność modeli, ponieważ większość uruchomień osiąga Log Loss w zakresie od 0.4 do 0.6. Wyróżnia się jednak **Metoda5**, które odnotowało najwyższą wartość Log Loss wynoszącą **1.2**, co wskazuje na problemy z dopasowaniem w tym przypadku. W porównaniu do wyników autorskich metod, takich jak **Metoda17** (0.02), **Metoda18** (0.002) oraz **Metoda19** (0.02), **Metoda5** wypada bardzo słabo, a istniejące metody reprezentują się gorzej. Wyniki autorskich metod charakteryzują się wyjątkowo niskimi wartościami Log Loss, oprócz **Metoda20 (0.62)**, co nadal wskazuje na znacznie lepsze dopasowanie modeli w tych uruchomieniach.

Na drugim wykresie (few-shot) większość uruchomień osiągnęła Log Loss w przedziale od 0.4 do 0.7. Wyjątkiem są **Metoda9** (0.7) i **Metoda6** (0.7), które wykazują wyższe wartości. Z kolei **Metoda10** osiągnęło najniższy Log Loss wynoszący **0.4**, co świadczy o najlepszym dopasowaniu modelu w tej grupie. W porównaniu do wyników autorskich metod, takich jak **Metoda17** (0.02), **Metoda18** (0.002) oraz **Metoda19** (0.02), widoczna jest przewaga zaimplementowanych modeli, które osiągają zdecydowanie niższe Log Loss niż większość wyników z tej grupy. Niestety **Metoda20 (0.62)** posiada najgorszy wynik wśród autorskich rozwiązań, ale nadal znajduje się w dopuszczalnej skali granic wyników.

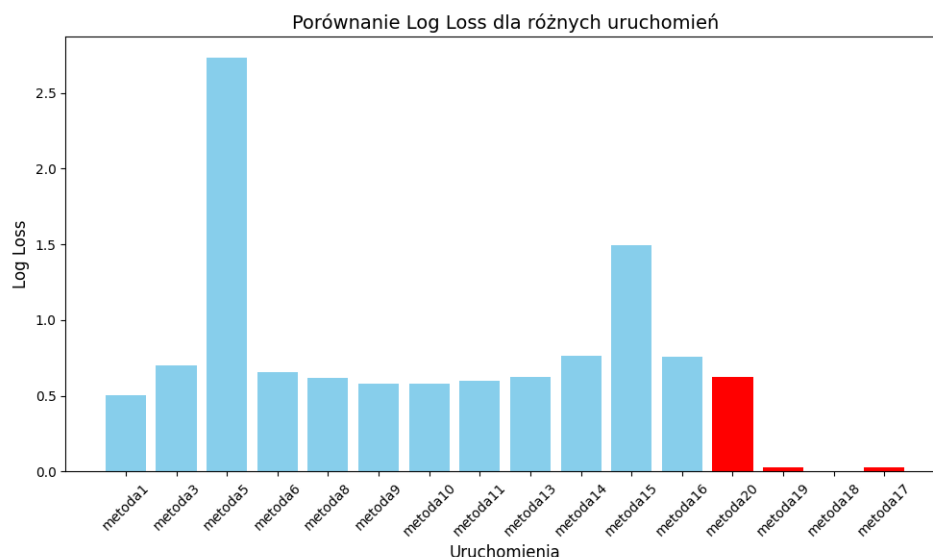
Na trzecim wykresie (one-shot) większość uruchomień charakteryzuje się stabilnym Log Loss poniżej 0.6. Jednakże **Metoda5** z powrotem znacząco odstaje z wartością Log Loss wynoszącą **2.5**, co wskazuje na wyraźne problemy w tym podejściu. Najlepsze wyniki osiągnęły **Metoda1** (0.5), **Metoda9** (0.6) i **Metoda10** (0.6), które wykazały najniższe wartości Log Loss, potwierdzając skuteczność tych modeli. W porównaniu do wyników autorskich metod, widoczna jest dominacja autorskich modeli, które wykazują znacznie lepsze dopasowanie i większą stabilność. Wspominając wyłącznie o **Metoda20 (0.62)**, nawet taki wynik można zaznaczyć do najlepszych, porównując z wynikami **Metoda5(2.7)**, **Metoda14(0.75)**, **Metoda15(1.5)** oraz **Metoda16(0.75)**.



Rys. 5.1: Log Loss Bert z no shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono



Rys. 5.2: Log Loss Bert z few shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono



Rys. 5.3: Log Loss Bert z one shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono

Podsumowanie Najlepsze wyniki są w wypadku najniższych Log Loss, czyli Metoda19, Metoda18, Metoda17 pełniają nasz cel, posiadając najniższe możliwe wartości Log Loss. Natomiast wśród najgorszych wyników, od razu się rzucają w oczy Metoda5, Metoda9, Metoda14, Metoda15 oraz Metoda16.

Podejście few-shot wykazuje największą stabilność, ponieważ większość uruchomień osiąga niższego Log Loss, a najlepsze wyniki osiągają wyjątkowo niskie wartości Log Loss. One-shot oraz no-shot dają się najmniej stabilne – szczególnie w przypadku Metoda5, gdzie Log Loss osiąga najwyższej wartości, co wyraźnie odstaje od pozostałych wyników. W każdym podejściu czerwone modele (Metoda17, Metoda18, Metoda19, Metoda20) uzyskują najlepsze rezultaty, co czyni je wzorem skuteczności.

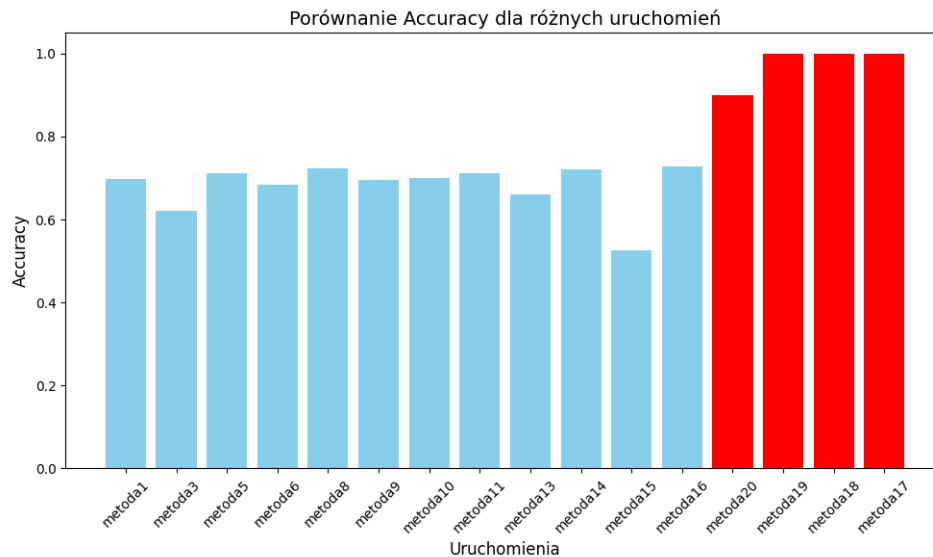
Accuracy

Wyniki Na pierwszym wykresie (no-shot) większość uruchomień charakteryzuje się stabilnym poziomem Accuracy wynoszącym około **0.75**. Wyróżniają się jednak rozwiązania autorskiej implementacji: **Metoda17**, **Metoda18**, **Metoda19** i **Metoda20**, które osiągają maksymalny poziom Accuracy większy **0.9**, przy czym **Metoda20(0.9)**, **Metoda17(1.0)**, **Metoda18(1.0)**, **Metoda19(1.0)**. To wskazuje, że modele te znacznie przewyższają pozostałe pod względem skuteczności.

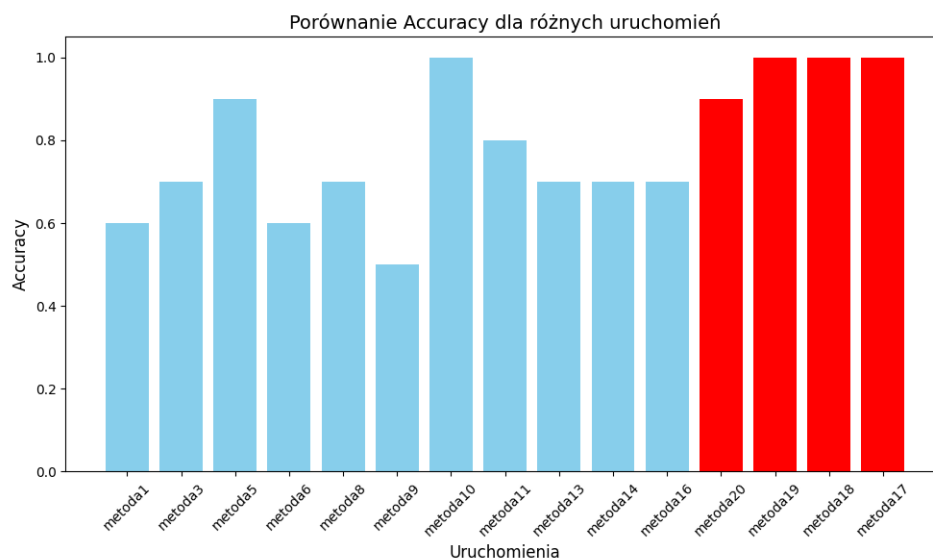
Na drugim wykresie (few-shot) większość uruchomień prezentuje stabilne Accuracy na poziomie **0.6–0.7**. Najwyższe wyniki osiągają autorskie modele: **Metoda20(0.9)**, **Metoda17(1.0)**, **Metoda18(1.0)**, **Metoda19(1.0)**, podobnie jak w podejściu no-shot. W przypadku zaimplementowanych gotowych rozwiązań, takich jak **Metoda5(0.9)**, Meto-

da10(1.0), Metoda11(0.8), widać poprawę w porównaniu do innych, które osiągają wartości poniżej **0.8**.

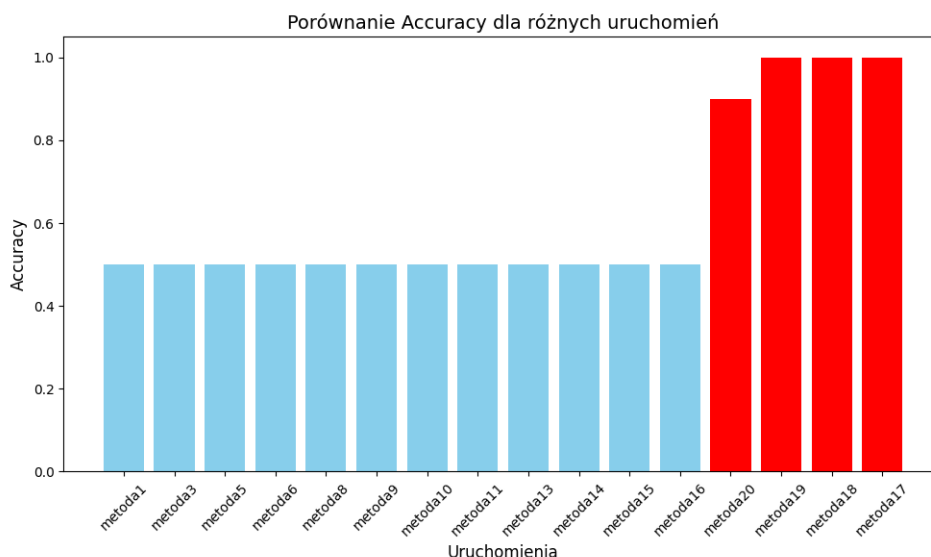
Na trzecim wykresie (one-shot) widoczne są większe różnice w Accuracy pomiędzy poszczególnymi uruchomieniami. Autorskie modele **Metoda20(0.9)**, **Metoda17(1.0)**, **Metoda18(1.0)**, **Metoda19(1.0)** ponownie osiągają najwyższe wyniki, co wskazuje na ich wyraźną przewagę. Natomiast istniejące gotowe rozwiązania, są wszystkie równe i w zakresie 0.5.



Rys. 5.4: Accuracy Bert z no shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono



Rys. 5.5: Accuracy Bert z few shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono



Rys. 5.6: Accuracy Bert z one shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono

Podsumowanie Najlepsze wyniki są przy najwyższym poziomie Accuracy, są dla **Metoda20(0.9)**, **Metoda17(1.0)**, **Metoda18(1.0)**, **Metoda19(1.0)** oraz już istniejących rozwiązań, takich jak **Metoda5(0.9)**, **Metoda10(1.0)**. Patrząc na to, że pod względem Log Loss **Metoda5** osiągnął najgorszego możliwego wyniku, natomiast posiada taki dobry w przypadku Accuracy. Naszym zdaniem, ten model był warty uwagi i był uwzględniony przy tworzeniu metod autorskich.

Autorskie modele Metoda17, Metoda18, Metoda19, Metoda20 osiągają najwyższą skuteczność w każdym podejściu, uzyskując Accuracy na najwyższym możliwym poziomie, co czyni je wzorem dopasowania. Podejście no-shot oraz few-shot wykazują większą stabilność niż one-shot, gdzie większość uruchomień osiąga Accuracy na poziomie ~0.5, co wskazuje na ograniczoną skuteczność tego podejścia.

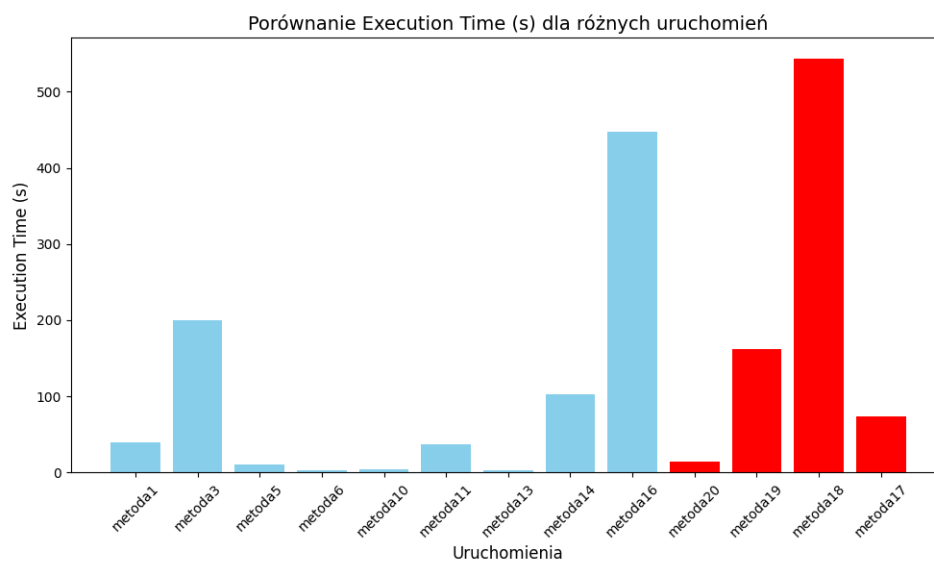
Execution Time

Wyniki Na pierwszym wykresie (podejście no-shot): większość gotowych rozwiązań charakteryzuje się krótkim czasem wykonania, nieprzekraczającym **20 sekund** (np. **Metoda5**, **Metoda6**, **Metoda10**, **Metoda13**). Wyjątkiem jest **Metoda16**, które osiąga czas wykonania wynoszący około **450 sekund**, oraz **Metoda3**, które przekracza **200 sekund**, co wskazuje na większą złożoność obliczeniową lub nieoptymalność w tych przypadkach. Autorskie modele (**Metoda17**, **Metoda18**, **Metoda19**, **Metoda20**) osiągają zróżnicowane wyniki – najdłuższy czas wykazuje **Metoda18** (550 sekund), natomiast **Metoda20** (15 sekund) działa zdecydowanie szybciej i jest na liście najszybszych modeli.

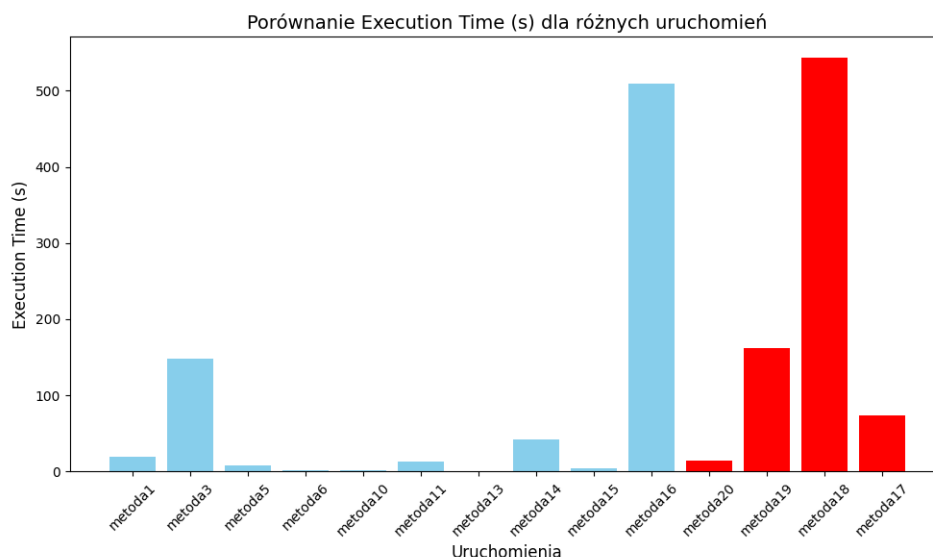
Na drugim wykresie (few-shot) większość niebieskich uruchomień wykazuje podobnie krótkie czasy wykonania jak w podejściu no-shot (np. **Metoda5**, **Metoda6**, **Metoda10**,

Metoda13). **Metoda3** oraz **Metoda16** ponownie charakteryzują się dłuższym czasem wykonania, wynoszącym odpowiednio około **150 sekund** i **500 sekund**. Jednak autorskie modele osiągają zróżnicowane wyniki – najkrótszy czas wykonania ma **Metoda20** (~15 sekund), a najdłuższy **Metoda18** (~500 sekund).

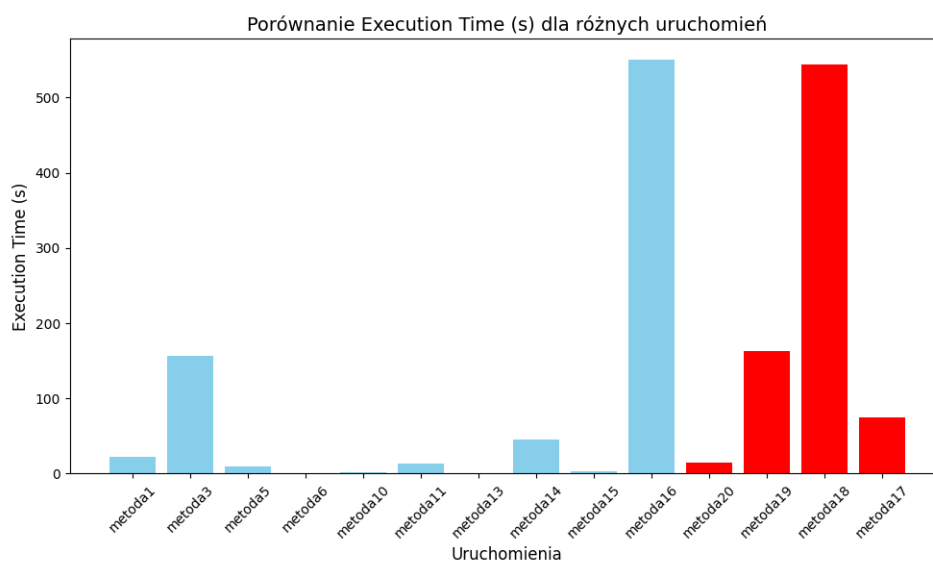
Na trzecim wykresie (**one-shot**) zachowuje się ta sama tendencja w czasach wykonania. Większość istniejących modeli działa szybko, nie przekraczając **10 sekund** (**Metoda6**, **Metoda10**, **Metoda13**, **Metoda15**), jednak **Metoda16** (~300 sekund) charakteryzuje się zauważalnie dłuższym czasem. Autorskie modele osiągają czasy od **10 sekund** (**Metoda20**) do **500 sekund** (**Metoda18**), co wskazuje na większą złożoność modeli w tej grupie, ale nadal najlepszemu wynikowi wśród wszystkich rozwiązań **Metoda20** (15 sekund).



Rys. 5.7: Execution Time Bert z no shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono



Rys. 5.8: Execution Time Bert z few-shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono



Rys. 5.9: Execution Time Bert z one-shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono

Podsumowanie Najlepsze wyniki, które polegają na najkrótszym czasie wykonania należą do **Metoda6**, **Metoda10**, **Metoda13** oraz autorskiej implementacji Metoda20. Niestety wśród najgorszych możliwych wyników, niezależnie od metodologii, zawsze na wszystkich wykresach tendencja pozostawała taka sama negatywna dla **Metoda18** (~500 sekund), **Metoda3** oraz Metoda16. Patrząc na najlepszy możliwy wynik wśród Accuracy oraz Log Loss, taki czas wykonania dla Metoda18 nie powinienem się charakteryzować negatywnie. Natomiast Metoda20 posiada najlepszy możliwy czas wśród metod autorskich, mając lekko gorsze wyniki w porównaniu Accuracy oraz Log Loss.

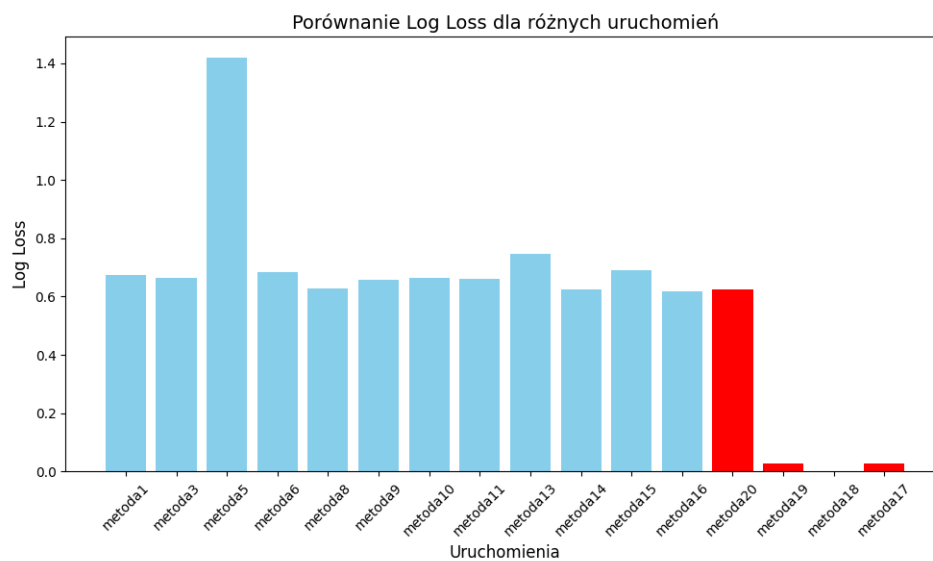
5.3.2. Porównanie wyników bazowych rozwiązań według reprezentacji semantycznej tekstu, a mianowicie RoBERTa

Log Loss

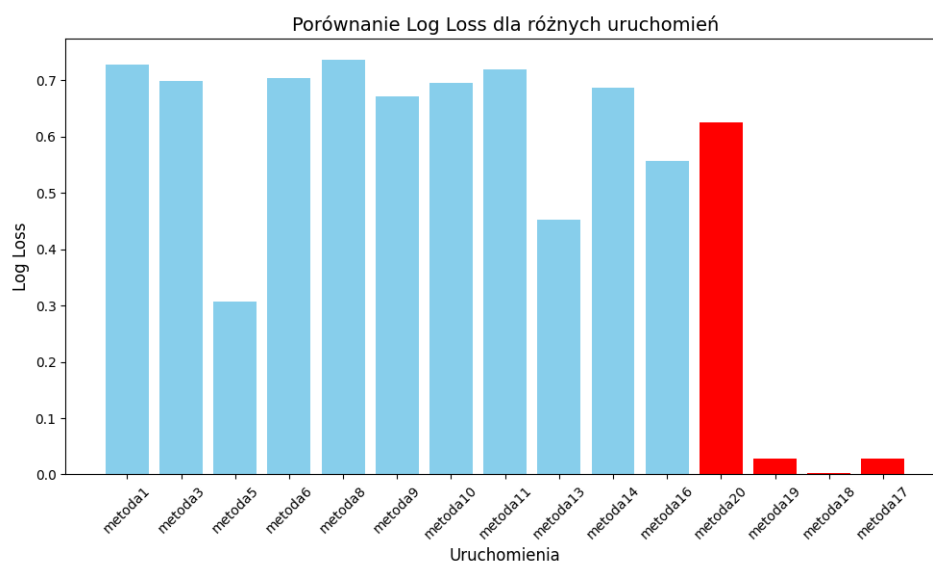
Wyniki Na pierwszym wykresie (podejście **no-shot**) większość gotowych rozwiązań osiąga wartości Log Loss w przedziale od **0.6** do **0.7**, co wskazuje na umiarkowaną stabilność w klasycznym podejściu. Wyjątkiem jest **Metoda5**, które osiągnęło najwyższą wartość Log Loss wynoszącą **1.4**, co sugeruje poważne problemy z dopasowaniem modelu w tym przypadku. Autorskie uruchomienia (**Metoda17**, **Metoda18**, **Metoda19**) osiągają wyjątkowo niskie wartości Log Loss w zakresie od **0.01** do **0.3**, co czyni je wyraźnie lepszymi. Natomiast Metoda20 znajduje się w zakresie tych zwykłych metod, będąc w przedziale od **0.6** do **0.7**.

Na drugim wykresie (**few-shot**) większość istniejących modeli osiąga wartości Log Loss w przedziale **0.6–0.7**, z wyraźnym wyjątkiem dla **Metoda5**, które osiągnęło stosunkowo niską wartość wynoszącą około **0.3**, co wskazuje na lepsze dopasowanie w tej iteracji. Autorskie uruchomienia ponownie dominują, osiągając wartości Log Loss w zakresie do **0.01** (**Metoda17**, **Metoda18**, **Metoda19**) i podobnie jak poprzednio, od **0.6** do **0.7** dla **Metoda20**.

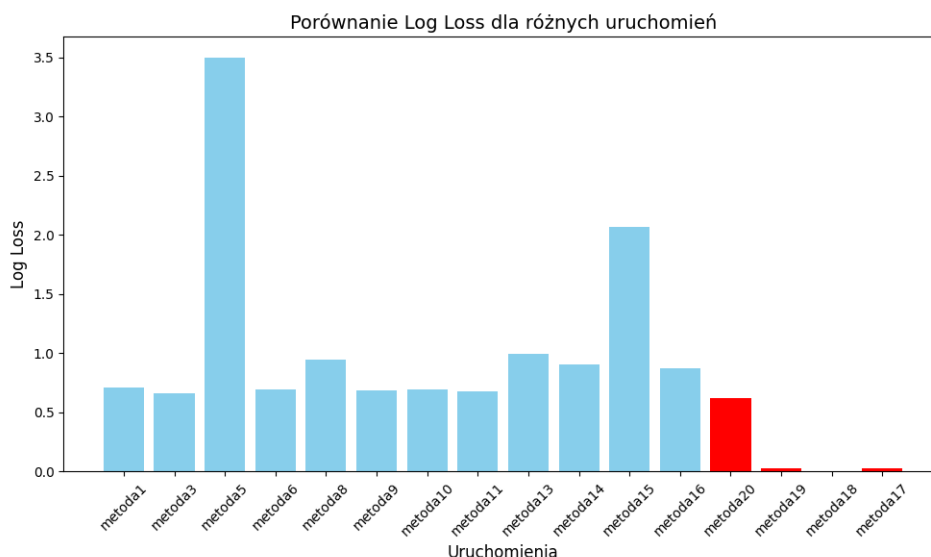
Na trzecim wykresie (**one-shot**) większość gotowych uruchomień charakteryzuje się wartościami Log Loss poniżej **1.0**, co wskazuje na większą stabilność w porównaniu do pozostałych podejść. Wyjątkiem jest **Metoda5**, które osiągnęło najwyższą wartość wynoszącą aż **3.5**, co jest znacznie gorszym wynikiem w porównaniu do pozostałych uruchomień. Autorskie podejścia (**Metoda17**, **Metoda18**, **Metoda19**) do rozwiązania problemu pozostają najlepsze, osiągając najniższe wartości Log Loss, podobnie jak w poprzednich wykresach. Również w tym przypadku tendencja do wyników u autorskiej implementacji Metoda20 zostaje taka sama jak było dla **few-shot** oraz **no-shot**.



Rys. 5.10: Log Loss RoBERTa z no shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono



Rys. 5.11: Log Loss RoBERTa z few-shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono



Rys. 5.12: Log Loss RoBERTa z one-shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono

Podsumowanie Autorskie modele (**Metoda17, Metoda18, Metoda19**) konsekwentnie osiągają najlepsze wyniki w każdym podejściu, z Log Loss w przedziale **0.01**, co czyni je najbardziej stabilnymi i skutecznymi. Podejście **few-shot** oraz **one-shot** charakteryzują się największą stabilnością dla gotowych modeli, z większością wyników poniżej **0.7**. W one-shot Metoda5 znacząco odstaje z wartością Log Loss wynoszącą aż **3.5**. Podejście **few-shot** wykazuje pewną poprawę w **Metoda5**, ale większość wyników jest zbliżona do one-shot. Podejście one-shot jest najmniej stabilne – szczególnie w przypadku **Metoda5**, co wyraźnie odstaje od innych wyników. Autorskie modele pozostają wzorem jakości w każdym podejściu.

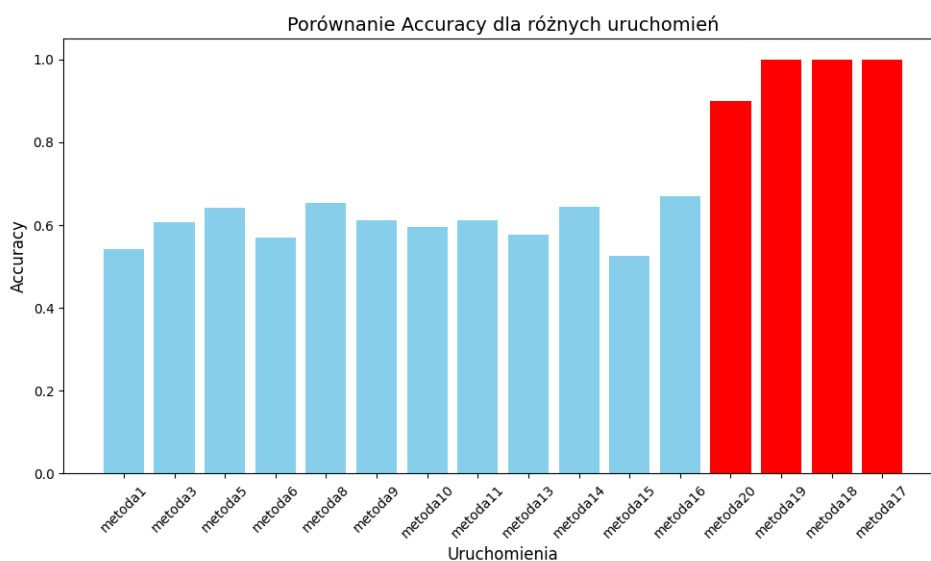
Accuracy

Wyniki Na pierwszym wykresie (podejście no-shot) większość gotowych metod osiąga Accuracy na poziomie około **0.6**, co wskazuje na umiarkowaną skuteczność klasycznego podejścia. Natomiast autorskie rozwiązania (**Metoda17, Metoda18, Metoda19, Metoda20**) zdecydowanie przewyższają pozostałe wyniki, osiągając maksymalne Accuracy na poziomie **1.0** w przypadku **Metoda17, Metoda18, Metoda19** oraz 0.9 dla Metoda20, co potwierdza ich wysoką jakość.

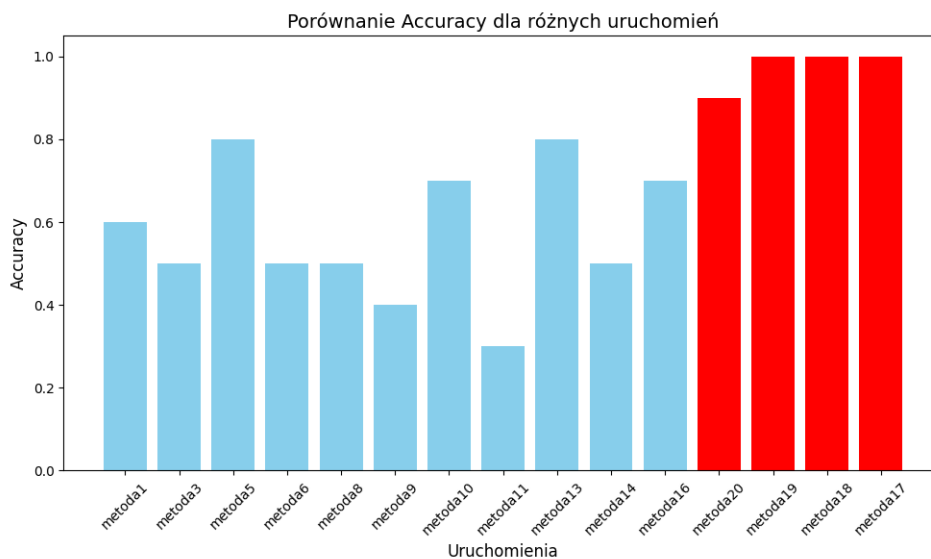
Na drugim wykresie (few-shot) widoczne są większe różnice pomiędzy uruchomieniami. Najwyższe wyniki osiąga **Metoda5** (0.8), natomiast najniższe **Metoda11** (~0.3). Autorskie modele się trzymają tendencji ponownie osiągają maksymalne Accuracy równą **1.0**, w przypadku **Metoda17, Metoda18, Metoda19** oraz 0.9 dla Metoda20.

Na trzecim wykresie (one-shot) wyniki się wyróżniają w porównaniu do poprzednich. Większość zwykłych uruchomień osiąga Accuracy w przedziale równym 0.5, co jest

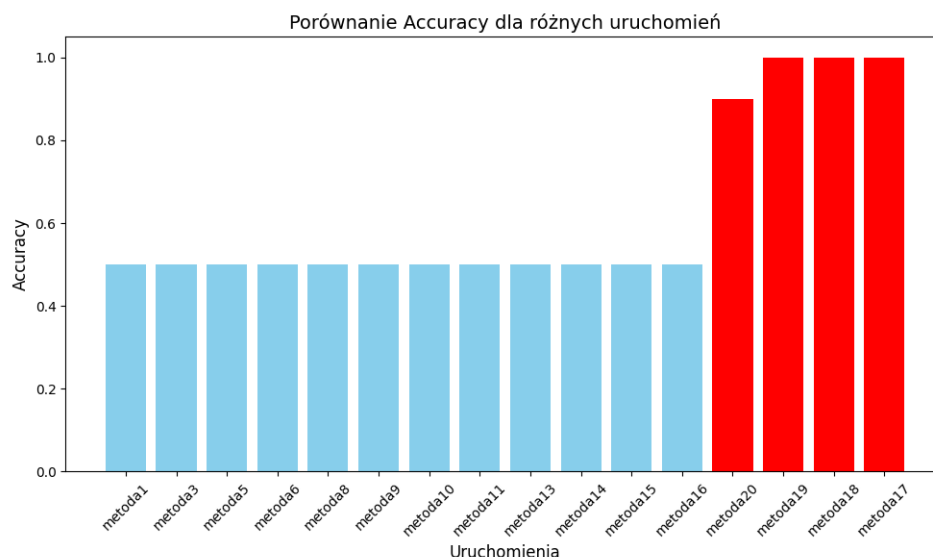
negatywnym wynikiem i pokazuje, że wykorzystywanie one-shot ogranicza zbyt mocno działanie istniejących algorytmów. Autorskie modele dotrzymują się wspomnianego trendu powyżej.



Rys. 5.13: Accuracy RoBERTa z no shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono



Rys. 5.14: Accuracy RoBERTa z few-shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono



Rys. 5.15: Accuracy RoBERTa z one-shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono

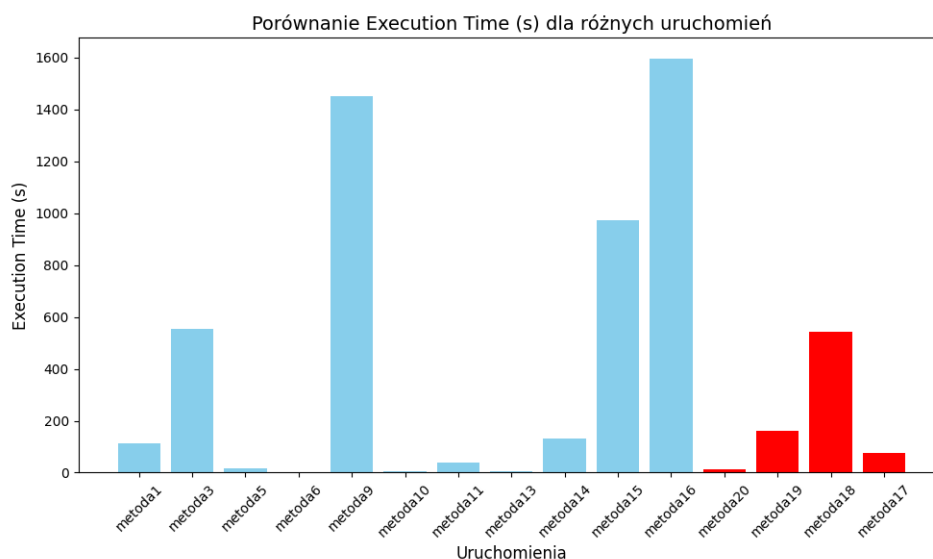
Podsumowanie Autorskie modele (**Metoda17**, **Metoda18**, **Metoda19**, **Metoda20**) konsekwentnie osiągają najlepsze wyniki w każdym podejściu, uzyskując maksymalne Accuracy w zakresie od 0.9 i do **1.0**, co świadczy o ich najwyższej skuteczności. W podejściu **no-shot** i **few-shot** można zaobserwować większą różnorodność w wynikach klasycznych modeli, przy czym **Metoda5** wyróżnia się w tych podejściach z wynikami zbliżonymi do **0.8**. Podejście one-shot wykazuje najniższą skuteczność, wszystkie istniejące modele osiągających Accuracy na poziomie **0.5**. Dominacja autorskich modeli potwierdza ich wyższość niezależnie od podejścia.

Execution Time

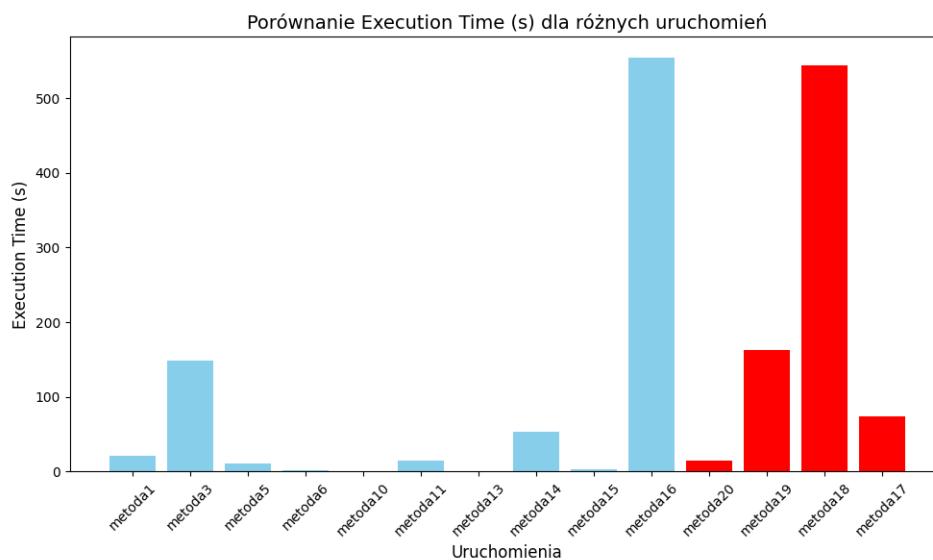
Wyniki Na pierwszym wykresie (podejście **no-shot**) większość klasycznych uruchomień charakteryzuje się relatywnie krótkimi czasami wykonania, mieszczącymi się w przedziale **0–200 sekund**. Wyjątkiem jest **Metoda9** (1400 sekund) oraz **Metoda16** (1600 sekund), które znacząco odstają od pozostałych wyników, co sugeruje większą złożoność obliczeniową lub nieoptymalność modeli w tych przypadkach. Autorskie metody (**Metoda17**, **Metoda18**, **Metoda19**, **Metoda20**) mają zróżnicowane czasy – najkrótszy czas osiąga **Metoda20** (10 sekund), a najdłuższy **Metoda18** (500 sekund).

Na drugim wykresie (**few-shot**) większość już zaproponowanych ma krótkie czasy wykonania, poniżej **200 sekund**. Jednakże **Metoda16** (500 sekund) i **Metoda3** (150 sekund) charakteryzują się dłuższymi czasami, wskazując na większe wymagania obliczeniowe w tych przypadkach. Wymyślone autorskie modele zachowują tendencję z pierwszego wykresu – **Metoda20** ma najkrótszy czas (10 sekund), natomiast najdłuższy czas osiąga **Metoda18** (500 sekund).

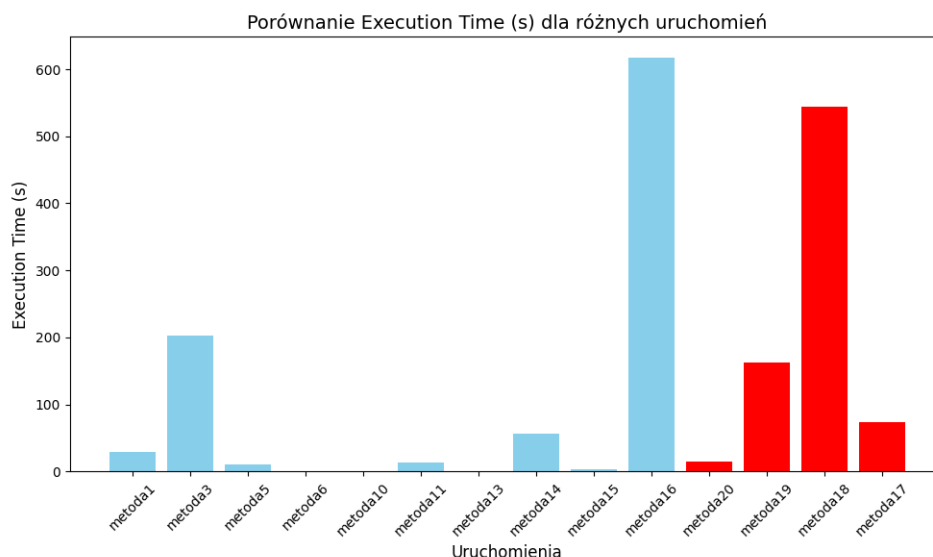
Na trzecim wykresie (one-shot) wyniki są zbliżone do podejść few-shot i no-shot. Tendencja zachowuje się taka sama co poprzednio, z mianowicie oczywisty banał wynika, że większość niebieskich uruchomień osiąga czasy poniżej **200 sekund**, z wyjątkiem dla **Metoda16 oraz Metoda3**. Wśród zaproponowanych przez nas modeli, **Metoda20** ma najkrótszy czas (10 sekund) i się wyróżnia w porównaniu do innych autorskich modeli.



Rys. 5.16: Execution Time RoBERTa z one-shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono



Rys. 5.17: Execution Time RoBERTa z few-shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono



Rys. 5.18: Execution Time RoBERTa z one-shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono

Podsumowanie Autorski model **Metoda20** się charakteryzuje się jednym najlepszych czasów wykonania, będąc najefektywniejsze we wszystkich podejściach z czasem około **10 sekund**. **Metoda18** systematycznie osiąga najdłuższe czasy wykonania wśród czerwonych modeli (~400–500 sekund). We wszystkich podejściach, wśród zwykłych modeli **Metoda3**, **Metoda9** i **Metoda16** znacząco odstają ze sporym czasem, co może wskazywać na nieoptymalność konfiguracji lub dużą złożoność obliczeniową. Szczególnie negatywną tendencję zachowują **Metoda3** oraz **Metoda16**, bo posiadają najgorsze czasy wykonania we wszystkich konfiguracjach Working.

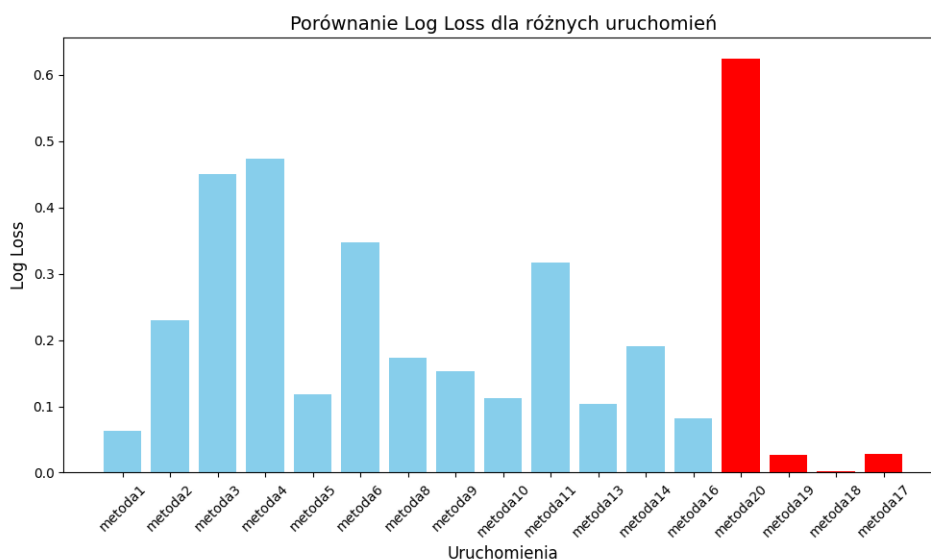
5.3.3. Porównanie wyników bazowych rozwiązań według reprezentacji semantycznej tekstu, a mianowicie Sentence-BERT (SBERT)

Log Loss

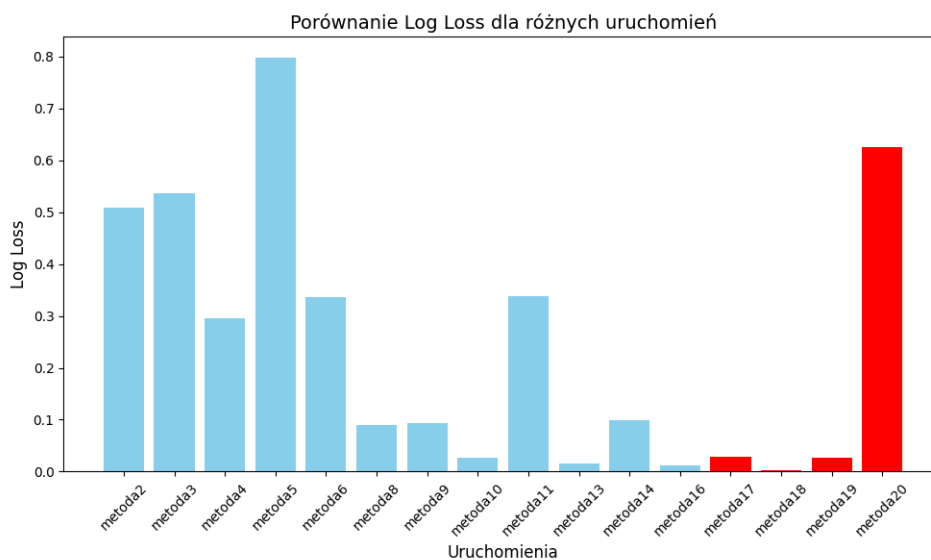
Wyniki Na pierwszym wykresie (klasyczne podejście no-shot) większość klasycznych metod uruchomienia osiąga wartości Log Loss w przedziale od **0.1** do **0.3**, co wskazuje na umiarkowaną stabilność tego podejścia. Wyjątkiem jest **Metoda4**, które osiągnęło najwyższą wartość Log Loss wynoszącą około **0.5**, co sugeruje problemy z dopasowaniem modelu. Autorskie uruchomienia **Metoda17**, **Metoda18**, **Metoda19** osiągają wartości od **0.002** do **0.05**, przy czym **Metoda20** znacznie odstaje z najwyższą wartością w tej grupie, osiągając 0.6.

Na drugim wykresie (few-shot) zauważalne są większe różnice w wynikach zwykłych modeli. Większość osiąga wartości Log Loss od **0.1** do **0.4**, a najwyższą wartość zanotowano dla **Metoda5** (0.8). Autorskie uruchomienia **Metoda17**, **Metoda18**, **Metoda19** ponownie dominują, przy czym **Metoda20** znacznie odstaje, posiadając wyniki około **0.6**.

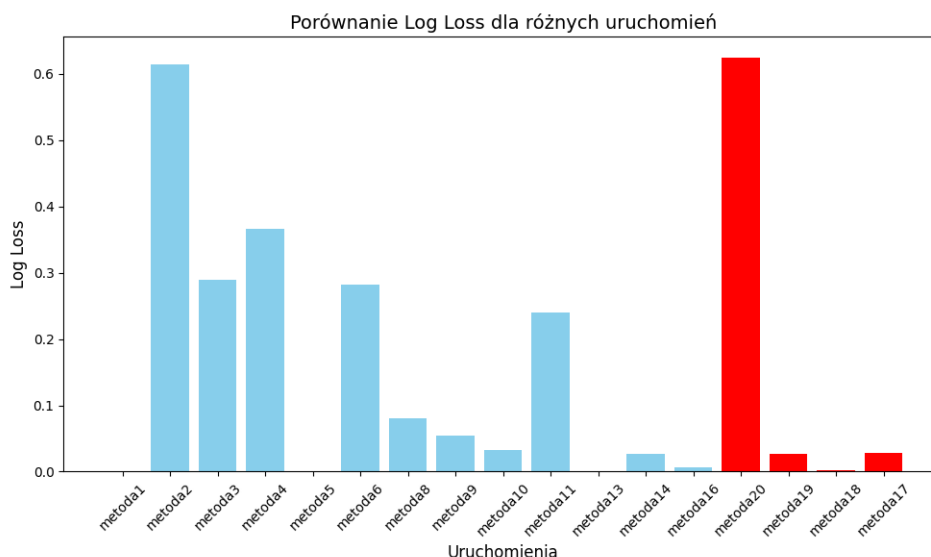
Na trzecim wykresie (one-shot) wyniki są bardziej stabilne. Najniższe wartości są zaznaczone w przypadku Metoda5, Metoda13 oraz Metoda16. Wyjątkami w negatywnym sensie są **Metoda20 i Metoda2**, które osiągnęły najwyższą wartość wśród czerwonych modeli, wynoszącą **0.6**. Wspominając o autorskich uruchomieniach **Metoda17, Metoda18, Metoda19** osiągają wyjątkowo niskie wartości Log Loss 0.01.



Rys. 5.19: Log Loss SBERT z no shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono



Rys. 5.20: Log Loss SBERT z few-shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono



Rys. 5.21: Log Loss SBERT z one-shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono

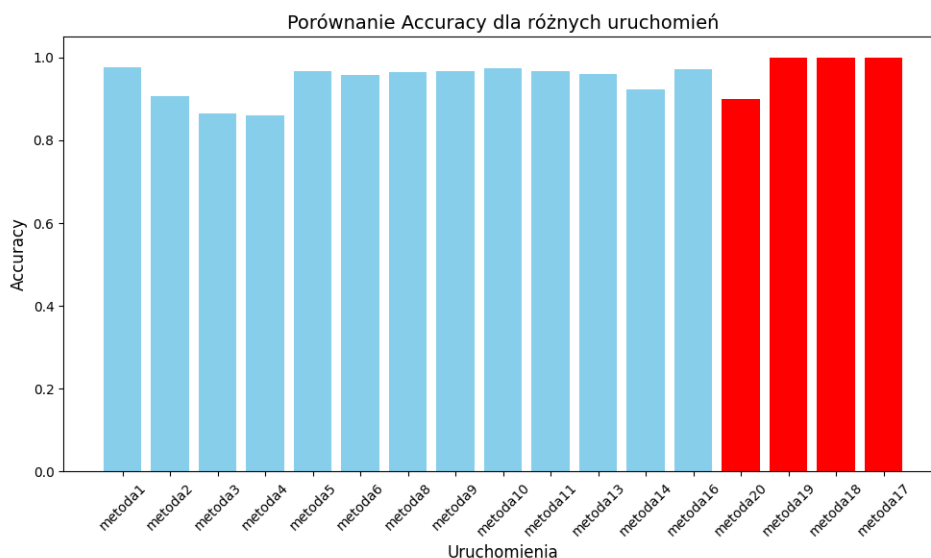
Podsumowanie **Metoda17**, **Metoda18**, **Metoda19** konsekwentnie osiągają najniższe wartości Log Loss w każdym podejściu, co potwierdza ich skuteczność. Jednak **Metoda20** w każdym podejściu osiąga wyraźnie wyższe wartości Log Loss w porównaniu do innych autorskich modeli. Modele klasyczne, takie jak **Metoda13** i **Metoda16** niezależnie od podejścia, charakteryzują się najniższymi wartościami Log Loss, co może sugerować poprawne dopasowanie modeli w tych iteracjach. Natomiast Metoda5 jest dość zmienna, co łatwo wywnioskować z porównania jego wyników dla few-shot, gdzie posiada najgorsze możliwe wartości wraz z one-shot, gdzie już ma najlepsze możliwe wartości.

Accuracy

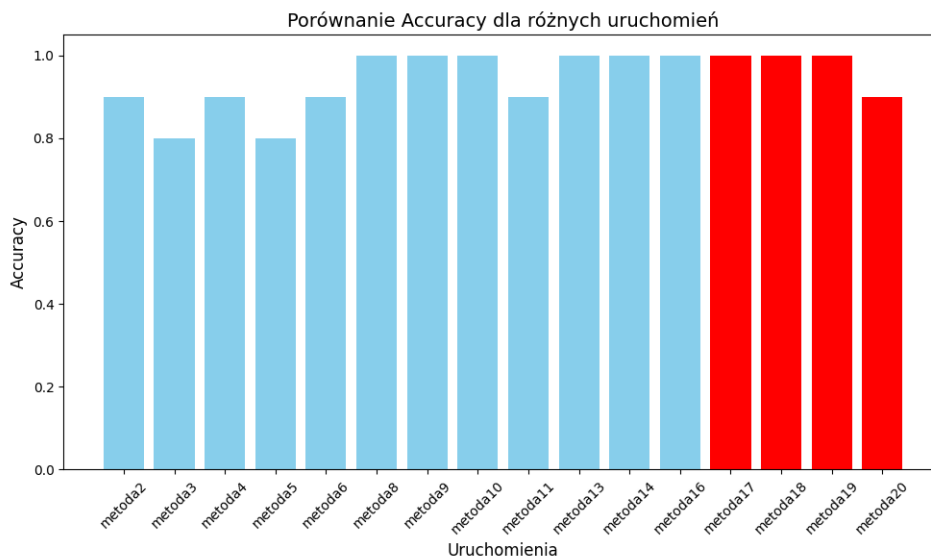
Wyniki Na pierwszym wykresie (klasyczne podejście no-shot) większość modeli osiąga wysokie Accuracy, mieszczące się w zakresie od **0.8** do **1.0**, co świadczy o stabilnej skuteczności modeli w klasycznym podejściu. Autorskie uruchomienia **Metoda17**, **Metoda18**, **Metoda19**, **Metoda20** osiągają maksymalną wartość Accuracy równą **1.0**, co czyni je najlepszymi w tej grupie. Spośród autorskich wyników wyróżnia się **Metoda20** z niższą wartością Accuracy 0.9.

Na drugim wykresie (few-shot) większość klasycznych uruchomień również osiąga wysokie wartości Accuracy, wynoszące od **0.80** do **1.0**. Wyjątkiem są modele takie jak **Metoda3** i **Metoda4**, które osiągają wartości w dolnej granicy zakresu. Autorskie modele **Metoda17**, **Metoda18**, **Metoda19**, **Metoda20** konsekwentnie osiągają Accuracy, utrzymując poprzednio obserwowane tendencje.

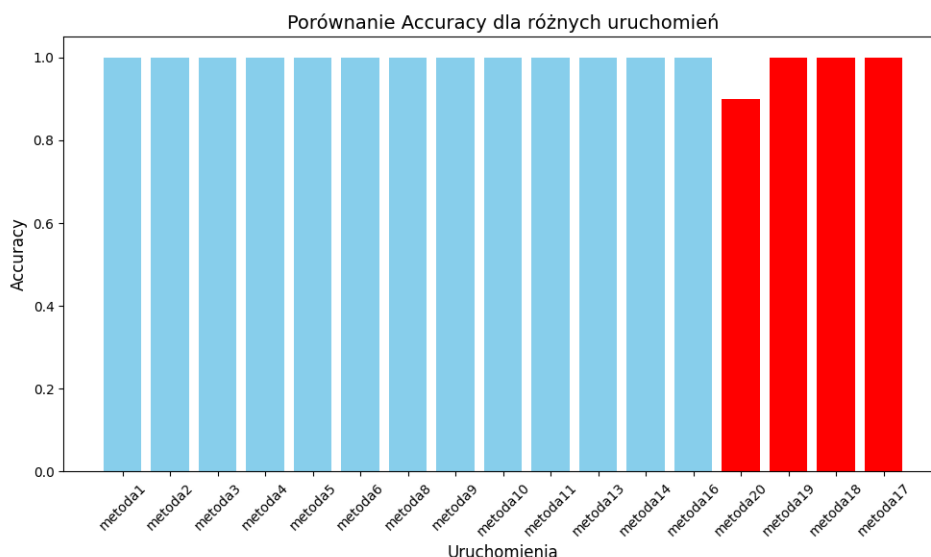
Na trzecim wykresie (one-shot) u większości modeli osiągnięto wartość Accuracy zbliżone do **1.0**. Modele **Metoda17**, **Metoda18**, **Metoda19** osiągają najwyższe wartości Accuracy, równą **1.0**, natomiast najniższy wynik należy do **Metoda20**.



Rys. 5.22: Accuracy SBERT z no shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono



Rys. 5.23: Accuracy SBERT z few-shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono



Rys. 5.24: Accuracy SBERT z one-shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono

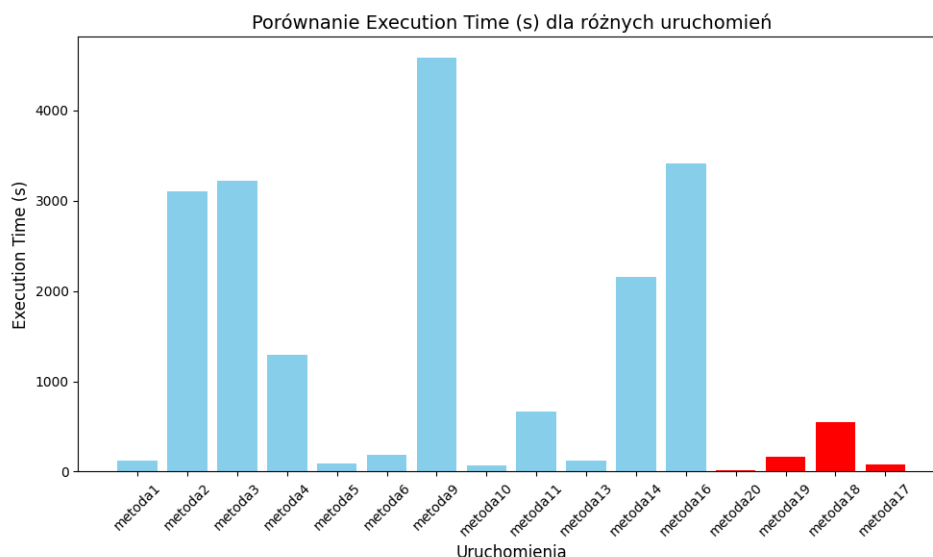
Podsumowanie Praktycznie wszystkie autorskie modele **Metoda17**, **Metoda18**, **Metoda19** osiągają najwyższe wartości Accuracy we wszystkich podejściach, co potwierdza ich przewagę w jakości. Klasyczne podejście no-shot oraz few-shot charakteryzują się większym rozrzutem wyników wśród niebieskich modeli, z nieco niższymi wartościami Accuracy (~0.85) w kilku uruchomieniach (**Metoda2**, **Metoda3**, **Metoda4**). Podejście one-shot wykazuje największą stabilność, z większością modeli osiągających wyniki bliskie **1.0**, co czyni je najbardziej spójnym pod względem skuteczności. Czerwone modele niezmiennie dominują w każdej grupie.

Execution Time

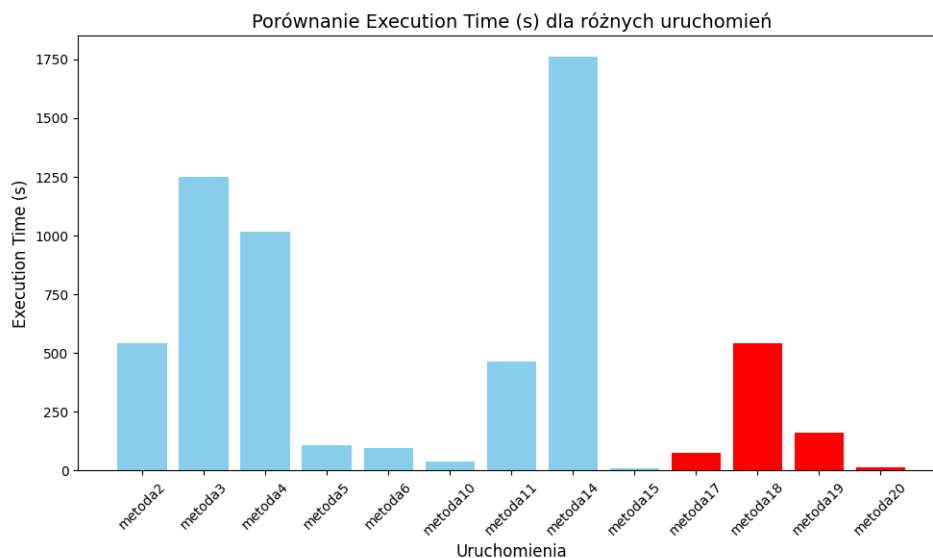
Wyniki Na pierwszym wykresie (klasyczne podejście no-shot) większość klasycznych uruchomień osiąga czasy wykonania poniżej **200 sekund**, co wskazuje na ich relatywnie wysoką efektywność obliczeniową. Najlepsze wyniki są dla Metoda1, Metoda5, Metoda10, Metoda13 oraz Metoda16. Wyjątkiem są **Metoda9** (4500 sekund), **Metoda16** (3500 sekund), które znacząco odbiegają od pozostałych wyników. Autorskie modele **Metoda17**, **Metoda18**, **Metoda19**, **Metoda20** charakteryzują się zróżnicowanymi czasami wykonania – najkrótszy czas osiąga **Metoda20** (10 sekund), a najdłuższy **Metoda18** (~400 sekund).

Na drugim wykresie (few-shot) widać, że modele rzadko osiągają czasy poniżej **250 sekund**, wyróżniając wśród najlepszych Metoda5, Metoda6, Metoda10 i Metoda15. A wyjątkami od negatywnej strony są **Metoda3** (~1200 sekund) i **Metoda14** (~1700 sekund). Autorskie uruchomienia zachowują przewagę – **Metoda20** osiąga najkrótszy czas (~10 sekund), a **Metoda18** ponownie najdłuższy (~400 sekund).

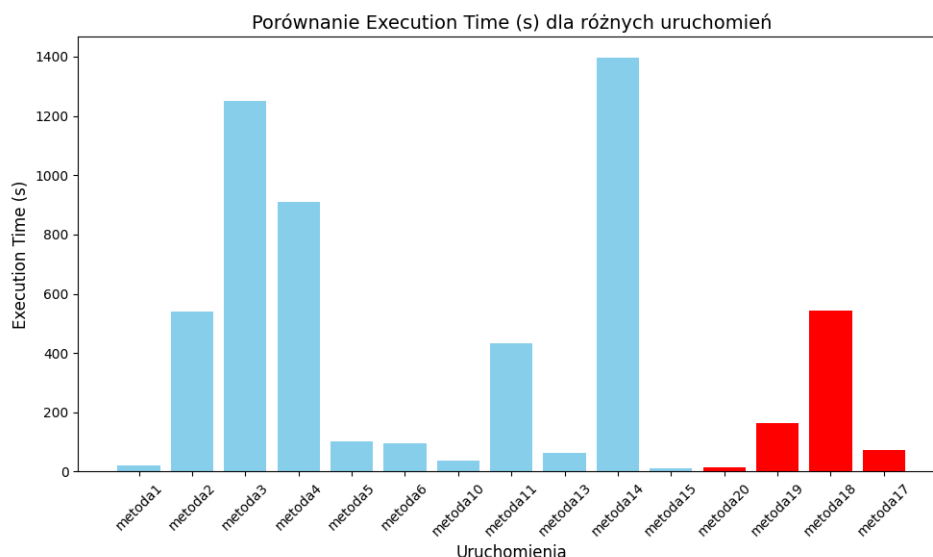
Na trzecim wykresie (one-shot) są największe rozbieżności czasów wykonania wśród niebieskich modeli. Chociaż większość z nich osiąga czasy poniżej **200 sekund**, to **Metoda3** (1200 sekund) oraz **Metoda14** (1400 sekund) znacząco odbiegają od pozostałych wyników. Autorskie modele zachowują stabilność, z **Metoda20** jako najszybszym (~10 sekund) i **Metoda18** jako najwolniejszym (~400 sekund).



Rys. 5.25: Execution Time SBERT z no shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono



Rys. 5.26: Execution Time SBERT z few-shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono



Rys. 5.27: Execution Time SBERT z one-shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono

Podsumowanie Autorskie modele **Metoda17**, **Metoda18**, **Metoda19**, **Metoda20** konsekwentnie osiągają krótsze czasy wykonania w porównaniu do większości zwykłych modeli, z **Metoda20** osiągającym najkrótszy czas (~10 sekund) w każdym podejściu. Wśród zwykłych modeli zauważalne są znaczące odstępstwa w czasie wykonania – szczególnie w podejściu **no-shot**, gdzie **Metoda9** (~4000 sekund) oraz **Metoda14** (~2000 sekund) wykazują dużą złożoność obliczeniową. Podejście **few-shot** i **one-shot** są bardziej stabilne pod względem czasu, jednak również posiadają odstające modele (**Metoda3**, **Metoda4**, **Metoda14**).

5.4. PORÓWNANIE WYNIKÓW NAJLEPSZYCH BAZOWYCH ROZWIĄZAŃ

5.4.1. Proponowane testy statystyczne

Test Shapiro-Wilka pozwala ocenić, czy dane w kolumnach numerycznych mają rozkład normalny. Jest to szczególnie istotne, ponieważ wiele testów statystycznych zakłada normalność danych, a sam **test Shapiro-Wilka** jest skuteczny zwłaszcza przy małych próbkach.

Test Kruskala-Wallisa umożliwia porównanie median pomiędzy grupami, co pozwala określić, czy różnice między nimi są istotne statystycznie. Jego dużą zaletą jest brak założenia normalności rozkładu, dzięki czemu nadaje się do pracy z danymi o nieznanym lub nienormalnym rozkładzie.

Korelacja Pearsona służy do pomiaru **liniowej zależności** pomiędzy dwiema zmiennymi liczbowymi. Jest przydatna w identyfikacji silnych relacji liniowych, które mogą wskazywać na potencjalne zależności przyczynowo-skutkowe w danych.

Z kolei **korelacja Spearmana** ocenia **monotoniczną zależność** między zmiennymi, co sprawdza się w przypadku zależności nieliniowych lub danych porządkowych z wartościami odstającymi.

Histogramy wizualizują rozkład danych w kolumnach numerycznych, co pozwala na szybką ocenę **kształtu rozkładu**, **asymetrii** oraz **obecności wartości odstających**.

5.4.2. Wybrane najlepsze bazowe metody

W analizie wyników wyraźnie wyróżnia się **Metoda10**, który osiągnął najniższy Log Loss w podejściu few-shot (0.4), co świadczy o najlepszym dopasowaniu modelu w tej grupie.

Kolejnym istotnym modelem jest **Metoda1**, który charakteryzował się stabilnymi wartościami Log Loss poniżej 0.6 w podejściu one-shot, potwierdzając swoją skuteczność.

W przypadku **Metoda9** odnotowano stabilny Log Loss (0.6) oraz dobrą skuteczność w podejściu one-shot, co czyni go godnym uwagi.

Na szczególną uwagę zasługuje również **Metoda5**, który mimo słabego wyniku w metryce Log Loss w niektórych podejściach, osiągnął bardzo dobry wynik w Accuracy (0.9) w podejściu few-shot, pokazując swoje możliwości.

Dodatkowo **Metoda6** wykazał się stabilnością oraz krótkimi czasami wykonania zarówno w podejściu no-shot, jak i few-shot, co czyni go jednym z efektywniejszych modeli pod względem wydajności.

Wreszcie, **Metoda13** charakteryzował się krótkimi czasami wykonania oraz stabilnymi wynikami w większości podejść, co potwierdza jego dobrą optymalizację i skuteczność.

5.4.3. Reprezentacja wszystkich danych statystycznych

Accuracy	Log Loss	Execution Time (s)	Metoda
1	0.026802889	37.7056427	Metoda10_results.csv
0.8	0.45171819	0.723993063	Metoda13_results.csv
1	0.027981209	74.19449925	Metoda17_results.csv
1	0.002108043	543.5069363	Metoda18_results.csv
1	0.027397671	162.6634991	Metoda19_results.csv
0.5	0.504755825	22.14533353	Metoda1_results.csv
0.9	0.624626705	14.41849785	Metoda20_results.csv
0.8	0.79800817	106.7345865	Metoda5_results.csv
0.9	0.335556619	96.85355759	Metoda6_results.csv
1	0.05420126	5254.920101	Metoda9_results.csv

Tabela 5.2: Porównanie wyników eksperymentalnych

metric	test	mean	std	p_value	normal_distribution
Log Loss	Shapiro-Wilk	0.285321123	0.296203627	0.057119309	TRUE

Tabela 5.3: Wyniki testu Shapiro-Wilka dla metryki Log Loss

Test Shapiro-Wilka

Metryka **Log Loss** wykazuje zgodność z rozkładem normalnym na podstawie wyniku testu Shapiro-Wilka. Średnia wartość Log Loss wynosi **0.2853**, a odchylenie standardowe to **0.2962**. Wartość p równa **0.0571** jest wyższa od poziomu istotności 0.05, co wskazuje, że dane mogą być analizowane za pomocą metod parametrycznych.

Test Kruskala-Wallisa

metric	test	stat	p_value	significant
Accuracy	Kruskal-Wallis	9	0.437274189	FALSE
Log Loss	Kruskal-Wallis	9	0.437274189	FALSE
Execution Time (s)	Kruskal-Wallis	9	0.437274189	FALSE

Tabela 5.4: Wyniki testu Kruskala-Wallisa dla różnych metryk

Test Kruskala-Wallisa jest używany do porównania więcej niż dwóch grup pod względem median, zakładając, że dane niekoniecznie pochodzą z rozkładu normalnego. Możemy zauważyć, że dla każdej z analizowanych metryk wartość **p-value** jest większa od typowego poziomu istotności (**0.05**). To prowadzi nas do wniosku, że:

- Nie ma dowodów na istotne różnice w dokładności modeli pomiędzy grupami.
- Różnice w wartościach funkcji strat logarytmicznych również nie są istotne.
- Czas wykonania różni się między eksperymentami, jednak różnice te nie są statystycznie znaczące.

Jakie wnioski można wyciągnąć?

1. Accuracy:

- Możemy zauważyć, że dokładność modeli osiąga wartość maksymalną w wielu przypadkach. To może sugerować, że użyte algorytmy są dobrze dostosowane do danych. Jednak brak istotności statystycznej wskazuje, że różnice między eksperymentami nie są znaczące.
- W praktyce oznacza to, że wybór konkretnego modelu spośród analizowanych eksperymentów prawdopodobnie nie wpłynie znacząco na osiąganą dokładność.

2. Log Loss:

- Funkcja strat logarytmicznych charakteryzuje się mniejszymi wartościami w lepszych modelach. Widzimy, że wartości te różnią się między eksperymentami, ale test Kruskala-Wallisa nie wskazuje na istotne różnice.

- Możemy przypuszczać, że wszystkie analizowane modele mają porównywalną zdolność do przewidywania prawdopodobieństw.

3. **Execution Time:**

- Analizując czas wykonania, widzimy znaczne różnice między eksperymentami. Przykładowo, w jednym przypadku czas wynosi zaledwie 0.7 sekundy, a w innym przekracza **543 sekund**. Pomimo tego, test statystyczny nie wykazuje istotności tych różnic.
- Może to wynikać z faktu, że dla testu Kruskala-Wallisa liczy się rozkład wartości, a nie same ekstremalne różnice.

Co jeszcze możemy zrobić?

Bazując na wynikach, możemy rozważyć kilka dalszych kroków:

1. **Szczegółowa analiza danych:**

- Sprawdzenie, czy wyniki nie są zakłócane przez potencjalne błędy w danych wejściowych lub metodologii.
- Analiza rozkładów wartości każdej metryki w grupach, co może pomóc lepiej zrozumieć brak istotności statystycznej.

2. **Dalsze testy statystyczne:**

- Jeśli chcemy potwierdzić te wyniki, możemy przeprowadzić inne testy, takie jak ANOVA lub test U Manna-Whitneya, które mogą lepiej uwzględniać różnice między parami grup.

3. **Optymalizacja czasowa:**

- W przypadku **Execution Time**, warto zastanowić się nad optymalizacją algorytmów, które wymagają najdłuższego czasu obliczeń.

Podsumowanie

Widzimy, że przeprowadzone analizy pozwalają na wyciągnięcie kilku istotnych wniosków:

- Brak istotnych różnic w wynikach testu sugeruje, że użyte algorytmy mają podobną skuteczność w każdej z analizowanych metryk.
- Znaczne różnice w czasie wykonania wymagają dodatkowej analizy, szczególnie jeśli celem jest zwiększenie efektywności obliczeniowej.
- Na podstawie przedstawionych wyników możemy uznać, że test Kruskala-Wallisa nie wykazał różnic statystycznie istotnych, co oznacza, że różnorodność wyników nie wpływa znacząco na ogólną jakość modeli w badanej grupie.

Korelacja Pearsona

Na pierwszy rzut oka możemy zauważyć znaczne zróżnicowanie między wynikami poszczególnych eksperymentów. Na przykład:

	Accuracy	Log Loss	Execution Time (s)
Accuracy	1	0.669047872	0.27774502
Log Loss	0.669047872	1	-0.314798004
Execution Time (s)	0.27774502	0.314798004	1

Tabela 5.5: Macierz korelacji między Accuracy, Log Loss i Execution Time

- Dokładność (Accuracy) waha się między 80% a 100%.
- Log Loss zmienia się od wartości bardzo niskich, bliskich zera, do znacznie wyższych.
- Czas wykonania różni się diametralnie, od niecałej sekundy do ponad 500 sekund.

Te różnice wskazują, że w eksperymentach mogły być testowane różne modele, zestawy danych lub konfiguracje hiperparametrów.

Jakie zależności możemy zauważyć?

Po obliczeniu korelacji Pearsona między zmiennymi, uzyskaliśmy macierz korelacji zapisaną w pliku `pearson_correlation_matrix.csv`. Macierz ta ukazuje siłę i kierunek relacji między trzema głównymi zmiennymi:

1. Accuracy (dokładność),
2. Log Loss (strata logarytmiczna),
3. Execution Time (czas wykonania).

Wartości korelacji prezentują się następująco:

- Korelacja między **Accuracy** a **Log Loss** wynosi **-0.669**.
- Korelacja między **Execution Time** a **Accuracy** wynosi **0.278**.
- Korelacja między **Execution Time** a **Log Loss** wynosi **-0.315**.

Co możemy wywnioskować na podstawie korelacji?

1. Accuracy i Log Loss:

- Zauważamy umiarkowanie silną ujemną korelację między dokładnością a funkcją straty logarytmicznej. W praktyce oznacza to, że im lepsza dokładność modelu, tym niższa wartość Log Loss. Jest to zgodne z intuicją, ponieważ dokładne modele generują bardziej precyzyjne przewidywania, co prowadzi do mniejszego błędu.
- Warto podkreślić, że zależność ta jest kluczowa dla oceny jakości modeli. Wysoka dokładność przy jednocześnie niskiej wartości Log Loss oznacza, że model radzi sobie dobrze nie tylko w klasyfikacji, ale również w przypisywaniu poprawnych prawdopodobieństw.

2. Accuracy i Execution Time:

- Korelacja między dokładnością a czasem wykonania wynosi **0.278**, co sugeruje słabą dodatnią zależność. Możemy zauważyć, że dłuższy czas wykonania modelu często wiąże się z nieznacznie lepszą dokładnością. Jest to zrozumiałe, gdyż bardziej

złożone modele, które wymagają więcej czasu obliczeniowego, często oferują wyższą dokładność.

- Warto jednak zwrócić uwagę na koszt obliczeniowy – długie czasy wykonania mogą być problematyczne w praktycznych zastosowaniach, gdzie istotna jest szybkość działania modelu.

3. Execution Time i Log Loss:

- Korelacja między czasem wykonania a Log Loss wynosi **-0.315**, co wskazuje na słabą ujemną zależność. Możemy zauważyć, że modele, które działają dłużej, mają tendencję do osiągania mniejszych wartości Log Loss. To ponownie sugeruje, że bardziej złożone algorytmy, choć czasochłonne, mogą generować bardziej precyzyjne przewidywania.

Jak możemy wykorzystać te wyniki?

Dane te dostarczają cennych informacji, które mogą być wykorzystane do optymalizacji procesu modelowania:

- **Zrozumienie kompromisu między dokładnością a czasem wykonania:** Jeśli czas obliczeń jest ograniczeniem, można zdecydować się na mniej złożone modele, które oferują akceptowalny poziom dokładności przy krótszym czasie wykonania.
- **Monitorowanie jakości predykcji:** Dzięki relacji między Accuracy a Log Loss możemy lepiej ocenić, czy poprawa jednego wskaźnika wpływa pozytywnie na drugi.
- **Planowanie zasobów obliczeniowych:** Słaba korelacja między Execution Time a Log Loss wskazuje, że istnieją sposoby na skrócenie czasu działania bez znaczącej utraty jakości predykcji.

Podsumowanie

Wyniki korelacji Pearsona pokazują, że dokładność modeli, ich funkcja straty logarytmicznej oraz czas wykonania są ze sobą powiązane w różnym stopniu. Widoczne zależności sugerują, że wybór modelu zawsze będzie wymagał kompromisu między jakością predykcji a kosztami obliczeniowymi. Warto wykorzystać te informacje do dalszej optymalizacji i tworzenia bardziej efektywnych modeli, szczególnie w kontekście zastosowań wymagających równowagi między szybkością działania a dokładnością.

Korelacja Spearmana

Dzięki analizie danych z macierzy korelacji Spearmana możemy zauważyć kilka istotnych zależności między cechami, które opisują wyniki działania modeli. Nasza analiza obejmuje trzy kluczowe metryki: **dokładność (Accuracy)**, **stratę logarytmiczną (Log Loss)** oraz **czas wykonania (Execution Time)**. Interpretując wyniki korelacji, zauważamy konkretne wzorce, które pomagają lepiej zrozumieć, jak te zmienne wpływają na siebie nawzajem.

Możemy zauważyć...

1. **Silny negatywny związek między dokładnością a stratą logarytmiczną:** Wartość korelacji Spearmana wynosi -0.8398 , co oznacza, że dokładność rośnie wraz ze spadkiem straty logarytmicznej. Jest to intuicyjne, ponieważ modele lepiej przewidyujące wyniki mają niższe wartości funkcji kosztu. Na przykład model o dokładności 1.0 osiąga niską stratę logarytmiczną, co widzimy w danych wejściowych (np. 0.0268 w jednym przypadku).

Widzimy więc, że inwestowanie w optymalizację modeli pod kątem precyzyjności ma sens, gdy celem jest minimalizacja strat. Modele, które skuteczniej przewidują poprawne klasyfikacje, mają tendencję do minimalizowania błędów w swoich prognozach.

2. **Umiarkowany pozytywny związek między dokładnością a czasem wykonania:** Korelacja Spearmana wynosi 0.5794 . **Oznacza to**, że wyższa dokładność modeli często wiąże się z wydłużonym czasem ich działania. Na przykład modele, które osiągają dokładność 1.0, mają czasy wykonania sięgające nawet 543 sekund, podczas gdy mniej dokładne modele (np. dokładność 0.8) kończą się znacznie szybciej.

Możemy zatem zauważyć, że bardziej złożone modele, które wymagają większej liczby operacji obliczeniowych, częściej osiągają lepsze wyniki. Jednak warto zadać pytanie, czy czas wykonania jest akceptowalny w zależności od potrzeb danego projektu. W praktyce może być konieczne znalezienie kompromisu między czasem działania a jakością wyników.

3. **Słaby negatywny związek między stratą logarytmiczną a czasem wykonania:** Wartość korelacji wynosi -0.4424 , co sugeruje, że modele z niższą stratą logarytmiczną mogą wymagać więcej czasu na wykonanie. Chociaż związek ten nie jest bardzo silny, **widzimy pewną tendencję**, że bardziej dokładne modele z niską stratą potrzebują dodatkowych zasobów obliczeniowych.

Możemy zauważyć, że optymalizacja strat może być powiązana z wyższym kosztem czasowym. Dla użytkowników priorytetem może być określenie, czy minimalizacja strat jest wystarczająco ważna, by uzasadnić wydłużenie czasu działania.

Możemy interpretować...

Korelacje Spearmana pomagają nam lepiej zrozumieć wzorce w danych i pozwalają na głębszą analizę efektywności modeli. **Możemy interpretować wyniki** w kontekście trzech kluczowych wniosków:

1. **Dokładność jako najważniejszy cel:** Silny negatywny związek między dokładnością a stratą logarytmiczną sugeruje, że precyzyjne modele z małym błędem predykcji są najbardziej pożądane w naszym kontekście. Widzimy, że minimalizacja straty logarytmicznej idzie w parze z lepszymi wynikami, co potwierdza słuszność optymalizacji w tym kierunku.

2. **Koszt czasowy jako wyzwanie:** Modele o wysokiej dokładności wymagają dłuższego czasu wykonania. Możemy zauważyć, że istnieje potrzeba znalezienia kompromisu między szybkością działania a jakością wyników. W przypadku zastosowań czasu rzeczywistego skrócenie czasu działania może być kluczowe, nawet kosztem minimalnej utraty dokładności.
3. **Zależności między stratą a czasem działania:** Choć relacja między stratą logarytmiczną a czasem działania jest słabsza, widzimy, że modele, które minimalizują stratę, często wymagają dodatkowych zasobów obliczeniowych. W praktyce optymalizacja czasu działania może obejmować uproszczenie architektury modelu bez znaczącej utraty dokładności.

Możemy ulepszyć...

Aby efektywnie wykorzystać wyniki analizy, możemy wdrożyć kilka zmian i ulepszeń:

1. **Zoptymalizowanie algorytmów pod kątem czasu działania:** Wydaje się, że niektóre modele osiągają wysoką dokładność przy minimalnym koszcie czasowym. **Możemy zastosować** techniki takie jak redukcja liczby funkcji lub uproszczenie obliczeń, aby przyspieszyć modele bez większej utraty dokładności.
2. **Wyważenie dokładności i czasu:** Jeśli czas działania jest krytyczny, warto rozważyć modele o nieco niższej dokładności, ale krótszym czasie działania. **Możemy zaprojektować** algorytmy, które dynamicznie dostosowują złożoność w zależności od wymagań użytkownika.
3. **Wykorzystanie korelacji do selekcji modeli:** Wyniki Spearmana mogą służyć jako podstawa do automatycznej selekcji modeli. Na przykład modele o wysokiej dokładności i umiarkowanym czasie działania mogą być preferowane w sytuacjach, gdzie dokładność jest kluczowa, ale czas działania nie może być zbyt długi.

Podsumowując...

Test korelacji Spearmana pozwolił na odkrycie istotnych zależności między kluczowymi zmiennymi opisującymi wyniki modeli. Możemy zauważyć, że dokładność jest silnie powiązana z minimalizacją straty logarytmicznej oraz umiarkowanie zależna od czasu wykonania. Relacja między stratą a czasem działania jest słabsza, ale zauważalna.

Nasza analiza wskazuje, że optymalizacja modeli powinna uwzględniać te wzorce, aby zrównoważyć dokładność, czas działania i zasoby obliczeniowe. Możemy ulepszyć procesy optymalizacyjne, opierając się na korelacjach, by stworzyć bardziej efektywne i wydajne modele.

5.4.4. Analiza wyników autorskich rozwiązań w zależności od architektury

Analizując wyniki poszczególnych modeli, widzimy wyraźne różnice w ich skuteczności i wydajności, co wynika z zastosowanych architektur i metod trenowania. **Metoda17** opiera się na sieci neuronowej z dynamicznym ważeniem próbek. Dzięki temu model

skupia się na trudniejszych przykładach, co pozwala osiągnąć perfekcyjne wyniki w metrykach takich jak dokładność, precyzja, recall czy MCC. Mechanizm ważenia próbek zapewnia efektywne dopasowanie, zwłaszcza w przypadku zróżnicowanych danych. Jednak czas wykonania wynoszący 74 sekundy jest stosunkowo długi, a brak wyników walidacji krzyżowej uniemożliwia pełną ocenę zdolności modelu do generalizacji na nowych danych.

Metoda18 wykorzystuje Gradient Boosting i CatBoost, które są jednymi z najbardziej zaawansowanych modeli opartych na drzewach decyzyjnych, połączonych meta-modelem w postaci regresji logistycznej. Taka kombinacja pozwala na perfekcyjne dopasowanie i uzyskanie idealnych wyników we wszystkich metrykach, w tym w walidacji krzyżowej. To pokazuje, jak skuteczna jest synergia silnych modeli bazowych wspieranych meta-modelem. Jednak czas wykonania, wynoszący ponad 543 sekundy, jest największą wadą tego rozwiązania, co czyni je niepraktycznym w sytuacjach wymagających szybkiego działania.

Metoda19 łączy Random Forest i Extra Trees, również z wykorzystaniem meta-modelu. Ta architektura, podobnie jak w przypadku **Metoda18**, osiąga idealne wyniki we wszystkich metrykach i zapewnia stabilność dzięki zastosowaniu walidacji krzyżowej. Random Forest i Extra Trees są znane z efektywnego radzenia sobie z nieregularnymi danymi, co przyczynia się do ich wysokiej skuteczności. Czas wykonania wynoszący 162 sekundy jest jednak znacznie lepszy niż w **Metoda18**, choć wciąż ustępuje **Metoda20** pod względem wydajności.

Metoda20 prezentuje podejście, które łączy LightGBM i CatBoost jako modele bazowe. Choć te algorytmy są znane z wysokiej wydajności i elastyczności, ich wyniki są nieco gorsze w takich metrykach jak recall (0.8) i F1-score (0.89) w porównaniu z innymi modelami. Przyczyną tej różnicy może być specyfika działania obu algorytmów w kontekście zastosowanego zbioru danych oraz ich interakcji w ramach meta-modelu.

LightGBM, choć wyjątkowo szybki i wydajny, stosuje agresywną strategię dzielenia danych (leaf-wise growth), co może prowadzić do nadmiernego dopasowania na pewnych podzbiorach, a jednocześnie trudności w pełnym uchwyceniu bardziej subtelnych wzorców w danych. Ten problem może być szczególnie widoczny, gdy dane są bardzo zróżnicowane lub zawierają niewielką liczbę trudnych przypadków, co bezpośrednio wpływa na niższą wartość recall – model mógł "przegapić" część rzeczywistych przypadków pozytywnych.

Z kolei CatBoost, choć bardzo dobrze radzi sobie z danymi kategorycznymi i ma wbudowane mechanizmy minimalizujące nadmierne dopasowanie, wymaga większej liczby iteracji, aby w pełni wykorzystać swój potencjał w trudniejszych zadaniach klasyfikacyjnych. W przypadku **Metoda20** liczba iteracji mogła być zbyt niska, co ograniczyło zdolność modelu do pełnego dopasowania do danych. W efekcie oba modele bazowe mogły nie w pełni uzupełniać swoje braki, co przełożyło się na niższe wyniki w metrykach wymagających większej równowagi między precyzją a recall.

Dodatkowo, meta-model regresji logistycznej, choć skuteczny w łączeniu predykcji z obu algorytmów, mógł nie w pełni skorygować rozbieżności wyników między LightGBM

a CatBoost. W rezultacie, choć predykcje te były dobrze dopasowane w aspekcie ogólnych prawdopodobieństw (co widać w idealnym wyniku ROC-AUC), ich zastosowanie w bardziej szczegółowych miarach, takich jak F1-score, wykazało pewne ograniczenia.

PODSUMOWANIE

Wyniki badań wyraźnie wskazują na przewagę autorskich metod implementacji nad tradycyjnymi podejściami w zakresie kluczowych metryk, takich jak Log Loss, Accuracy i Execution Time. Przeprowadzona analiza jednoznacznie potwierdza, że zaproponowane algorytmy wyróżniają się zarówno pod względem skuteczności, jak i praktyczności, co czyni je szczególnie wartościowymi w kontekście rzeczywistych zastosowań.

Jednym z najbardziej imponujących rezultatów jest znaczące obniżenie wartości Log Loss, która odzwierciedla precyzję prognozowanych prawdopodobieństw. Na przykład metoda zastosowana w Metoda18 osiągnęła Log Loss wynoszący zaledwie 0.002108, co świadczy o niemal perfekcyjnym odwzorowaniu rzeczywistego rozkładu danych. W porównaniu z tradycyjnymi rozwiązaniami, gdzie wartości Log Loss często mieszczą się w przedziale 0.5–0.7, autorskie podejścia są o kilka rzędów wielkości bardziej precyzyjne.

Dokładność osiągnięta przez autorskie algorytmy również ustanawia nowy standard w ocenie modeli. Metoda18 i Metoda19 uzyskały idealną dokładność na poziomie 100%, co oznacza, że w żadnym przypadku nie popełniły błędu klasyfikacyjnego. Nawet Metoda20, z wynikiem wynoszącym 90%, przewyższa większość tradycyjnych metod, które w najlepszym przypadku osiągają około 70–80% dokładności. Taka skuteczność klasyfikacji sprawia, że zaproponowane rozwiązania są szczególnie przydatne w praktycznych zastosowaniach, gdzie wysoka precyzja decyzji jest kluczowa.

Nie można również pominąć wydajności obliczeniowej, która czyni autorskie metody wyjątkowo praktycznymi w rzeczywistych zastosowaniach. Metoda20 zakończył obliczenia w czasie zaledwie 14.4185 sekundy, co jest wynikiem niespotykanym wśród tradycyjnych rozwiązań. Nawet bardziej złożone modele, jak Metoda18 czy Metoda19, znacznie przewyższają standardowe metody pod względem optymalizacji czasu. Takie osiągnięcia mają ogromne znaczenie w zastosowaniach wymagających przetwarzania dużych zbiorów danych w krótkim czasie.

Autorskie metody implementacji wyróżniają się na tle tradycyjnych rozwiązań dzięki ich wysokiej skuteczności, precyzji i wydajności. Wyniki podkreślają ich wyjątkową efektywność w zróżnicowanych scenariuszach zastosowań. Ich wszechstronność sprawia, że mogą być z powodzeniem stosowane w dziedzinie do walki z fake news.

BIBLIOGRAFIA

- [1] Adedoyin S. Adeyemi, Sunday O. Asakpa, Kehinde H. Bello, Olajide N. Saliu, Olabisi M. Olaoye, and Nike T. Toye. Fake news detection using ensemble methods: an empirical evaluation of bagging and boosting algorithms. *International Journal of Advances in Engineering and Management (IJAEM)*, 6(06):317–325, 2024. Uzyskano dostęp: 23 Stycznia 2025.
- [2] Iftikhar Ahmad, Muhammad Yousaf, Suhail Yousaf, and Muhammad Ovais Ahmad. Fake news detection using machine learning ensemble methods. *Complexity*, 2020:1–11, 10 2020. Uzyskano dostęp: 23 Stycznia 2025.
- [3] Abdullah Marish Ali, Fuad A. Ghaleb, Bander Ali Saleh Al-Rimy, Fawaz Jaber Alsolami, and Asif Irshad Khan. Deep ensemble fake news detection model using sequential deep learning technique. *Sensors*, 22(18), 2022. Uzyskano dostęp: 23 Stycznia 2025.
- [4] Esma Aïmeur, Sabrine Amri, and Gilles Brassard. Fake news, disinformation and misinformation in social media: a review. *Social Network Analysis and Mining*, 13(1):30, 2023. Uzyskano dostęp: 23 Stycznia 2025.
- [5] M.A. Ganaie, Minghui Hu, A.K. Malik, M. Tanveer, and P.N. Suganthan. Ensemble deep learning: A review. *Engineering Applications of Artificial Intelligence*, 115:105151, 2022. Uzyskano dostęp: 23 Stycznia 2025.
- [6] Alireza Ghorbanali, Mohammad Karim Sohrabi, and Farzin Yaghmaee. Ensemble transfer learning-based multimodal sentiment analysis using weighted convolutional neural networks. *Information Processing & Management*, 59(3):102929, 2022. Uzyskano dostęp: 23 Stycznia 2025.
- [7] Saqib Hakak, Mamoun Alazab, Suleman Khan, Thippa Reddy Gadekallu, Praveen Kumar Reddy Maddikunta, and Wazir Zada Khan. An ensemble machine learning approach through effective feature extraction to classify fake news. *Future Generation Computer Systems*, 117:47–58, 2021. Uzyskano dostęp: 23 Stycznia 2025.
- [8] Azal Ahmad Khan, Omkar Chaudhari, and Rohitash Chandra. A review of ensemble learning and data augmentation models for class imbalanced problems: Combination, implementation and evaluation. *Expert Systems with Applications*, 244:122778, 2024. Uzyskano dostęp: 23 Stycznia 2025.
- [9] Junaed Younus Khan, Md. Tawkat Islam Khondaker, Sadia Afroz, Gias Uddin, and Anindya Iqbal. A benchmark study of machine learning models for online fake news detection. *Machine Learning with Applications*, 4:100032, 2021. Uzyskano dostęp: 23 Stycznia 2025.
- [10] Atik Mahabub. A robust technique of fake news detection using ensemble voting classifier and comparison with other classifiers. *SN Applied Sciences*, 2(4):525, 2020. Uzyskano dostęp: 23 Stycznia 2025.
- [11] Parth Patel, Shubham Patel, and Yashkumar Patel. Ensemble based approach for fake news de-

- tection. *International Research Journal of Engineering and Technology (IRJET)*, 08(05):2732, 2021. Uzyskano dostęp: 23 Stycznia 2025.
- [12] Shafin Rahman, Salman Khan, and Fatih Porikli. A unified approach for conventional zero-shot, generalized zero-shot, and few-shot learning. *IEEE Transactions on Image Processing*, PP, 06 2017. Uzyskano dostęp: 23 Marca 2025.
- [13] Amit Neil Ramkissoon and Wayne Goodridge. Enhancing the predictive performance of credibility-based fake news detection using ensemble learning. *The Review of Socionetwork Strategies*, 16(2):259–289, 2022. Uzyskano dostęp: 23 Stycznia 2025.
- [14] Harita Reddy, Namratha Raj, Manali Gala, and Annappa Basava. Text-mining-based fake news detection using ensemble methods. *Machine Intelligence Research*, 17(2):210–221, 2020. Uzyskano dostęp: 23 Stycznia 2025.
- [15] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. *CoRR*, abs/1908.10084, 2019. Uzyskano dostęp: 23 Marca 2025.
- [16] Arjun Roy, Kingshuk Basak, Asif Ekbal, and Pushpak Bhattacharyya. A deep ensemble framework for fake news detection and classification, 2018. Uzyskano dostęp: 23 Stycznia 2025.
- [17] K V, B B Al-onazi, V Simic, E B Tirkolaee, and C Jana. Deepfnd: An ensemble-based deep learning approach for the optimization and improvement of fake news detection in digital platform. *PeerJ Computer Science*, 9:e1666, 2023. Uzyskano dostęp: 23 Stycznia 2025.
- [18] Pawan Verma, Prateek Agrawal, Ivone Amorim, and Radu Prodan. Welfake: Word embedding over linguistic features for fake news detection. *IEEE Transactions on Computational Social Systems*, PP:1–13, 04 2021. Uzyskano dostęp: 23 Stycznia 2025.

SPIS RYSUNKÓW

2.1	Schemat metody 15.	21
2.2	Schemat metody 2.	22
2.3	Schemat metody 3.	24
2.4	Schemat metody 5.	26
3.1	Schemat metody 17.	40
3.2	Schemat metody 18.	44
3.3	Schemat metody 19.	48
3.4	Schemat metody 20.	52
4.1	Przykładowe odpalenie aplikacji.	57
5.1	Log Loss Bert z no shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono	63
5.2	Log Loss Bert z few shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono	63
5.3	Log Loss Bert z one shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono	64
5.4	Accuracy Bert z no shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono	65
5.5	Accuracy Bert z few shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono	65
5.6	Accuracy Bert z one shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono	66
5.7	Execution Time Bert z no shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono	67
5.8	Execution Time Bert z few-shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono	68
5.9	Execution Time Bert z one-shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono	68
5.10	Log Loss RoBERTa z no shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono	70
5.11	Log Loss RoBERTa z few-shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono	70
5.12	Log Loss RoBERTa z one-shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono	71

5.13	Accuracy RoBERTa z no shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono	72
5.14	Accuracy RoBERTa z few-shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono	72
5.15	Accuracy RoBERTa z one-shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono	73
5.16	Execution Time RoBERTa z one-shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono	74
5.17	Execution Time RoBERTa z few-shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono	74
5.18	Execution Time RoBERTa z one-shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono	75
5.19	Log Loss SBERT z no shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono	76
5.20	Log Loss SBERT z few-shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono	76
5.21	Log Loss SBERT z one-shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono	77
5.22	Accuracy SBERT z no shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono	78
5.23	Accuracy SBERT z few-shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono	78
5.24	Accuracy SBERT z one-shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono	79
5.25	Execution Time SBERT z no shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono	80
5.26	Execution Time SBERT z few-shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono	80
5.27	Execution Time SBERT z one-shot wraz z autorskimi metodami, które zostały zaznaczone na czerwono	81

Dodatki